

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

UNIVERSITY OF INFORMATION TECHNOLOGY

FACULTY OF INFORMATION SYSTEMS



REPORT

INFORMATION SYSTEM ANALYSIS AND DESIGN

TOPIC:

Apartment management application

Instructor:

Instructor: Cao Thi Nhan

Instructor: Duong Phi Long

Member:

1	Ho Tan Loc	22520786
2	Ngo Thanh Trung	22521561
3	Pham Duy Tuan	22521609
4	Tang Kim Son	22521258

HO CHI MINH CITY, June 2025

ASSIGNMENT AND MEMBER EVALUATION TABLE

I. Team Member Roles and Responsibilities

No.	Full Name	Student ID	Assigned Role(s)	Key Responsibilities
1	Ho Tan Loc	22520786	Project Leader, System Analyst, Database Designer Sub- Backend Developer	Overseeing project progress, coordinating team efforts, facilitating communication with stakeholders, leading requirement analysis (e.g., stakeholder interviews, survey design for residents and management), defining functional/non-functional requirements, developing DFDs and ERDs.
2	Ngo Thanh Trung	22521561	Backend Developer	Designing the database schema (ERD, RDM), implementing the database (PostgreSQL), developing data access layer (Repositories and DbContext), implementing business logic layer (Services), and handling multi-schema mechanism.
3	Pham Duy Tuan	22521609	UI/UX Designer,	Designing and implementing user interfaces (XAML/WPF

			Frontend Developer, Sub-Backend Developer	Views), developing ViewModels, ensuring data binding and command implementation (MVVM pattern), and enhancing user experience.
4	Tang Kim Son	22521258	Tester, Documentation Preparing	Developing test cases, performing functional and non-functional testing, documenting test results, assisting with report writing and formatting, and compiling project documentation.

II. Member Contribution and Evaluation

No.	Full Name	Student ID	Self-Assessment (Contribution %)	Peer Assessment (Average %)	Remarks / Specific Contributions
1	Ho Tan Loc	22520786			
2	Ngo Thanh Trung	22521561			
3	Pham Duy Tuan	22521609			
4	Tang Kim Son	22521258			

TABLE OF CONTENTS

CHAPTER 1. GENERAL INTRODUCTION	14
1.1 Overview of the project	14
1.2 Problems of the project	14
1.3 Objectives of the project	15
1.4 Scope of project	16
CHAPTER 2. REQUIREMENT SPECIFICATION	17
2.1 Survey of the current situation	17
2.1.1 Stakeholder Analysis	17
2.1.2 Interview	24
2.1.3 Conclusion of Stakeholder Surveys	31
2.1.4 Survey of the organization/ Current organizational situation	34
2.1.5 Survey of business processes, major activities	38
2.1.6 Survey of the existing system	44
2.2 Software requirements	45
2.2.1 Functional requirements	45
2.2.2 Non-functional requirements	52
2.2.3 Report Forms	Error! Bookmark not defined.
CHAPTER 3. ANALYSIS AND DESIGN	54
3.1 Business Function Diagram – BFD	54
3.1.1 Level 0: Overall Business Function	Error! Bookmark not defined.
3.1.2 Level 1: Major Business Functions (Primary Domains/Lifecycle Stages)	Error! Bookmark not defined.

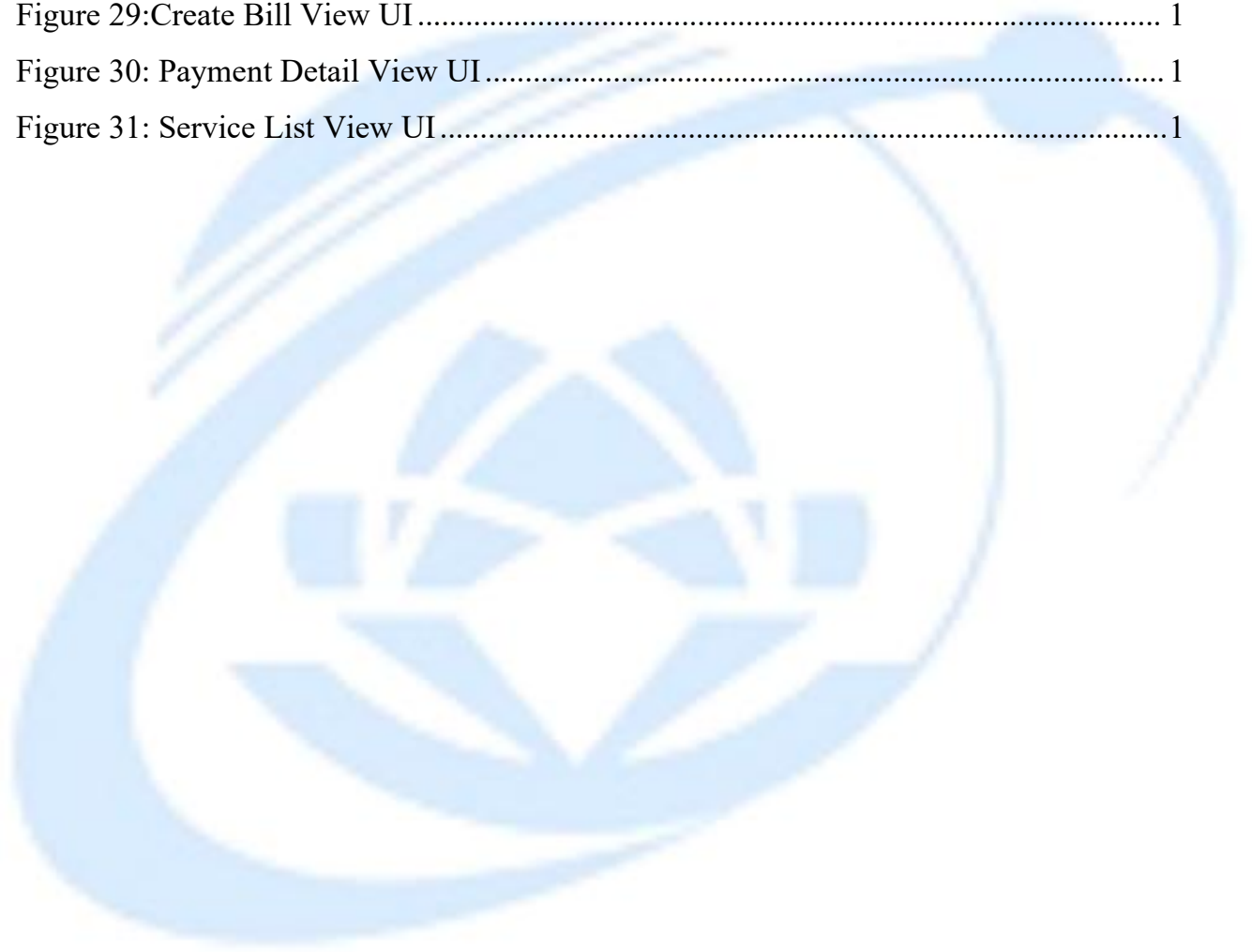
3.1.3	Level 2: Detailed Business Functions (Specific Activities)	Error!
	Bookmark not defined.	
3.2	Data Flow Diagram – DFD	58
3.2.1	Context Diagram	Error! Bookmark not defined.
3.2.2	Level-0 DFD	Error! Bookmark not defined.
3.2.3	Level-1 DFD	Error! Bookmark not defined.
3.2.4	Level-2 DFD	Error! Bookmark not defined.
3.3	Entity Relational Data model – ERD	58
3.4	Relational Data model – RDM	92
3.5	System Architecture Design	93
3.5.1	3-Tier Architecture	94
3.5.2	MVVM (Model-View-ViewModel) Model	96
3.5.3	Multi-Tenant Schema Mechanism	98
3.5.4	Overall System Architecture Diagram	100
CHAPTER 4.	IMPLEMENTATION AND TESTING	102
4.1	Introduction to the implementation environment	102
4.1.1	Programming Language	102
4.1.2	User Interface Framework	103
4.1.3	Database Management System	103
4.1.4	Object-Relational Mapping (ORM)	103
4.1.5	Configuration Management	104
4.1.6	Development Environment	104
4.2	User Interface – UI	104

4.2.1	Design Philosophy and Usability Paradigm	104
4.2.2	Modular UI Architecture and Key Components	105
4.2.3	Resident Management Module	112
4.2.4	Billing and Payment Management Module	115
4.2.5	Service Request Management Module	117
4.3	Database Design	118
4.3.1	Relational Data Model (RDM) Overview	118
4.4	Test Cases	121
CHAPTER 5.	CONCLUSION	122
5.1	Evaluation of Achieved Results	122
5.2	Project Limitations	123
5.3	Future Development Directions	125
5.4	Team's Lessons Learned	126

LIST OF FIGURES

Figure 1: Power/Interest Grid	1
Figure 2: Resident Survey Form	1
Figure 3: Resident Survey Question	1
Figure 4: Current Organizational Chart of BCONS Management Board	1
Figure 5: Current Information Flow and Inter-Departmental Interactions in BCONS Management Board	1
Figure 6: BFD - level 1	Error! Bookmark not defined.
Figure 7: BFD - level 2 - Apartment Lifecycle Management	Error! Bookmark not defined.
Figure 8: BFD - level 2 - Resident Lifecycle Management	Error! Bookmark not defined.
Figure 9: BFD - level 2 - Financial Transaction Management	Error! Bookmark not defined.
Figure 10: BFD - level 2 - Service & Support Operations	Error! Bookmark not defined.
Figure 11: BFD - level 2 - Reporting & Analytics	Error! Bookmark not defined.
Figure 12: DFD Context Diagram	Error! Bookmark not defined.
Figure 13: DFD Level 1 Diagram	Error! Bookmark not defined.
Figure 14: 3-Tier Architecture Diagram	1
Figure 15: MVVM Model Diagram	1
Figure 16: Multi-Tenant Schema Mechanism Diagram	1
Figure 17: Overall System Architecture Diagram	1
Figure 18: Dashboard UI 1	1
Figure 19: Dashboard UI 2	1
Figure 20: Apartmnet List View UI	1
Figure 21: Apartmnet List View UI - Search Function	1
Figure 22: Create Apartment View UI	1

Figure 23: Apartment List View UI - Filter Function	1
Figure 24: Apartment Edit View UI	1
Figure 25: Resident List View UI	1
Figure 26: Resident Info Detail View	1
Figure 27: Create Resident View UI	1
Figure 28: Payment List View UI	1
Figure 29: Create Bill View UI	1
Figure 30: Payment Detail View UI	1
Figure 31: Service List View UI	1



LIST OF TABLES

Table 1: Stakeholder Matrix	1
Table 2: Management Interview Records	1



LIST OF ABBREVIATIONS



PROJECT SUMMARY

This report details the analysis, design, and implementation of the **Apartment Management System**, a C# WPF desktop application. Developed using Entity Framework Core and a PostgreSQL database, its primary goal is to streamline administrative and operational tasks for apartment complexes, addressing inefficiencies in manual or fragmented management processes.

Key functional outcomes include:

- **Comprehensive Management:** Effective management of apartments (status, transfer), residents (details, relationships), and service requests.
- **Financial Tracking:** Robust billing and payment processing with status tracking (pending, completed, overdue).
- **Operational Insights:** A centralized dashboard provides real-time KPIs on apartments, residents, revenue, and service requests for informed decision-making.

The project followed a structured development process, from detailed requirement elicitation via stakeholder interviews and surveys to design using BFD, DFD, ERD, and RDM. This system significantly improves operational efficiency, reduces errors, and enhances service quality. Future work will focus on automating billing, integrating payment gateways, and developing a resident portal.

ACKNOWLEDGEMENTS

We extend our deepest gratitude to our instructors, **Cao Thi Nhan** and **Duong Phi Long**, for their invaluable guidance, unwavering support, and constructive feedback throughout the entire duration of the System Analysis and Design course and the subsequent development of this project. Their expertise and dedication were instrumental in shaping our understanding of system analysis principles and in successfully navigating the complexities of software development. Furthermore, we are thankful for all the resources, tools, and libraries that provided essential support for our work, enabling us to implement our designs effectively. Lastly, we appreciate the collaborative spirit, hard work, and commitment demonstrated by every member of our team, which were crucial in bringing this Apartment Management System project to a successful completion.

CHAPTER 1. GENERAL INTRODUCTION

1.1 Overview of the project

This project focuses on the development of an **Apartment Management System**, a robust desktop application crafted using C# with WPF (Windows Presentation Foundation) for its user interface, backed by Entity Framework Core for data interactions and a PostgreSQL database for persistent storage. The overarching goal is to address and overcome the inherent challenges associated with traditional, often manual or fragmented, apartment management processes. These challenges typically include inefficiencies in tracking apartment statuses, managing resident information, handling complex billing and payment procedures, and processing various service requests. By automating these core functions and centralizing data, the system aims to significantly enhance the operational efficiency of apartment complex management, reduce manual errors, and ultimately improve the overall service quality provided to residents. Furthermore, the system incorporates a forward-thinking design, emphasizing scalability and maintainability, ensuring it can adapt to future needs and varying scales of operation. The project was approached through a structured system analysis and design methodology, moving from detailed requirement elicitation to comprehensive architectural planning, culminating in a functional and user-friendly application.

1.2 Problems of the project

The development of the Apartment Management System is driven by a critical need to address significant inefficiencies and challenges inherent in current apartment complex management practices. Predominantly, existing methods often rely on fragmented manual processes, extensive use of disparate spreadsheets, or outdated software solutions, leading to considerable operational hurdles. Key among these is the pervasive issue of **decentralized and inconsistent information management**, where data regarding apartments, residents, billing, and service requests is scattered across

various departments or stored in incompatible formats. This fragmentation results in difficulties with accurate data retrieval, frequent inconsistencies, and a higher propensity for human error, directly impacting operational efficiency and decision-making accuracy. Furthermore, the **lack of streamlined financial tracking** poses a substantial problem, making it challenging to effectively monitor bill issuance, payment statuses, and outstanding debts, which can lead to revenue leakage and resident dissatisfaction. Lastly, the **absence of a centralized and efficient channel for managing resident service requests** often results in delayed responses, poor tracking of issue resolution, and a diminished resident experience. These interconnected problems collectively highlight the urgent need for a unified, automated, and comprehensive system to centralize information, optimize workflows, and enhance the overall management and living experience within apartment communities.

1.3 Objectives of the project

The primary objective of this project is to develop and implement a comprehensive Apartment Management System that directly addresses the identified challenges within current management practices. Specifically, the system aims to centralize and digitize all critical information pertaining to apartments, residents, billing, and service requests, thereby eliminating data fragmentation and enhancing accessibility. A core objective is to streamline the financial management process by enabling efficient bill generation, accurate payment tracking, and real-time monitoring of outstanding dues. Furthermore, the project seeks to establish an intuitive and effective channel for residents to submit and track service requests, ensuring timely resolution and improved communication. By integrating these functionalities within a unified platform, the system is designed to significantly boost the operational efficiency of the management team, minimize human error, and ultimately elevate the overall living experience for residents. The strategic aim is to transform fragmented manual processes into a cohesive,

automated, and insightful management framework, providing clear visibility into key operational metrics through a centralized dashboard.

1.4 Scope of project

The Apartment Management System is designed to encompass core functionalities essential for the efficient administration of an apartment complex. Its primary scope includes comprehensive management of apartment details, such as status (vacant/occupied) and transfer information, along with robust management of resident data, including personal details, occupancy, and owner/member relationships. The system also provides integrated modules for billing and payment tracking, allowing for the management of various bill types and payment statuses, alongside a structured approach to handling resident service requests. A key feature within its scope is the dashboard, offering an overview of vital operational metrics for management oversight. Furthermore, the system incorporates user authentication and role-based access, with a unique multi-tenant schema mechanism supporting multiple buildings within a single database instance. However, this study operates within defined limitations; the system does not include advanced accounting features, direct integration with external systems like online payment gateways or security systems, nor does it provide a resident-facing web or mobile interface. The current implementation is confined to a standalone desktop application.

CHAPTER 2. REQUIREMENT SPECIFICATION

2.1 Survey of the current situation

2.1.1 Stakeholder Analysis

To effectively analyze and manage the diverse interests and influences surrounding the Apartment Management System project, a comprehensive stakeholder analysis was conducted. This process involved identifying all key individuals and groups affected by or having an impact on the project, assessing their level of interest and influence, and understanding their specific needs and concerns.

2.1.1.1 Stakeholder Matrix

This matrix provides a structured overview of identified stakeholders, outlining their engagement levels, key expectations, current challenges, and the optimal communication strategies.

Table 1: Stakeholder Matrix

Stakeholder	Interest Level	Influence Level	Key Needs from the System	Current Problem	Communication Strategy
Management Team	High	High	Efficient management of apartment, resident, financial, and service information; Accurate and timely reports; Automation of	Difficulty tracking outstanding payments, time-consuming service request processing	In-depth interviews, regular meetings.

			routine tasks.	g, data inconsistency, lack of comprehensive overview.	
Residents	High	Medium	Easy bill payment; Convenient service request submission and tracking; Timely notifications; Access to personal and apartment information.	Difficulty in obtaining accurate information for repairs, manual tracking of service requests.	Online surveys, selective interviews.
Technical Staff	Medium	Medium	Easy-to-use and maintain system; Clear documentation; Efficient assignment and tracking of maintenance tasks.	Potential for project delays, unexpected costs, system not meeting	Interviews, workshops.

				strategic goals.	
Project Sponsor	Medium	High	Project completion within budget and schedule; System's contribution to overall operational efficiency and resident satisfaction; Return on investment.	Manual coordination for service requests.	Executive summaries, key progress reports, strategic meetings.
External Service Providers	Medium	Low	Clear communication regarding service needs (e.g., specific repair requests); Timely payment for services.		Standardized communication channels (e.g., email, system notifications for tasks).

2.1.1.2 Power/Interest Grid

The Power/Interest Grid is utilized to visually map stakeholders based on their level of power (ability to impact the project) and interest (degree to which they are affected by the project). This mapping guides the strategic approach to managing each stakeholder group's engagement.

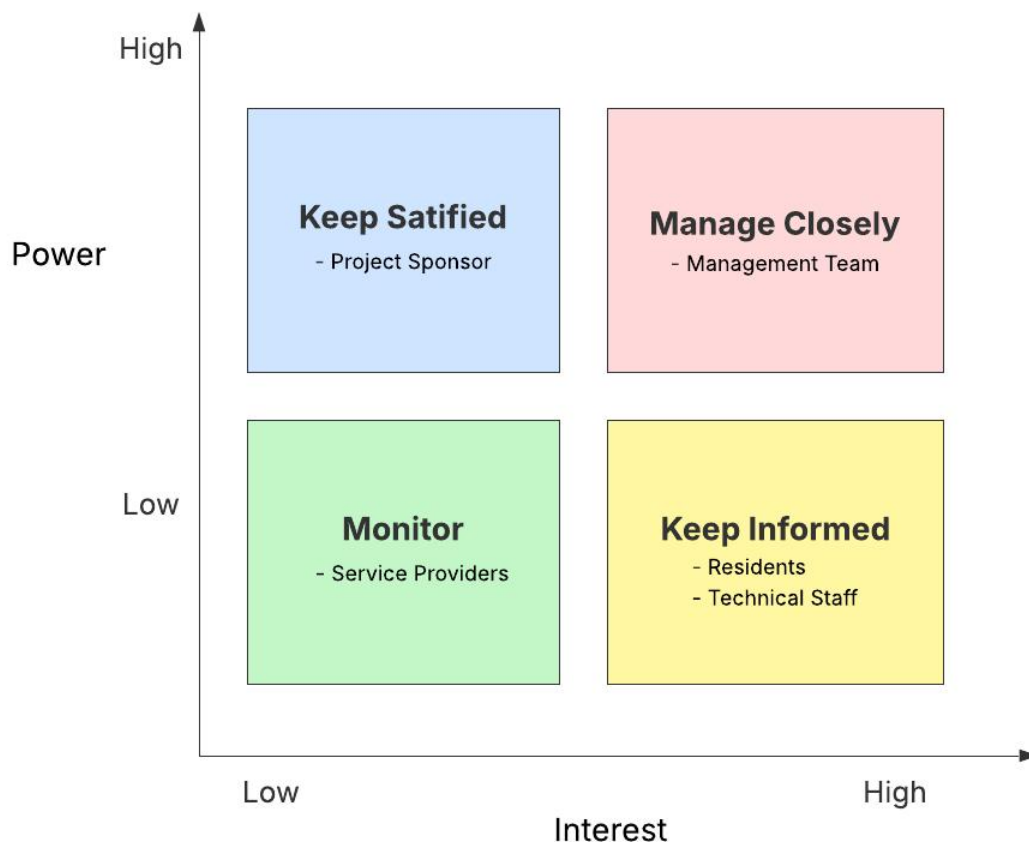


Figure 1: Power/Interest Grid

- **Manage Closely (High Power, High Interest):** Stakeholders in this quadrant are crucial to the project's success. They have the authority to make key decisions and are directly impacted by the system's outcome. For this project, the **Management Team** falls into this category. Active and regular engagement, including in-depth

interviews and collaborative workshops, is paramount to ensure their requirements are fully understood and met, and their expectations are continuously managed. Their direct involvement in shaping the system's features and functionalities, particularly for core modules like apartment, resident, and payment management, is critical.

- **Keep Satisfied (High Power, Low Interest):** These stakeholders possess significant influence but may not be deeply concerned with the project's day-to-day details. The strategy here is to ensure their overarching requirements are met without over-communicating. The **Project Sponsor/Investor** or external regulatory bodies might fit here, requiring high-level updates and assurance of compliance or strategic alignment.
- **Keep Informed (Low Power, High Interest):** This group is highly affected by the project's outcomes but has limited direct power over its decisions. Regular communication and providing sufficient information are vital to maintain their support and address any concerns. **Residents** and **Technical Staff/Maintenance personnel** are key examples. Informing residents about new system features, payment processes, and service request channels is essential for user adoption and satisfaction.
- **Monitor (Low Power, Low Interest):** Stakeholders in this quadrant require minimal engagement. They are neither highly powerful nor highly interested, so a passive monitoring approach is usually sufficient, with basic information provided if necessary. **External Service Providers** (if not directly integrated) might fall into this category.

2.1.1.3 Detailed Description of Stakeholder Groups

Following the matrix and grid, a more narrative description solidifies the understanding of each group's role and unique relationship with the project.

- **Management Team :** Playing a central role, the Management Team serves as the primary user group and decision-makers for the system. Their current responsibilities include overseeing apartment occupancy, resident registration, financial transactions (billing and payments), and coordinating service requests. Their key needs revolve around gaining centralized control over all data, automating repetitive tasks, and generating accurate financial and operational reports. Current concerns include data fragmentation, time-consuming manual processes, and the lack of real-time insights into building performance. Their influence is paramount as they dictate the system's functional scope and usability.
- **Residents:** As the ultimate beneficiaries of the system's efficiencies, residents' needs are focused on ease of interaction with management and clarity regarding their living arrangements. They require simplified processes for viewing and paying bills, submitting and tracking service requests, and accessing their apartment-related information. Their current problems often stem from opaque billing processes, slow response times for service issues, and limited channels for communication. While their direct influence on system design may be low, their collective satisfaction is a critical success factor for the project.
- **Technical Staff/Maintenance:** These individuals are responsible for the physical upkeep of the apartment complex. Their primary interest in the system lies in how it facilitates the assignment, tracking, and resolution of maintenance tasks. They currently face challenges in receiving clear and timely service requests and maintaining accurate records of completed work. Their influence is derived from their role in operationalizing the system's service request module.
- **Project Sponsor/Investor:** This stakeholder group provides the necessary financial resources and strategic direction for the project. Their primary interest lies in the successful completion of the project within the allocated budget and timeline, and in ensuring the system contributes positively to the overall

operational efficiency, resident satisfaction, and ultimately, the profitability or value of the apartment complex. Their concerns typically include potential project delays, cost overruns, and whether the system will effectively meet its strategic business objectives. Their high influence stems from their funding and oversight authority.

- **External Service Providers:** This group includes various third-party entities that offer services to the apartment complex, such as utility companies, security firms, or specialized repair services. Their interest in the system is generally low, primarily focusing on clear communication regarding service needs (e.g., specific repair tasks) and timely payment for their rendered services. Their influence on the system's design is minimal, but the system should accommodate efficient interaction channels with them (e.g., generating reports or communications for external use).

2.1.1.4 Integration with Surveys and Interviews

The insights gathered from the **Resident Survey Form** (detailed in Section 2.1.2.1) and the **Management Interview Questions** (detailed in Section 2.1.2.2) were instrumental in populating the Stakeholder Matrix and informing the Power/Interest Grid. Direct quotes or summarized feedback from these qualitative and quantitative data collection methods are used to substantiate the identified needs, problems, and expectations of each stakeholder group, ensuring the system design is truly user-centric and addresses real-world requirements. For instance, feedback from the Management Team regarding the "difficulty tracking outstanding payments" directly informed the development of detailed payment status tracking functionalities, while resident survey responses highlighting the desire for "online bill viewing" directly guided the design of the payment information view.

2.1.2 Interview

2.1.2.1 Resident Survey Form

Purpose: To gather residents' opinions on current apartment management services and their expectations for a new management system.

Target Audience: All residents currently living in the apartments.



RESIDENT SURVEY: FUTURE APARTMENT MANAGEMENT SYSTEM

Dear Residents,

We are conducting a survey to better understand your needs and expectations for a new apartment management system. Our goal is to build a system that makes interactions between residents and the Management Board more convenient and efficient. We kindly ask you to take a few moments to complete this survey. All your information will be kept confidential and used solely for system improvement purposes.

Thank you very much for your participation!

loctran18022000@gmail.com [Chuyển đổi tài khoản](#)

Không được chia sẻ



A. General Information (Optional)

Your apartment number:

Câu trả lời của bạn

Figure 2: Resident Survey Form

B. Billing & Payments Management

How do you currently receive notifications for bills (electricity, water, management fees, etc.)? *

(Please select all that apply)

- ☐ Paper bills/notices delivered to apartment
- ☐ Notices posted on public bulletin boards
- ☐ Email
- ☐ Text message (SMS/Zalo/Viber)
- ☐ Mobile application (if applicable)
- ☐ Mục khác: _____

Do you encounter any problems with the current bill reception or checking process? *

(Please select all that apply)

- ☐ Delayed notifications/bills
- ☐ Difficulty finding old bills when needed
- ☐ Bill information is unclear/hard to understand
- ☐ Concern about losing/damaging paper bills
- ☐ No problems
- ☐ Mục khác: _____

Which methods do you typically use to pay your apartment service bills? *

(Please select all that apply)

- ☐ Cash payment at the Management Office
- ☐ Direct bank transfer

Figure 3: Resident Survey Question

The "Resident Survey: Future Apartment Management System" is designed to gather comprehensive feedback from residents regarding their experiences with current apartment management services and their expectations for a new digital system. The survey aims to identify pain points in existing processes and prioritize desired functionalities to ensure the new system is user-centric and highly effective.

The survey is structured into four main sections:

1. **General Information (Optional):** This section collects basic demographic data, specifically the resident's apartment number and whether they are the apartment owner or main tenant. This helps in segmenting responses if needed.
2. **Billing & Payments Management:** This is a core focus, inquiring about current bill notification methods (e.g., paper, email, SMS, mobile app) and any associated problems (e.g., delays, clarity, difficulty tracking old bills). It then explores preferred payment methods (e.g., cash, bank transfer, e-wallets) and gauges interest in specific features for the new system, such as viewing detailed bills online, accessing payment history, making online payments, receiving digital notifications, and tracking outstanding balances.
3. **Service Requests & Communication:** This section probes into how residents currently submit requests or communicate with the Management Board (e.g., phone, email, in-person, messaging apps) and any challenges faced (e.g., difficulty describing issues, slow responses, lack of status updates). It then asks residents to rate the importance of features like online request submission, status tracking, notification alerts, attaching media to requests, and rating service quality.
4. **General Suggestions:** This open-ended section allows residents to provide any additional suggestions or desires for the new apartment management system that could further enhance their living experience.

Overall, the survey is a crucial tool for eliciting direct resident needs, emphasizing transparency, convenience, and efficiency in financial transactions and service interactions, thereby guiding the development of a highly relevant and impactful management system.

2.1.2.2 Management Interview Questions

To gain an in-depth understanding of the Management Team's needs, challenges, and expectations for the Apartment Management System, a structured interview was conducted. The key questions posed and the summarized responses are detailed below:

Table 2: Management Interview Records

Topic	Questions	Records / Summarized Answers
Topic 1: Overview of Current Status and Problems	1. Could you please provide an overview of your current processes for managing apartment information, residents, finances, and services?	Management is currently a hybrid process involving manual ledgers, Excel spreadsheets, and some disparate software for specific tasks, such as accounting.
	2. What are the biggest difficulties or challenges you currently face in daily management and operations?	Key challenges include decentralized data, making retrieval and report generation difficult and time-consuming. Manual tasks, like bill issuance and payment tracking, are inefficient and prone to errors. Overall, there's a lack of real-time operational overview.
	3. Which urgent problems do you expect the new system to solve?	The new system is expected to automate processes, centralize data, enhance efficiency, minimize errors, and provide

		comprehensive operational insights.
Topic 2: Apartment Management	1. What specific information about an apartment is currently managed?	Currently managed information includes Apartment ID, associated Building ID, Max Population, Current Population, Transfer Status (e.g., Available, Pending, Sold), and Vacancy Status (e.g., Vacant, Occupied).
	2. How do the processes for adding, updating, and deleting apartment information work? Are there any specific validation rules required?	Processes are currently manual. Specific validation rules are required for apartment number format (e.g., AXX.YY), Max Population (1-6), and predefined Transfer/Vacancy Status values.
	3. How are apartment status changes (e.g., from "Vacant" to "Occupied", or transfer statuses) currently handled and controlled?	Status changes are currently manual. The system is expected to automate or provide clear interfaces for these updates, including historical logging.
Topic 3: Resident Management	1. What detailed resident information is mandatory to manage?	Mandatory resident information includes Name, Phone Number, Email, Gender, and Identification Number.
	2. How do you distinguish between the apartment owner/main tenant and other family members or occupants within the same apartment? Is it necessary	Distinction is currently manual. The system should link residents to apartments, identify the owner (owner_id), and manage associated members.

	to define their relationships?	
	3. What is the current process for registering new residents and updating their information?	Registration is manual, requiring an easy interface for adding/updating/deleting residents, with validation for unique identification numbers and phone number formats.
Topic 4: Billing & Payment Management	1. Please list all types of bills that residents typically pay. What is the frequency and calculation method for each?	Main bill types include management fees, electricity, water, and parking fees, typically calculated monthly.
	2. What is the process for generating new bills? Is there a need for recurring bill generation?	Bills are generated manually or semi-automatically using Excel. Automated recurring bill generation (e.g., monthly) is a high priority.
	3. When a resident makes a payment, what is the current process for recording it?	Payments are manually recorded in ledgers. The system should automatically record payment status (Pending, Completed, Overdue) and payment methods.
	4. Is it necessary to manage the detailed breakdown of charges within a bill?	Yes, detailed management of individual charges within a bill (Payment Details / Bill Details) is necessary for transparency.
Topic 5: Service	1. What are the common types of service requests	Common requests include repairs (electrical, plumbing), general cleaning,

Request	residents typically submit?	security concerns, and noise complaints.
	2. What is the current process for receiving and assigning service requests?	Requests are received via phone, email, or in-person, manually logged, and then manually assigned.
	3. What information needs to be recorded for each request?	Required information includes request type, detailed description, request date, completion date, assigned personnel, and status.
Topic 6: Reporting & Dashboard	1. What are the most important metrics you want to see on an overall dashboard to monitor the building's status?	Key metrics desired on the dashboard include total/occupied apartments, total residents, current/previous month's revenue, and service request counts, along with month-over-month changes.
	2. How frequently do you need these metrics updated?	Real-time or at least daily updates for metrics are preferred.
Topic 7: General System Requirements (Non- functional)	1. Regarding the user interface, do you have any specific preferences?	The UI should be intuitive, user-friendly for non-technical staff, and visually clear.
	2. Concerning performance and processing speed, what are your expectations?	The system is expected to provide fast response times (under 3 seconds) for searches, filtering, and data loading.
	3. Regarding security, what levels of access control (permissions) are needed?	Clear access permissions are essential for various roles (Admin, Manager, User) to ensure data security.
	4. Does the system need to	Yes, multi-schema management is a

	be scalable to manage multiple buildings/schemas? How would you prefer the process of switching between schemas to work?	critical requirement. A seamless and quick process for switching between building schemas is preferred.
Topic 8: Other Suggestions & Feedback	1. Do you have any other suggestions or feedback for the new apartment management system that could further improve your work?	Desired features include automated reminders (e.g., for due payments, overdue requests) and the ability to customize fields in the future.

2.1.3 Conclusion of Stakeholder Surveys

2.1.3.1 Insights from the Resident Survey

The Resident Survey highlighted a strong desire among residents for enhanced convenience and transparency in their interactions with apartment management. Key findings indicated:

- **Preference for Digital Access to Financial Information:** A significant majority expressed a strong desire to view detailed bills online, track payment history, and monitor outstanding balances, suggesting a move away from traditional paper-based methods.
- **Demand for Online Payment Options:** Residents showed a high interest in the ability to make online payments via bank transfers or e-wallets, indicating a need for modern, flexible payment solutions.
- **Streamlined Service Request Process:** Many residents encountered difficulties with current service request methods, particularly regarding unclear status tracking

and slow responses. There is a clear demand for an online platform to submit, track, and receive updates on service requests, including the ability to attach media.

- **Improved Communication:** Residents seek more efficient and transparent communication channels with the management, minimizing delays and ensuring timely notifications.

2.1.3.2 Insights from the Management Interview

The Management Interview provided a detailed understanding of the operational inefficiencies, data management challenges, and strategic automation needs from the management's perspective. Key takeaways included:

- **Urgent Need for Data Centralization and Automation:** Management heavily emphasized the current fragmentation of data across various manual and disparate digital tools, leading to inconsistencies, errors, and time-consuming processes. There is a critical need for a centralized system to automate tasks such as bill generation and status updates for apartments and residents.
- **Enhanced Financial Oversight:** Management requires real-time visibility into financial status, including comprehensive tracking of payment statuses (Pending, Completed, Overdue) and detailed breakdowns of charges within bills. Automated financial reporting and comprehensive dashboards are high-priority requirements.
- **Improved Operational Efficiency:** Challenges in tracking service requests, assigning tasks, and monitoring resolution progress were highlighted. Management seeks a system that streamlines these workflows, provides clear status updates, and retains historical data for analysis.
- **Scalability and Robustness:** A significant requirement is the system's ability to manage multiple buildings/schemas within a single database, with seamless switching capabilities, indicating a need for a robust and scalable architecture.

- **Non-Functional Requirements:** High importance was placed on intuitive UI/UX, fast performance, strong data security with clear access controls, and maintainability.

2.1.3.3 Overall Conclusion and Implications for System Design

The convergence of insights from both stakeholder groups underscores a collective need for a unified, efficient, and user-friendly apartment management system. Both residents and management aspire for greater transparency, automation, and convenience in their respective interactions and tasks.

- **Core Alignment:** The system's primary functional areas (Apartment Management, Resident Management, Billing & Payments, Service Requests) are directly validated by the expressed needs of both residents and management.
- **Emphasis on Automation and Digitalization:** The findings strongly support the development of automated processes for billing, payment tracking, and service request management, which will reduce manual effort for management and offer convenience for residents.
- **Data Centralization and Accessibility:** The project's commitment to a centralized database and the multi-tenant schema mechanism is directly reinforced as a solution to address data fragmentation and enable comprehensive reporting and dashboard functionalities for management.
- **User Experience Focus:** The clear demand for intuitive interfaces, efficient processes, and reliable performance from both sides emphasizes the importance of a user-centric design approach.

In essence, the surveys confirm that the proposed Apartment Management System is poised to address critical operational gaps and enhance the overall experience for all stakeholders involved in apartment complex management.

2.1.4 Survey of the organization/ Current organizational situation

This section aims to provide a clear understanding of the current organizational structure, roles, responsibilities, and general workflows within the Apartment Management Board (or the operating entity of the apartment complex). A thorough understanding of the organization is crucial for identifying how the new system will integrate, which departments and individuals will use it, and how information flows between them.

Methods of Survey:

The information for this section was primarily gathered through:

1. **In-depth interviews** with key management personnel, including the Head of the Management Board and department heads, to gain insights into the overall structure, strategic objectives, and major operational challenges.
2. **Review of internal documentation** (if available), such as existing organizational charts and job descriptions, to verify reporting lines and departmental functions.
3. **Observation** of daily operations (where feasible) to understand the practical flow of information and interactions between departments.

2.1.4.1 Current Organizational Chart of the Apartment Management Board

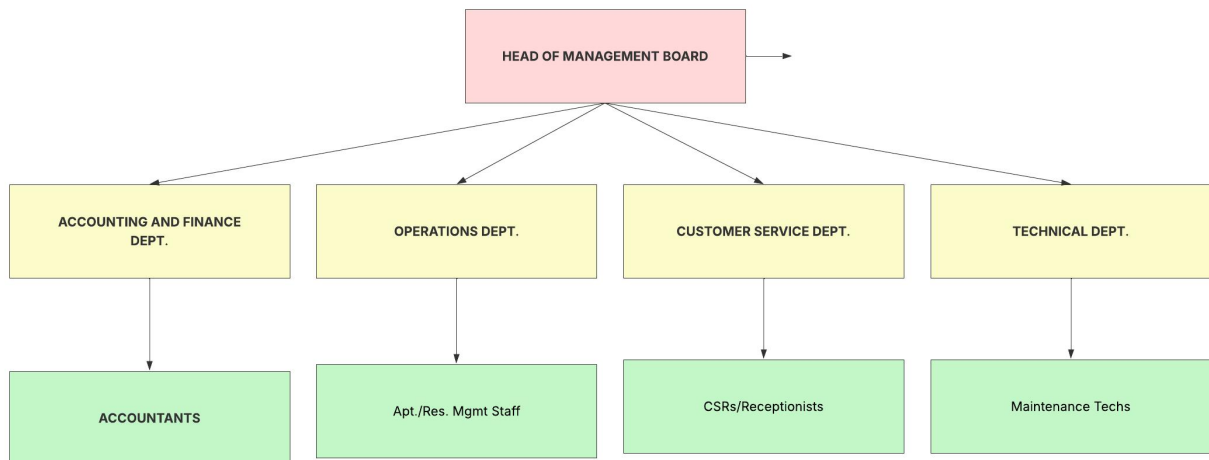


Figure 4: Current Organizational Chart of BCONS Management Board

The organizational structure of the Apartment Management Board is designed to manage various aspects of the apartment complex efficiently. It is hierarchical, with the **Head of Management Board** at the apex, overseeing all operational and administrative functions.

2.1.4.2 Functions and Responsibilities of Relevant Departments

Head of Management Board

- **Function:** Oversees and directs all activities of the Apartment Management Board.
- **Responsibilities:** Approves policies, supervises financial operations, resolves escalated issues, and evaluates overall performance.

Accounting and Finance Department

- **Function:** Manages all financial aspects of the apartment complex.
- **Responsibilities:** Issues and processes bills (electricity, water, management fees), collects payments, tracks outstanding debts from residents, and prepares monthly/quarterly/annual financial reports.

Operations Department (Apartment & Resident Management)

- **Function:** Manages all information related to apartment units and residents.
- **Responsibilities:** Registers/updates apartment information (status, capacity), registers/updates resident information (personal details, ID, contact, email), identifies apartment owners and members, and handles property transfer/rental procedures.

Customer Service / Reception Department

- **Function:** Serves as the primary point of contact for residents.
- **Responsibilities:** Receives service requests, feedback, and inquiries from residents; channels requests to the relevant departments; and disseminates general announcements to residents.

Technical / Maintenance Department

- **Function:** Ensures the stable operation of the building's facilities and technical systems.
- **Responsibilities:** Receives repair/maintenance requests (via Customer Service), plans and executes routine maintenance, and addresses technical malfunctions.

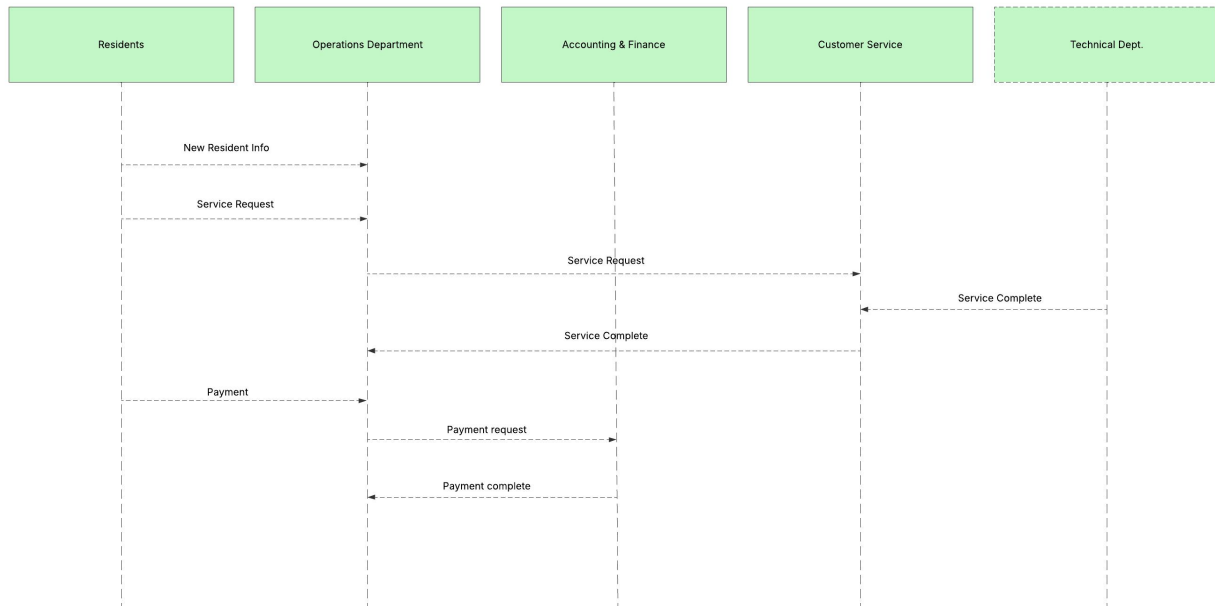


Figure 5: Current Information Flow and Inter-Departmental Interactions in BCONS Management Board

2.1.4.3 Information Flow and Key Interactions

The existing information exchange among departments heavily relies on traditional, often manual methods, including email, phone calls, and physical paper documents. This manual relay of information frequently introduces delays and potential points of error within the operational workflows.

For instance, a typical scenario involves the **Operations Department** collecting new resident information upon move-in. This critical data is then manually relayed to the **Accounting & Finance Department** to initiate the billing processes for services and fees. This manual handover can lead to delays in generating initial invoices or inconsistencies if data is not transferred accurately or promptly.

Similarly, when residents submit service requests, these are primarily received by the **Customer Service / Reception Department** through phone calls or in-person interactions. This department then manually forwards these requests, often via written notes or email, to the **Technical / Maintenance Department** for action. The lack of a

centralized tracking mechanism means that updates on the request status must also be communicated manually, potentially leading to residents being uninformed or requests being overlooked.

2.1.4.4 Identified Organizational Problems Related to Information Management

- **Lack of Centralized Information:** Resident and apartment data are often fragmented across multiple departments and disparate manual records, leading to inefficiencies in retrieval and updates.
- **Delays in Information Exchange:** The reliance on manual processes for transferring requests and notifications between departments causes significant delays in operational workflows, particularly for urgent service requests.
- **Risk of Data Inaccuracies:** Repetitive data entry or transcription across different files and ledgers introduces a high risk of errors and inconsistencies.
- **Inefficient Coordination:** The absence of an integrated system hinders seamless collaboration between departments, impacting overall response times and service quality.

The new Apartment Management System aims to digitize and centralize these information exchanges, creating a seamless and automated flow of data between departments, thereby addressing the identified inefficiencies and enhancing overall operational effectiveness.

2.1.5 Survey of business processes, major activities

2.1.5.1 Apartment Management Process

This process covers the lifecycle of an apartment unit from initial setup to changes in occupancy or ownership.

Current Activities:

- **Apartment Registration:** Manual recording of new apartment details (e.g., apartment number, building, floor, maximum capacity, initial vacancy status) in ledgers or spreadsheets.
- **Status Update:** Manual updating of an apartment's vacancy status (e.g., Vacant to Occupied) or transfer status (e.g., Available to Sold or Pending) upon resident move-in/out or ownership transfer. This often involves cross-checking with resident records.
- **Information Retrieval:** Searching through physical records or multiple digital files to find specific apartment details or historical status changes.

Challenges Identified:

- High risk of data inconsistency and errors due to manual entry and updates.
- Time-consuming process for status changes and verification.
- Difficulty in obtaining real-time availability of apartments, impacting potential sales or rentals.
- Lack of a centralized history log for apartment status changes.

System Impact: The new system will provide a centralized database for apartment information, enabling efficient creation, update, and deletion of apartment records. It will automate status updates and provide a real-time overview, addressing data accuracy and accessibility challenges.

2.1.5.2 Resident Management Process

This process focuses on the registration and ongoing management of individuals residing in the apartment complex.

Current Activities:

- **Resident Registration:** Manual collection and recording of new resident details (e.g., name, phone number, email, gender, identification number, apartment association) in physical forms and then transferred to spreadsheets.
- **Owner/Member Association:** Manually linking residents to their specific apartments and identifying the primary owner/tenant versus other family members or occupants.
- **Information Update:** Manual modification of resident details (e.g., contact information, status changes) as needed.
- **Resident Departure/Relocation:** Manual process of marking a resident as moved out and updating associated apartment records.

Challenges Identified:

- Duplication of records and inconsistencies in resident data across different forms/files.
- Difficulty in quickly identifying all residents associated with a particular apartment or finding contact information.
- Lack of a clear, structured way to represent owner-member relationships within an apartment.
- Time-consuming searches for specific resident profiles.

System Impact: The system will centralize resident data, allowing for efficient CRUD operations. It will enable clear linking of residents to apartments and defining owner/member relationships, improving data integrity and search capabilities.

2.1.5.3 Billing and Payment Process

This critical process manages the generation of invoices and the collection of payments for various services.

Current Activities:

- **Bill Generation:** Manual calculation of charges (e.g., electricity, water, management fees) often using separate spreadsheets or utility company statements, and then manually preparing physical or email invoices.
- **Bill Distribution:** Delivery of physical bills or sending invoices via email/messaging apps to residents.
- **Payment Collection:** Receiving payments (cash, bank transfer) and manually recording them in ledgers or spreadsheets.
- **Payment Status Tracking:** Manually updating payment status (e.g., Pending, Completed, Overdue) for each bill and resident.
- **Debt Reconciliation:** Manually identifying outstanding payments and following up with residents.

Challenges Identified:

- High labor cost and time consumption for manual bill generation and distribution.
- Frequent errors in calculations and data entry, leading to disputes with residents.
- Lack of real-time visibility into overall financial status and individual resident's outstanding debts.
- Inefficient tracking of different payment methods and dates.

System Impact: The system will automate aspects of bill generation and distribution, provide a centralized record of all transactions, and offer real-time payment status tracking. This will reduce errors, improve transparency, and enhance financial oversight.

2.1.5.4 Service Request Management Process

This process handles incoming service requests from residents and ensures their resolution.

Current Activities:

- **Request Reception:** Residents submit requests via phone calls, emails, or in-person visits to the reception or management office.
- **Manual Logging:** Requests are manually logged in a notebook or basic spreadsheet, often with limited detail.
- **Task Assignment:** Requests are manually assigned to relevant technical staff or external service providers.
- **Status Tracking:** Manual follow-up on request status, often by phone or personal inquiry.
- **Completion Confirmation:** Manual confirmation of completed tasks.

Challenges Identified:

- Lack of a standardized channel for residents to submit requests.
- Poor tracking of request progress, leading to requests being lost or delayed.
- Difficulty in prioritizing requests and assigning them efficiently.
- Limited visibility for residents into the status of their submitted requests.

- Lack of historical data on service requests for analysis (e.g., common issues, resolution times).

System Impact: The system will provide a centralized platform for service request submission and tracking, enabling automated assignment, status updates, and historical logging. This will significantly improve response times, accountability, and resident satisfaction.

2.1.5.5 Reporting and Dashboard Activities

This process involves gathering and presenting key operational and financial data for decision-making.

Current Activities:

- **Manual Data Aggregation:** Data is manually extracted from various sources (spreadsheets, ledgers) and consolidated.
- **Report Generation:** Reports are created manually using spreadsheet software (e.g., Excel) or basic templates.
- **Performance Monitoring:** Management relies on these manually generated reports to assess apartment occupancy, resident numbers, and financial performance.

Challenges Identified:

- Time-consuming and error-prone process for data aggregation and report generation.
- Reports are often outdated due to the manual nature of their creation.
- Limited ability to customize reports or perform deep data analysis.
- Lack of a real-time, consolidated overview of key performance indicators (KPIs).

System Impact: The system will provide an integrated dashboard that automatically aggregates data from all modules, offering real-time KPIs and easily accessible reports. This will significantly enhance decision-making capabilities and strategic planning for the management.

2.1.6 Survey of the existing system

2.1.6.1 Description of the Existing System's Components and Functionality

Manual Record-Keeping:

- **Apartment & Resident Data:** Information regarding apartment units (e.g., status changes, basic details) and resident demographics (e.g., names, contact numbers, identification numbers) is largely maintained in physical ledgers, notebooks, and paper files.
- **Service Requests:** Incoming service requests from residents are typically logged manually on paper forms or in a general notebook at the reception desk.
- **Payment Receipts:** Cash payments received are recorded in physical receipt books, and details are later transcribed.

Spreadsheet Applications (Microsoft Excel):

- **Billing & Payments:** Excel spreadsheets are widely used for calculating monthly utility bills (electricity, water, management fees) and for tracking resident payments. Separate spreadsheets may be maintained for each apartment unit or for different billing cycles.
- **Basic Reports:** Simple financial summaries or lists of occupied apartments are often generated by manually compiling data from various spreadsheets.

- **Employee/Task Tracking:** Some departments might use basic Excel sheets for assigning tasks or tracking employee attendance, though this is often informal.

Communication Methods:

- **Inter-departmental:** Communication and information relay between departments (e.g., Operations to Accounting, Customer Service to Technical) primarily occur via internal memos, phone calls, or direct verbal communication, and sometimes through email for more formal exchanges.
- **Resident-facing:** Residents receive bill notifications as physical printouts or simple email attachments, and submit requests via phone or in-person visits.

2.2 Software requirements

Based on the detailed situation analysis, stakeholder interviews, and a comprehensive survey of current business processes and the existing system, the following software requirements have been identified for the Apartment Management System. These requirements are categorized into functional, non-functional, and report forms, outlining what the system must do, and the outputs it must generate.

2.2.1 Functional requirements

2.2.1.1 Apartment management module

This module handles all operations related to apartment units within the system.

Storage Functions:

- The system shall store comprehensive apartment information including a unique apartment identifier, associated building identifier, an optional owner identifier, maximum occupancy capacity, current occupancy,

transfer status, vacancy status, creation timestamp, last updated timestamp, and registration date.

- The system shall support the retrieval, update, and deletion of apartment records based on various criteria.
- The system shall automatically generate creation and last updated timestamps upon initial record creation and subsequent modification of apartment details.

Feature Functions:

Query Function:

- The system shall display a comprehensive, paginated list of all apartment units.
- It shall allow users to search for apartments by their unique identifier (apartment number).
- It shall enable filtering of the apartment list by its occupancy status (e.g., "Vacant", "Occupied") and transfer status (e.g., "Available", "Pending", "Sold").
- It shall provide sorting capabilities for the apartment list by criteria such as unique apartment identifier or occupancy status.
- The system shall display a detailed view of a selected apartment, including its owner, building, and any related resident or financial data.

Update Function:

- The system shall allow users to add new apartment records via a dedicated input form.

- It shall provide an interface for editing existing apartment information, including maximum and current occupancy, and transfer and vacancy statuses.
- The system shall enable users to delete apartment records.

Automatic Functions:

- The system shall automatically validate input data for new and updated apartments, ensuring correct apartment number format, maximum occupancy capacity within range (1-6), and valid status values.
- The system shall automatically update the last updated timestamp upon any modification to an apartment record.

2.2.1.2 Residential management module

This module focuses on managing all information and operations pertaining to residents within the apartment complex.

Storage Functions:

- The system shall store resident information including a unique resident identifier, full name, contact phone, email address, gender, identification document number, associated apartment identifier, an optional primary resident identifier (for family heads), creation timestamp, last updated timestamp, and registration date.
- The system shall support the retrieval, update, and deletion of resident records, including their associated apartment and payment information.

Feature Functions:

Query Function:

- The system shall display a paginated list of all residents.

- It shall provide search capabilities for residents by name, phone number, or associated apartment identifier.
- It shall allow sorting of the resident list by criteria such as unique resident identifier or associated apartment identifier.
- The system shall display a detailed resident profile, including their associated apartment, the primary owner's name, and their most recent payment status.

Update Function:

- The system shall allow users to add new resident records, capturing personal details and linking them to an apartment.
- It shall provide an interface for editing existing resident details, including full name, contact phone, email address, gender, and identification document number.
- The system shall enable users to delete resident records.

Automatic Functions:

- The system shall automatically validate resident input data (e.g., identification document number length, email format, valid gender values).
- The system shall automatically manage primary owner relationships within an apartment for residents.

2.2.1.3 Bill and payment management module

This module manages the generation of invoices and the collection and tracking of payments for various services.

Storage Functions:

- The system shall store payment information including a unique payment identifier, total amount, payment recording date, payment creation date, payment status, associated apartment identifier, and payment method.
- It shall store individual bill records including a unique bill identifier, associated apartment identifier, bill type, bill amount, payment status, due date, and bill issuance date.
- It shall maintain payment details linking specific bills to payments via their unique identifiers.

Feature Functions:

Query Function:

- The system shall display a paginated list of all payments.
- It shall allow filtering of payments by their payment status (e.g., "Pending", "Completed", "Overdue").
- It shall enable searching payments by associated apartment identifier.
- The system shall provide sorting options for payments by payment creation date (Newest/Latest) or associated apartment identifier.
- The system shall display a detailed view for a selected payment, including all associated bills.

Update Function:

- The system shall allow users to create new payment records, specifying the amount, associated apartment, and resident responsible.
- The system shall enable users to delete payment records.

Automatic Functions:

- The system shall automatically validate payment creation input, ensuring the amount is a positive decimal and associated entities exist.

2.2.1.4 Service requirements management

This module handles the reception, processing, and tracking of service requests initiated by residents.

Storage Functions:

- The system shall store service request information including a unique request identifier, associated apartment identifier, associated resident identifier, request category, description, status, request submission date, and an optional completion date.

Feature Functions:

Query Function:

- The system shall display a paginated list of all service requests, showing relevant details like associated apartment, resident, category, description, and status.

Update Function:

- The system shall allow for the creation of new service requests.
- It shall enable updates to the status of service requests (e.g., from pending to completed).

2.2.1.5 Dashboard

This module provides an overview of key operational metrics and is responsible for generating various reports.

Storage Functions: (Data for reporting is aggregated from other modules, not stored separately in this context, but rather retrieved from various tables)

- The system relies on data stored in apartment, resident, payment, and service request tables for its reporting.

Feature Functions:**Statistical, Analysis, Reporting Functions:**

- The system shall display a dashboard with key performance indicators (KPIs) such as total apartment count, occupied apartment count, total resident count, current and last month's total revenue, and service request counts.
- It shall calculate and display month-over-month changes for revenue, occupancy rate, resident count, and service request count.
- The dashboard shall present monthly revenue figures for the current year.

2.2.1.6 System

This module handles overarching system operations, user access, and multi-tenant capabilities.

Storage Functions:

- The system shall store user information including a unique user identifier, email address, hashed password, assigned role, account status, creation timestamp, last updated timestamp, last login date, and permission level.
- It shall store building information including a unique building identifier, building name, address, associated manager identifier, creation timestamp, and last updated timestamp.

Feature Functions:**Query Function:**

- The system shall provide a login screen for user authentication.
- The system shall include a control (e.g., a ListBox for building selection) allowing users to select and switch between different building schemas.

Automatic Functions:

- The system shall automatically manage the current building schema context, ensuring all data operations target the correct tenant-specific schema.
- The system shall handle pagination automatically for large lists of data across all main modules to enhance performance and usability.

2.2.2 Non-functional requirements

Non-functional requirements specify the quality attributes, constraints, and general characteristics of the Apartment Management System. These requirements are crucial for defining how well the system performs its functions, its operational environment, and its overall user experience.

- **Performance:** The system shall ensure efficient and responsive operations. Specifically, main data grids (for Apartments, Residents, Payments, and Service Requests) are required to load within 3 seconds when displaying up to 1000 records per schema. Search, filter, and sort operations across these modules shall complete within 2 seconds to support quick data retrieval and analysis.
- **Security:** The system shall implement robust security measures to protect sensitive data and control access. This includes requiring user authentication for all system access, storing user passwords as securely hashed values in the database, and enforcing role-based access control (RBAC) to restrict functionalities based on assigned user roles (e.g., Admin, Manager, User). Crucially, data access shall be logically separated by building schema, ensuring users only access data pertaining to the currently selected building.
- **Usability:** The system shall be user-friendly and intuitive to minimize the learning curve for staff. The user interface (UI) must be clean, easy to navigate, and designed for efficient task completion, allowing common operations (such as adding a resident or recording a payment) to be performed with minimal clicks.

All system messages, especially error notifications, shall be clear, concise, and provide actionable guidance to the user.

- **Scalability:** The system's architecture is designed to support future growth and expansion. It shall accommodate the addition of new apartment buildings by effectively leveraging the multi-tenant schema mechanism in PostgreSQL, without necessitating significant code restructuring. The underlying database design shall also be capable of handling an increasing volume of data (including apartments, residents, payments, and service requests) without experiencing substantial performance degradation.
- **Reliability:** The system shall maintain data integrity and consistency across all operations. It must include robust error handling mechanisms to gracefully manage and recover from potential issues, such as database transaction failures, ensuring that data remains accurate and available.
- **Maintainability:** The software's structure is built upon the MVVM design pattern and a 3-tier architecture, promoting modularity, separation of concerns, and ease of future modifications or enhancements. The code base shall adhere to established coding standards and be well-documented to facilitate future maintenance and development efforts.
- **Compatibility:** The application is developed as a desktop solution and shall be compatible with modern Windows operating systems, specifically Windows 10 and 11, ensuring a broad user base and operational stability.

CHAPTER 3. ANALYSIS AND DESIGN

3.1 Business Function Diagram – BFD

The Business Function Diagram (BFD) represents the hierarchical decomposition of business functions within the Apartment Management System. This diagram provides a systematic approach to identifying and organizing the core functionalities required for effective apartment complex management, without concerning the implementation methods.

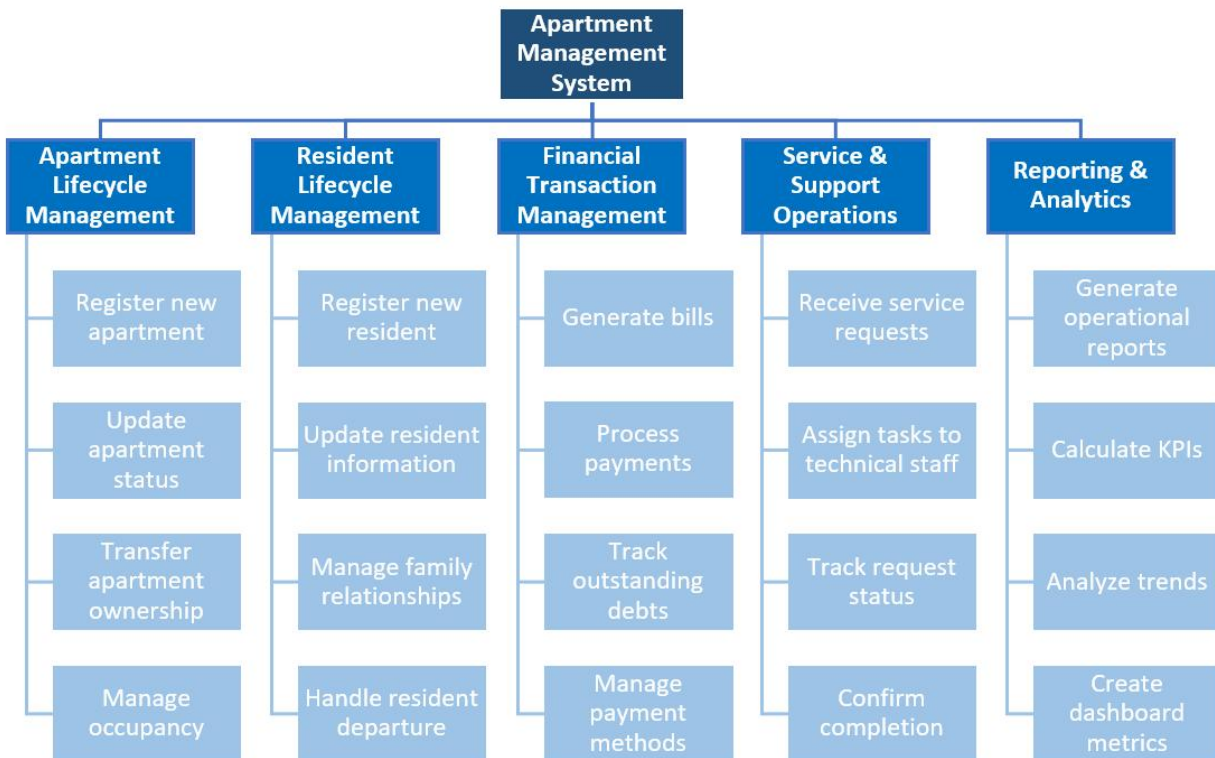


Figure 6: BFD diagram

3.1.1 Level 0: Overall Business Function

At the highest level, the Apartment Management System encompasses a single, overarching business function:

Apartment Management System: This represents the complete system designed to streamline and automate all operational aspects of apartment complex management. The primary objective is to provide a centralized platform that enhances operational efficiency, improves service quality, and delivers comprehensive management oversight for apartment complexes.

The system serves as a unified solution addressing the challenges of fragmented manual processes, data inconsistencies, and lack of real-time operational visibility that are commonly found in traditional apartment management approaches.

3.1.2 Level 1: Major Business Functions (Primary Domains/Lifecycle Stages)

The Level 1 decomposition identifies five major functional domains that collectively support the overall apartment management objective:

Apartment Lifecycle Management: Encompasses all activities related to the complete lifecycle of apartment units, from initial registration and status tracking to ownership transfers and occupancy management. This domain ensures accurate maintenance of apartment inventory and availability status.

Resident Lifecycle Management: Covers the comprehensive management of resident information and relationships, including registration of new residents, maintenance of personal details, family relationship tracking, and handling of resident departures or relocations.

Financial Transaction Management: Handles all financial aspects of apartment operations, including bill generation for various services, payment processing and tracking, debt management, and financial status monitoring. This domain ensures transparent and efficient financial operations.

Service & Support Operations: Manages the entire service request workflow, from initial request submission by residents to task assignment, progress tracking, and completion confirmation. This domain focuses on maintaining high service quality and resident satisfaction.

Reporting & Analytics: Provides comprehensive data analysis and reporting capabilities, generating operational insights, calculating key performance indicators, analyzing trends, and creating dashboard metrics for informed decision-making.

3.1.3 Level 2: Detailed Business Functions (Specific Activities)

Level 2 provides a detailed breakdown of specific activities within each major functional domain:

Apartment Lifecycle Management Functions:

- **Register new apartment:** Process of adding new apartment units to the system with complete specifications and initial status
- **Update apartment status:** Modification of apartment occupancy and availability status based on current conditions
- **Transfer apartment ownership:** Management of ownership change processes, including documentation and status updates
- **Manage occupancy:** Tracking and updating current occupancy levels and capacity management

Resident Lifecycle Management Functions:

- **Register new resident:** Complete registration process for new residents, including personal details and apartment association
- **Update resident information:** Maintenance and modification of existing resident data and contact information

- **Manage family relationships:** Tracking and management of family connections and primary owner designations within apartments
- **Handle resident departure:** Processing of resident move-out procedures and associated data updates

Financial Transaction Management Functions:

- **Generate bills:** Automated and manual creation of various bill types for residents, including utilities and service fees
- **Process payments:** Recording and processing of resident payments through various payment methods
- **Track outstanding debts:** Monitoring and management of overdue payments and debt collection activities
- **Manage payment methods:** Administration of available payment options and processing mechanisms

Service & Support Operations Functions:

- **Receive service requests:** Initial capture and logging of service requests submitted by residents
- **Assign tasks to technical staff:** Distribution and assignment of service requests to appropriate maintenance personnel
- **Track request status:** Monitoring progress and status updates of ongoing service requests
- **Confirm completion:** Verification and confirmation of completed service requests with quality assurance

Reporting & Analytics Functions:

- **Generate operational reports:** Creation of comprehensive reports on apartment operations, occupancy, and resident statistics
- **Calculate KPIs:** Computation of key performance indicators for management oversight and performance evaluation
- **Analyze trends:** Identification and analysis of operational patterns and performance trends over time
- **Create dashboard metrics:** Development of real-time dashboard displays with critical operational and financial metrics

3.2 Data Flow Diagram – DFD

3.2.1 Context Diagram

The Context Diagram, also referred to as the Level 0 DFD, offers a high-level overview of the entire Apartment Management System. It depicts the system as a single central process and highlights its interactions with key external entities. This diagram is crucial for defining the system's scope and its boundaries with the outside world.

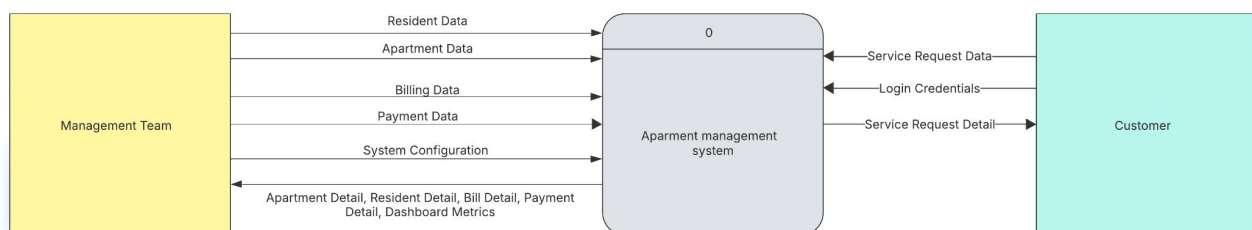


Figure 7: DFD Context Diagram

External Entities:

The Context Diagram identifies the primary external entities that interact with the Apartment Management System:

- **Management Team:** This entity represents the administrative and operational staff responsible for managing the apartment complex. They provide data inputs for

various management functions and receive processed information and reports from the system.

- **Customer (Resident):** This entity primarily represents the residents of the apartment complex. They interact with the system to access services, submit requests, and provide necessary personal information.

Central Process:

- **Apartment Management System (Process 0):** This is the central process that encapsulates all functionalities of the proposed system. It receives inputs from the external entities, processes this information according to predefined business logic, and produces outputs directed back to these entities.

Data Flows:

The diagram illustrates the following data flows between the external entities and the Apartment Management System:

- **Data Flows from Management Team to System:**
 - **Resident Data:** Information pertaining to residents, such as personal details, contact information, and occupancy status, provided by the Management Team for input into the system.
 - **Apartment Data:** Details regarding apartment units, including their specifications, status (e.g., vacant, occupied), and ownership information, entered by the Management Team.
 - **Billing Data:** Information used for generating invoices, such as utility consumption, service charges, and fee structures, supplied by the Management Team.
 - **Payment Data:** Records of payments made by residents, including amounts, dates, and methods, inputted by the Management Team.
 - **System Configuration:** Settings and parameters for configuring the system's behavior, such as user roles, access permissions, and building-specific rules, managed by the Management Team.
- **Data Flows from System to Management Team:**
 - **Apartment Detail:** Processed and structured information about specific apartment units, retrieved by the Management Team.

- Resident Detail: Comprehensive profiles of residents, including their history and associations, accessible to the Management Team.
- Bill Detail: Generated bills and financial statements for residents, available for review and distribution by the Management Team.
- Payment Detail: Summaries and detailed records of resident payments and outstanding dues, provided to the Management Team.
- Dashboard Metrics: Aggregated key performance indicators (KPIs) and analytical reports on various aspects of apartment management, presented to the Management Team for decision-making.
- Data Flows from Customer (Resident) to System:
 - Login Credentials: User authentication information (e.g., username and password) provided by residents to access system functionalities.
 - Service Request Data: Information submitted by residents regarding maintenance issues, facility bookings, or other service needs.
- Data Flows from System to Customer (Resident):
 - Service Request Detail: Updates, status information, and resolution details regarding service requests previously submitted by the resident.

This Context Diagram lays the groundwork for further decomposition in subsequent DFD levels, which will elaborate on the internal processes and data stores within the Apartment Management System.

3.2.2 Level-0 DFD

The Level-0 Data Flow Diagram provides a more detailed decomposition of the "Apartment Management System" process identified in the Context Diagram. This level illustrates the major sub-processes within the system, the data stores they utilize, and the flow of data between these processes, data stores, and the external entities.

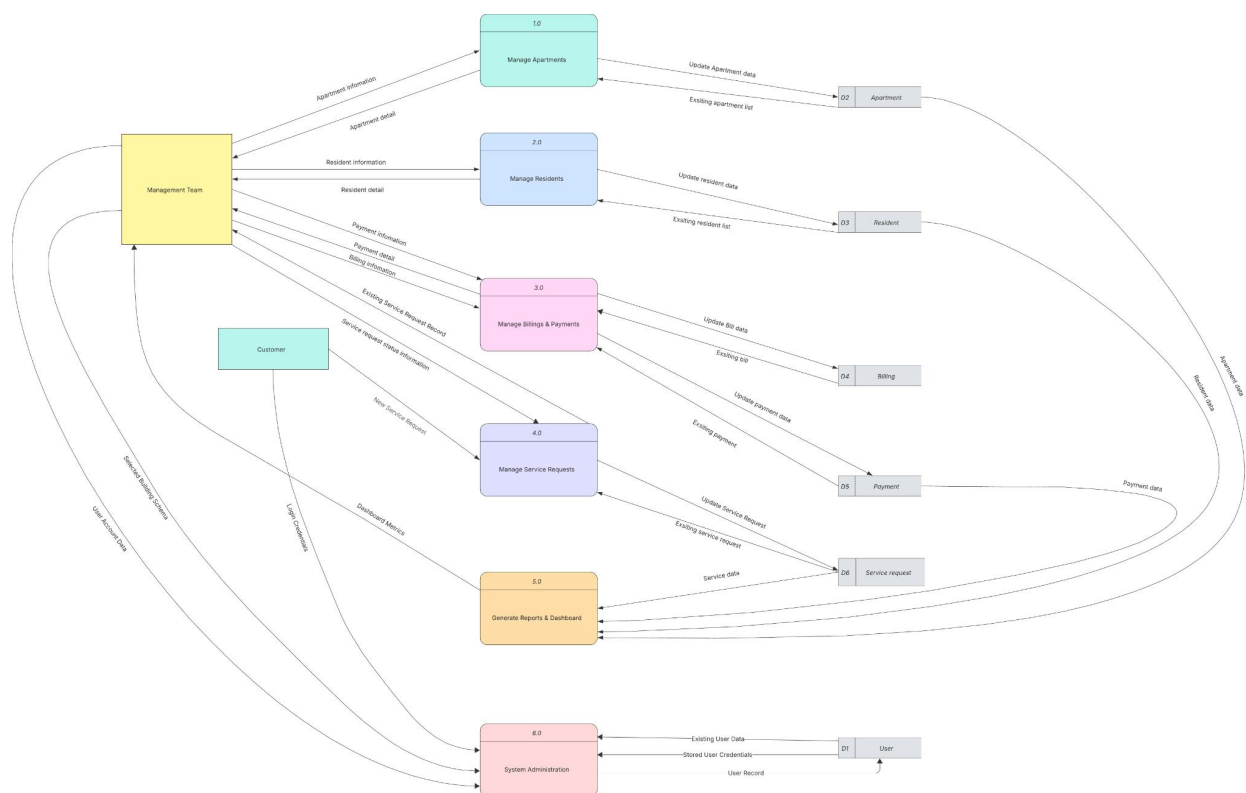


Figure 8: DFD Level 0 Diagram

Major Processes:

The Level-0 DFD identifies the following distinct high-level processes within the Apartment Management System:

- 1.0 Manage Apartments: Handles all operations related to the creation, modification, and tracking of apartment unit information.
- 2.0 Manage Residents: Manages resident profiles, including registration, updates, and associations with apartments.
- 3.0 Manage Bills & Payments: Deals with the generation of bills, recording of payments, and tracking of financial statuses.
- 4.0 Manage Service Requests: Processes service requests initiated by residents, from submission to resolution.
- 5.0 Generate Reports & Dashboard: Aggregates data from various parts of the system to produce reports and display key performance indicators on the dashboard.

- 6.0 System Administration: Manages user accounts, system access, and overall system configurations.

Data Stores:

The system utilizes the following data stores to persist information. Following the convention provided (D1 for User), the data stores are identified as:

- D1 User: Stores user credentials, roles, and access records.
- D2 Apartment: Contains detailed information about each apartment unit, such as ID, building, status, and population.
- D3 Resident: Stores comprehensive information about residents, including personal details, contact information, and associated apartment.
- D4 Billing: Holds records of all generated bills, including bill type, amount, due dates, and status.
- D5 Payment: Stores details of payment transactions, including payment ID, amount, date, and method.
- D6 Service Request: Contains information on all service requests, including request ID, category, description, status, and relevant dates.

Data Flows and Process Descriptions:

1. Process 1.0: Manage Apartments

- Inputs from Management Team: Apartment Information (for creating or updating apartments).
- Data Store Interactions:
 - Reads Existing Apartment Data from D2 Apartment.
 - Writes Update Apartment Data to D2 Apartment.
- Outputs to Management Team: Apartment Details.
- Outputs to Process 5.0: Apartment Data (for reporting).

2. Process 2.0: Manage Residents

- Inputs from Management Team: Resident Information (for new registrations or updates).
- Data Store Interactions:
 - Reads Existing Resident Data from D3 Resident.
 - Writes Update Resident Data to D3 Resident.
- Outputs to Management Team: Resident Detail.

- Outputs to Process 5.0: Resident Data (for reporting).
- 3. Process 3.0: Manage Bills & Payments**
 - Inputs from Management Team: Billing Information (for bill generation), Payment Information (for recording payments).
 - Data Store Interactions:
 - Reads Existing Bill Data from D4 Billing.
 - Writes Update Bill Data to D4 Billing.
 - Reads Existing Payment Data from D5 Payment.
 - Writes Update Payment Data to D5 Payment.
 - Outputs to Management Team: Bill Detail, Payment Detail.
 - Outputs to Process 5.0: Billing Data, Payment Data (for reporting).
- 4. Process 4.0: Manage Service Requests**
 - Inputs from Customer (Resident): Submit Service Request.
 - Data Store Interactions:
 - Reads Existing Service Request from D6 Service Request.
 - Writes Update Service Request to D6 Service Request.
 - Outputs to Customer (Resident): Service Details.
 - Outputs to Management Team: Service Request Detail.
 - Outputs to Process 5.0: Service Request Data (for reporting).
- 5. Process 5.0: Generate Reports & Dashboard**
 - Inputs from Data Stores: Reads Apartment Data from D2 Apartment, Resident Data from D3 Resident, Billing Data from D4 Billing, Payment Data from D5 Payment, Service Request Data from D6 Service Request.
 - Outputs to Management Team: Dashboard Metrics.
- 6. Process 6.0: System Administration**
 - Inputs from Management Team: User Access Data, System Configuration.
 - Data Store Interactions:
 - Reads Existing User Data from D1 User.
 - Writes Stored User Credentials and User Access Record to D1 User.

This Level-0 DFD clarifies how the primary functions of the Apartment Management System are interconnected and how they interact with the defined data stores and external entities. Further decomposition into Level-1 DFDs for each of these major processes would provide more granular detail on their internal workings.

3.2.3 Level-1 DFD

Level-1 DFDs provide a more detailed breakdown of each major process identified in the Level-0 DFD. Each Level-1 DFD focuses on a single process from the Level-0 diagram and illustrates its internal sub-processes, data flows, and interactions with data stores.

1. Level-1 DFD for Process 1.0: Manage Apartments

This diagram details the internal workings of the "Manage Apartments" process.

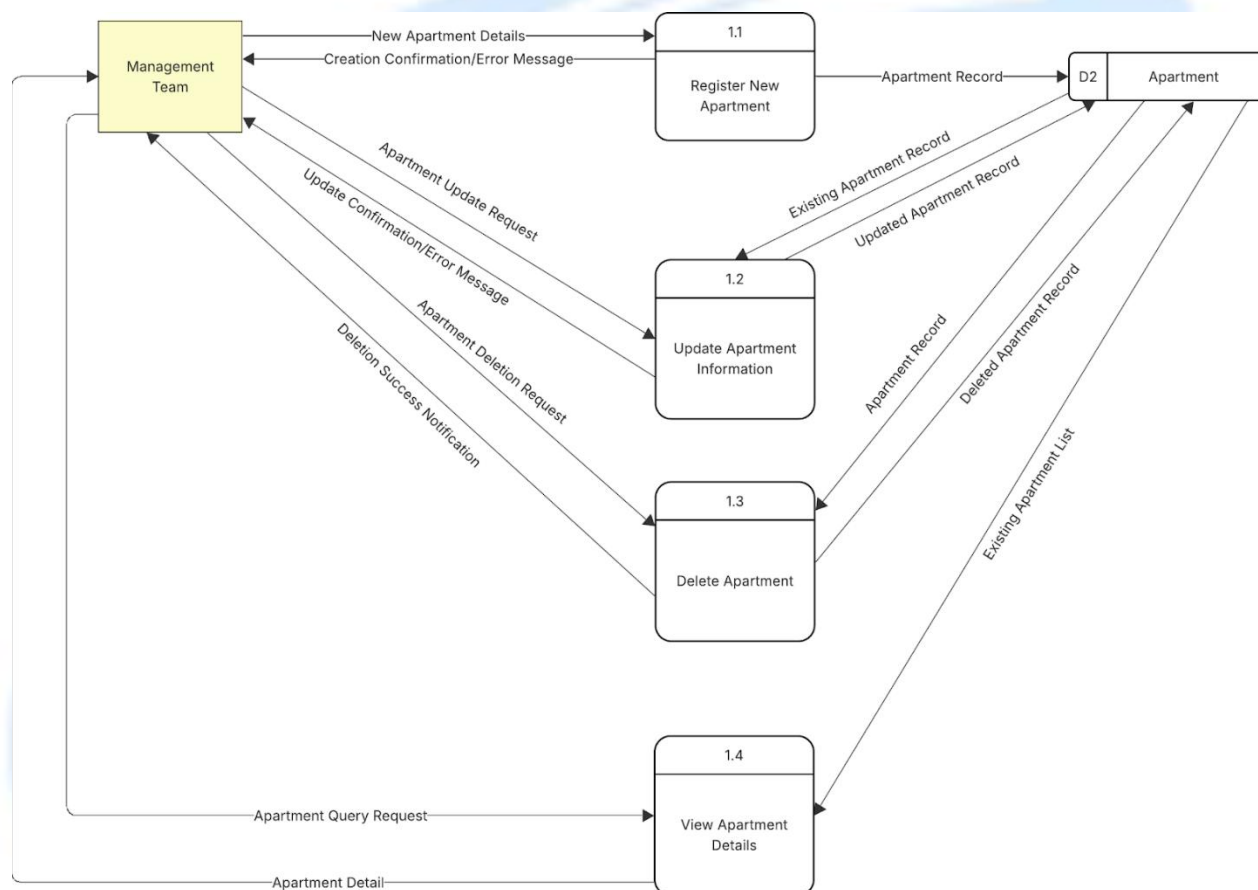


Figure 9: Level-1 DFD for Process 1.0

- **Sub-processes:**

- **1.1 Register New Apartment:** Handles the creation of new apartment records.

- **1.2 Update Apartment Information:** Manages modifications to existing apartment data.
- **1.3 Delete Apartment:** Processes the removal of apartment records.
- **1.4 View Apartment Details:** Allows for querying and displaying apartment information.
- **Data Flows and Interactions:**
 - **Management Team** provides New Apartment Details to sub-process **1.1**.
 - Sub-process **1.1** sends Apartment Record to Data Store **D2 Apartment** and returns Creation Confirmation/Error Message to the **Management Team**.
 - **Management Team** sends Apartment Update Request to sub-process **1.2**.
 - Sub-process **1.2** reads Existing Apartment Record from **D2 Apartment**, writes Updated Apartment Record to **D2 Apartment**, and sends Update Confirmation/Error Message to the **Management Team**.
 - **Management Team** initiates an Apartment Deletion Request to sub-process **1.3**.
 - Sub-process **1.3** reads Apartment Record from **D2 Apartment**, sends Deleted Apartment Record to **D2 Apartment** (logically, this would be a deletion operation against the data store), and returns Deletion Success Notification to the **Management Team**.
 - **Management Team** sends an Apartment Query Request to sub-process **1.4**.
 - Sub-process **1.4** reads Existing Apartment List from **D2 Apartment** and provides Apartment Detail to the **Management Team**.

2. Level-1 DFD for Process 2.0: Manage Residents

This diagram elaborates on the "Manage Residents" process.

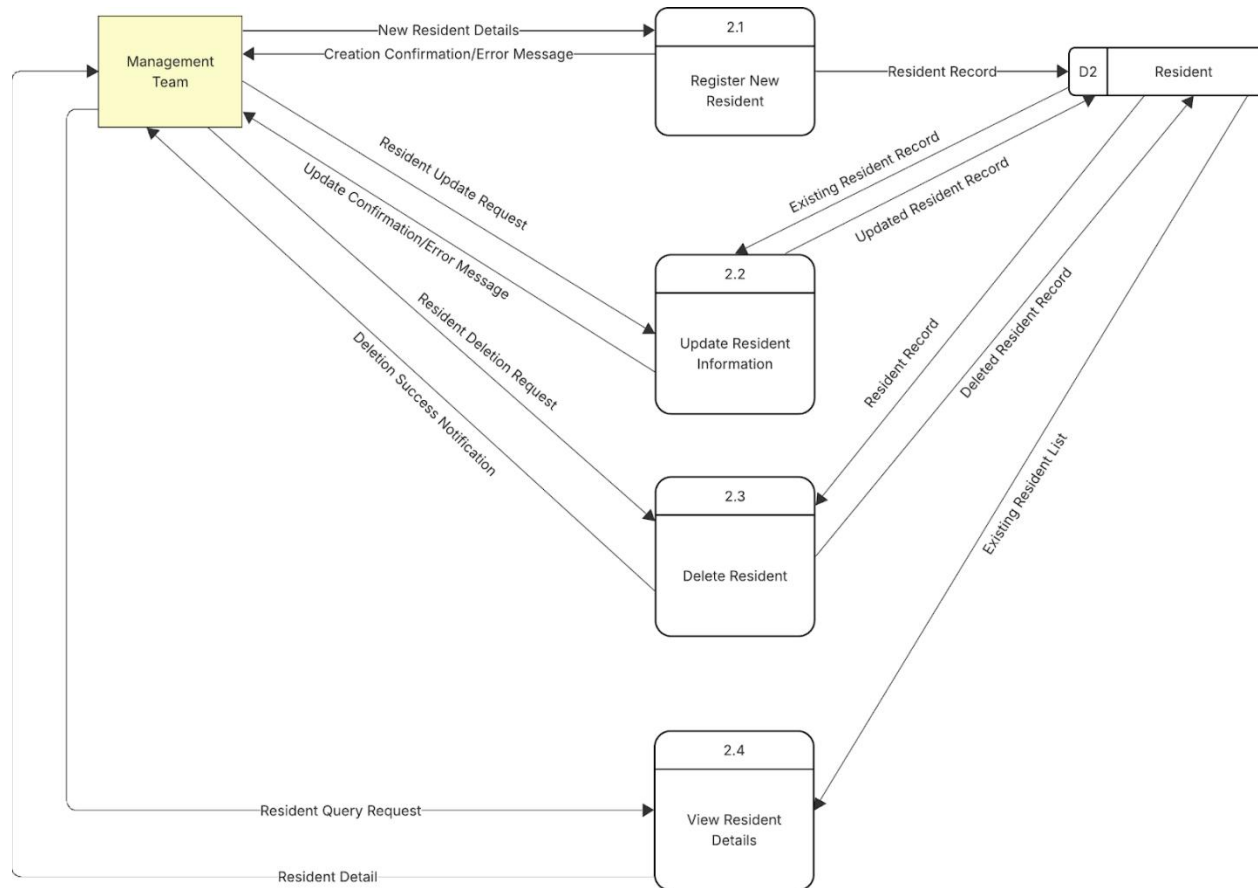


Figure 10: Level-1 DFD for Process 2.0

- **Sub-processes:**

- **2.1 Register New Resident:** Handles the addition of new resident profiles.
- **2.2 Update Resident Information:** Manages changes to existing resident data.
- **2.3 Delete Resident:** Processes the removal of resident records.
- **2.4 View Resident Details:** Enables querying and viewing of resident information.

- **Data Flows and Interactions:**

- **Management Team** submits New Resident Details to sub-process **2.1**.
- Sub-process **2.1** sends Resident Record to Data Store **D3 Resident** (the diagram shows D2, but based on our convention, it should be D3 Resident) and returns Creation Confirmation/Error Message to the **Management Team**.
- **Management Team** sends Resident Update Request to sub-process **2.2**.
- Sub-process **2.2** reads Existing Resident Record from **D3 Resident**, writes Updated Resident Record to **D3 Resident**, and sends Update Confirmation/Error Message to the **Management Team**.
- **Management Team** initiates a Resident Deletion Request to sub-process **2.3**.
- Sub-process **2.3** reads Resident Record from **D3 Resident**, sends Deleted Resident Record to **D3 Resident** (deletion operation), and returns Deletion Success Notification to the **Management Team**.
- **Management Team** sends a Resident Query Request to sub-process **2.4**.
- Sub-process **2.4** reads Existing Resident List from **D3 Resident** and provides Resident Detail to the **Management Team**.

3. Level-1 DFD for Process 3.0: Manage Bills & Payments

This diagram breaks down the "Manage Bills & Payments" process.

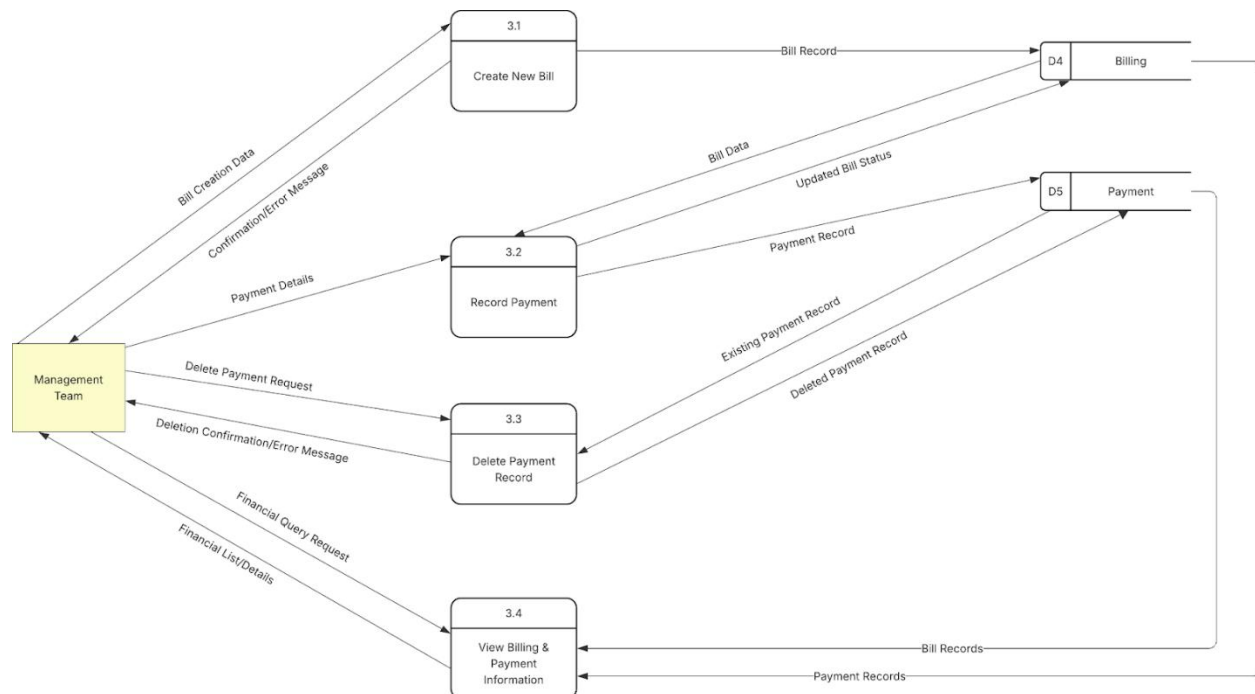


Figure 11: Level-1 DFD for Process 3.0

- **Sub-processes:**

- **3.1 Create New Bill:** Responsible for generating new bills for residents.
- **3.2 Record Payment:** Handles the recording of payments made by residents.
- **3.3 Delete Payment Record:** Manages the removal of payment records (potentially for corrections).
- **3.4 View Billing & Payment Information:** Allows for querying financial records.

- **Data Flows and Interactions:**

- **Management Team** provides Bill Creation Data to sub-process **3.1**.

- Sub-process **3.1** sends Bill Record to Data Store **D4 Billing** and returns Confirmation/Error Message to the **Management Team**.
- **Management Team** submits Payment Details to sub-process **3.2**.
- Sub-process **3.2** reads Bill Data from **D4 Billing**, writes Payment Record to Data Store **D5 Payment**, and sends Updated Bill Status to **D4 Billing**.
- **Management Team** sends Delete Payment Request to sub-process **3.3**.
- Sub-process **3.3** reads Existing Payment Record from **D5 Payment**, sends Deleted Payment Record to **D5 Payment** (deletion operation), and returns Deletion Confirmation/Error Message to the **Management Team**.
- **Management Team** sends a Financial Query Request to sub-process **3.4**.
- Sub-process **3.4** reads Bill Records from **D4 Billing** and Payment Records from **D5 Payment**, and provides Financial List/Details to the **Management Team**.

4. Level-1 DFD for Process 4.0: Manage Service Requests

This diagram details the "Manage Service Requests" process.

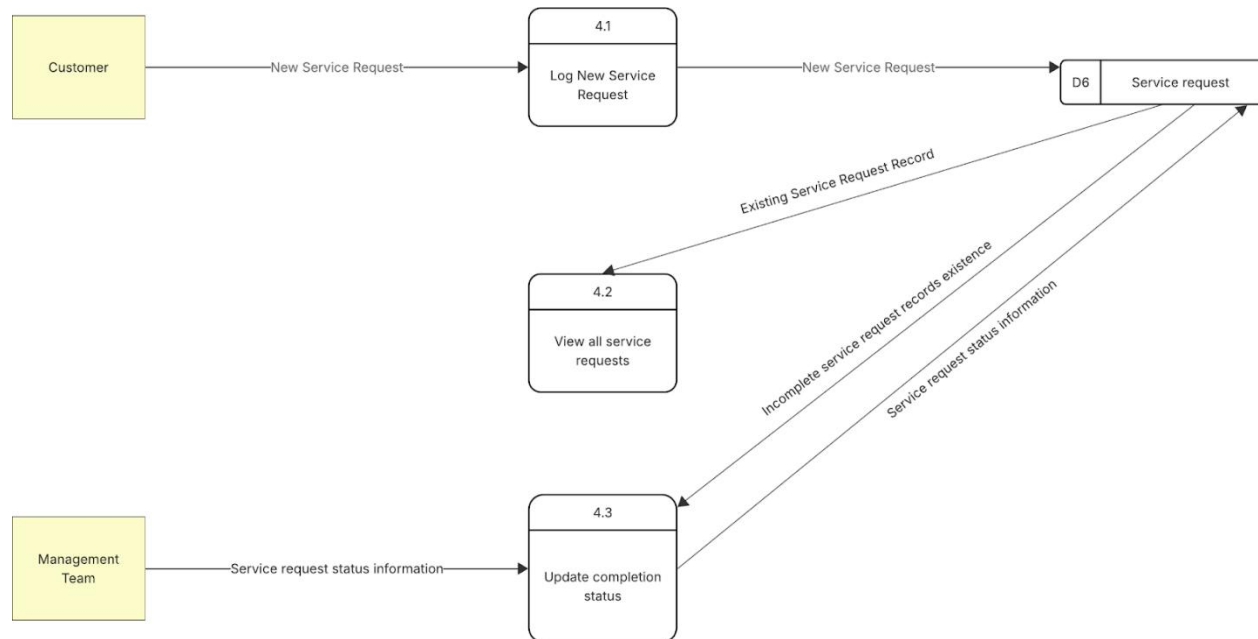


Figure 12: Level-1 DFD for Process 4.0

- **Sub-processes:**

- **4.1 Log New Service Request:** Captures and records new service requests from customers.
- **4.2 View All Service Requests:** Allows management to see existing service requests.
- **4.3 Update Completion Status:** Manages the status updates of service requests.

- **Data Flows and Interactions:**

- **Customer** submits New Service Request to sub-process **4.1**.
- Sub-process **4.1** sends New Service Request (data) to Data Store **D6 Service Request**.

- Sub-process **4.2** reads Existing Service Request Record from **D6 Service Request**. (The diagram implies this flow is internal for viewing, not directly outputting to an entity from 4.2).
- **Management Team** provides Service request status information to sub-process **4.3**.
- Sub-process **4.3** reads Incomplete service request records existence and Service request status information (likely specific record to update) from **D6 Service Request** and writes Service request status information (updated status) back to **D6 Service Request**.

5. Level-1 DFD for Process 5.0: Generate Reports & Dashboard

This diagram explains the "Generate Reports & Dashboard" process.

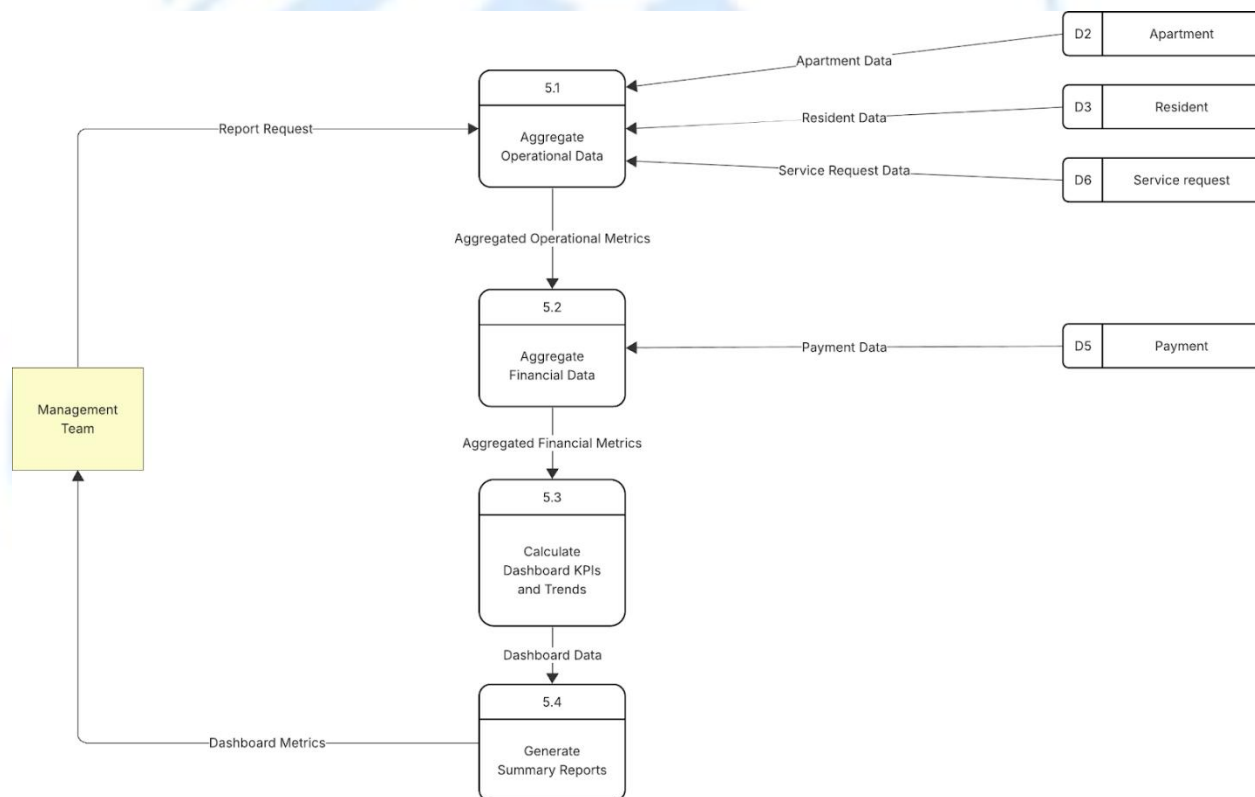


Figure 13: Level-1 DFD for Process 5.0

- **Sub-processes:**

- **5.1 Aggregate Operational Data:** Gathers non-financial data for reporting.
- **5.2 Aggregate Financial Data:** Gathers financial data for reporting.
- **5.3 Calculate Dashboard KPIs and Trends:** Processes aggregated data to compute metrics.
- **5.4 Generate Summary Reports:** Creates formatted reports for management.
- **Data Flows and Interactions:**
 - **Management Team** sends Report Request to sub-process **5.1**.
 - Sub-process **5.1** reads Apartment Data from **D2 Apartment**, Resident Data from **D3 Resident**, and Service Request Data from **D6 Service Request**. It then outputs Aggregated Operational Metrics to sub-process **5.3**.
 - Sub-process **5.2** (implicitly triggered or part of overall report generation) reads Payment Data from **D5 Payment**. It then outputs Aggregated Financial Metrics to sub-process **5.3**.
 - Sub-process **5.3** processes the aggregated metrics and sends Dashboard Data to sub-process **5.4**.
 - Sub-process **5.4** generates reports and sends Dashboard Metrics to the **Management Team**.

6. Level-1 DFD for Process 6.0: System Administration

This diagram details the "System Administration" process.

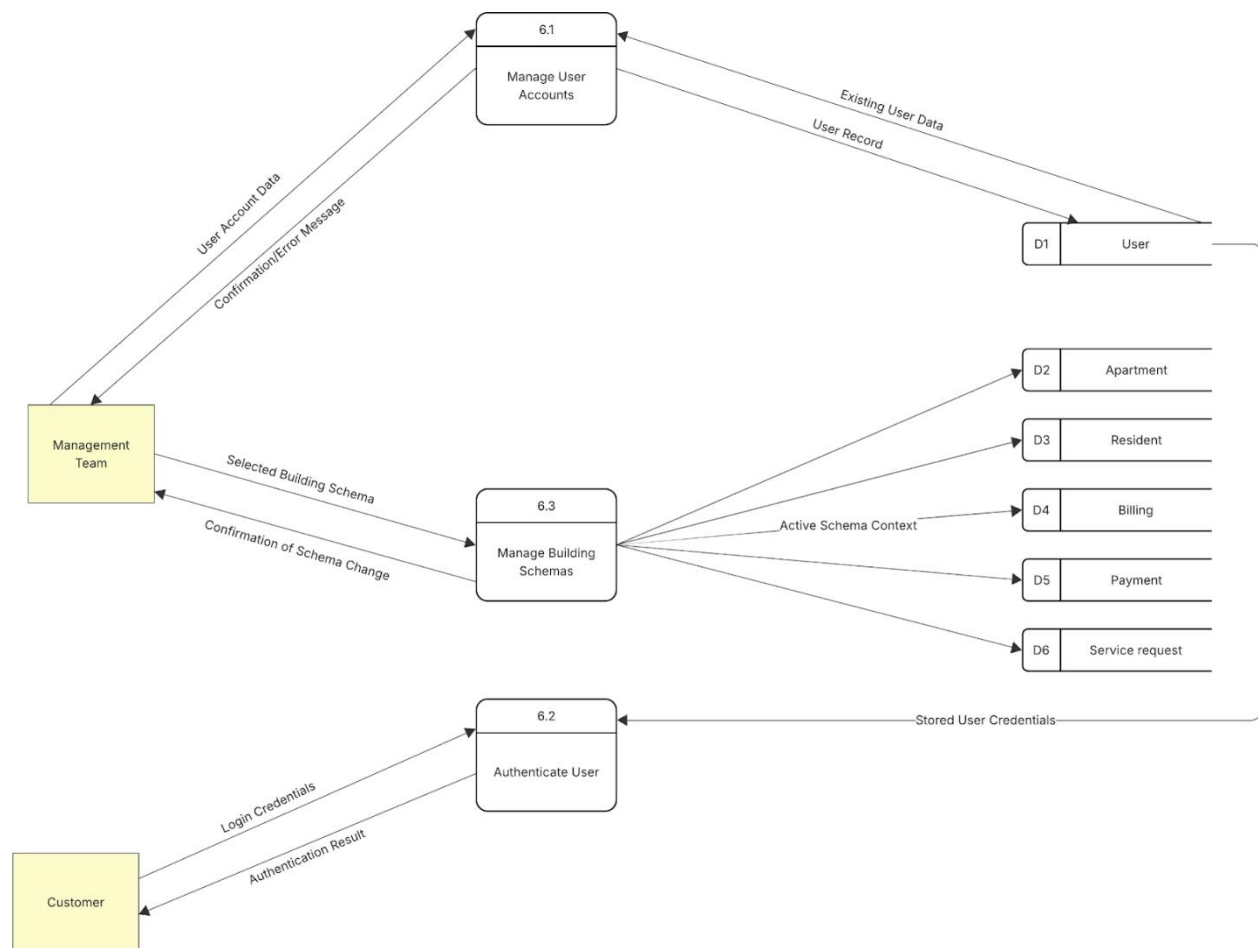


Figure 14: Level-1 DFD for Process 6.0

- **Sub-processes:**

- **6.1 Manage User Accounts:** Handles user creation, modification, and deletion.
- **6.2 Authenticate User:** Verifies user credentials for system access.
- **6.3 Manage Building Schemas:** Allows selection and switching between different building data schemas.

- **Data Flows and Interactions:**

- **Management Team** provides User Account Data to sub-process **6.1**.

- Sub-process **6.1** reads Existing User Data from Data Store **D1 User**, writes User Record to **D1 User**, and returns Confirmation/Error Message to the **Management Team**.
- **Customer** submits Login Credentials to sub-process **6.2**.
- Sub-process **6.2** reads Stored User Credentials from **D1 User** and returns Authentication Result to the **Customer**.
- **Management Team** provides Selected Building Schema to sub-process **6.3**.
- Sub-process **6.3** sends Active Schema Context to Data Stores **D2 Apartment**, **D3 Resident**, **D4 Billing**, **D5 Payment**, **D6 Service Request** (conceptually, this means subsequent queries to these data stores will be filtered/scoped by this schema). It returns Confirmation of Schema Change to the **Management Team**.

These Level-1 DFDs provide a clearer insight into the specific functionalities and data movements within each major component of the Apartment Management System.

3.2.4 Level-2 DFD

Level-2 DFDs offer a more granular view by decomposing selected sub-processes from the Level-1 DFDs. This level of detail helps in understanding the specific steps and data transformations occurring within more complex Level-1 sub-processes.

1. Level-2 DFD for Sub-process 1.1: Register New Apartment

This diagram details the internal steps of "1.1 Register New Apartment".

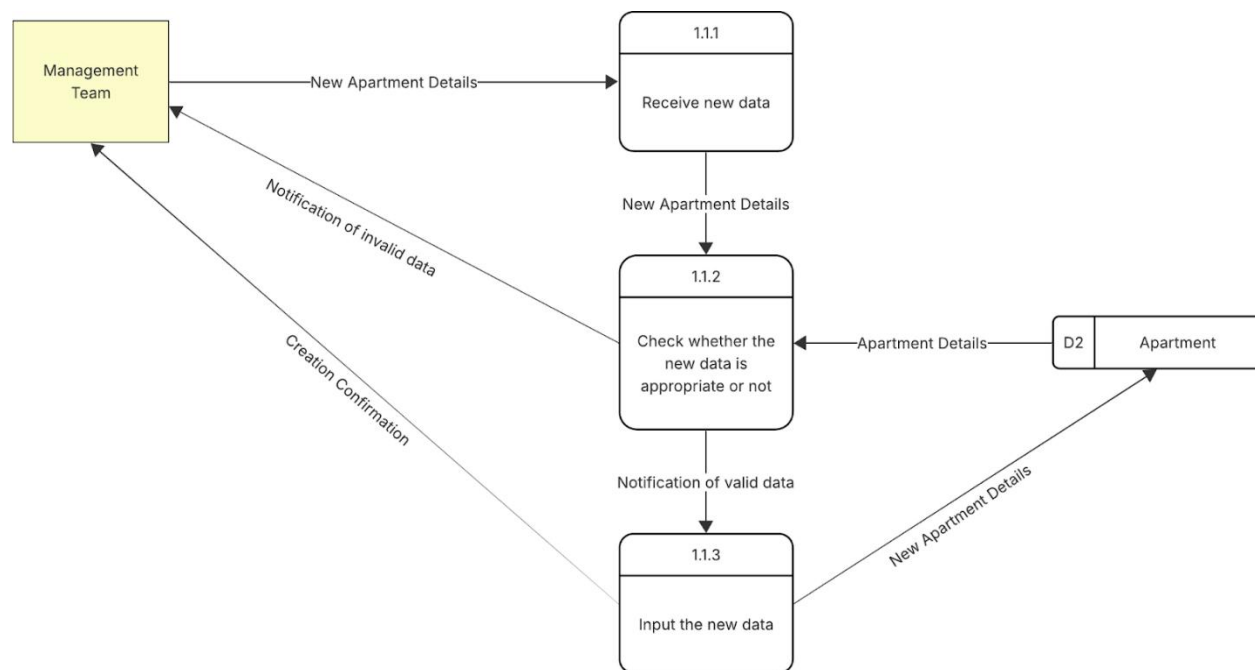


Figure 15: Level-2 DFD for Sub-process 1.1

- **Sub-processes:**

- **1.1.1 Receive New Data:** Accepts the initial apartment details from the Management Team.
- **1.1.2 Check Whether the New Data is Appropriate or Not:** Validates the received apartment data against predefined rules and existing records.
- **1.1.3 Input the New Data:** Persists the validated new apartment data into the data store.

- **Data Flows and Interactions:**

- **Management Team** provides New Apartment Details to sub-process **1.1.1**.
- Sub-process **1.1.1** passes New Apartment Details to sub-process **1.1.2**.
- Sub-process **1.1.2** reads Apartment Details (for checking duplicates or related info) from Data Store **D2 Apartment**.

- If data is invalid, sub-process **1.1.2** sends Notification of Invalid Data to the **Management Team**.
- If data is valid, sub-process **1.1.2** sends Notification of Valid Data to sub-process **1.1.3**.
- Sub-process **1.1.3** writes New Apartment Details to Data Store **D2 Apartment** and sends Creation Confirmation to the **Management Team**.

2. Level-2 DFD for Sub-process 1.2: Update Apartment Information

This diagram breaks down "1.2 Update Apartment Information".

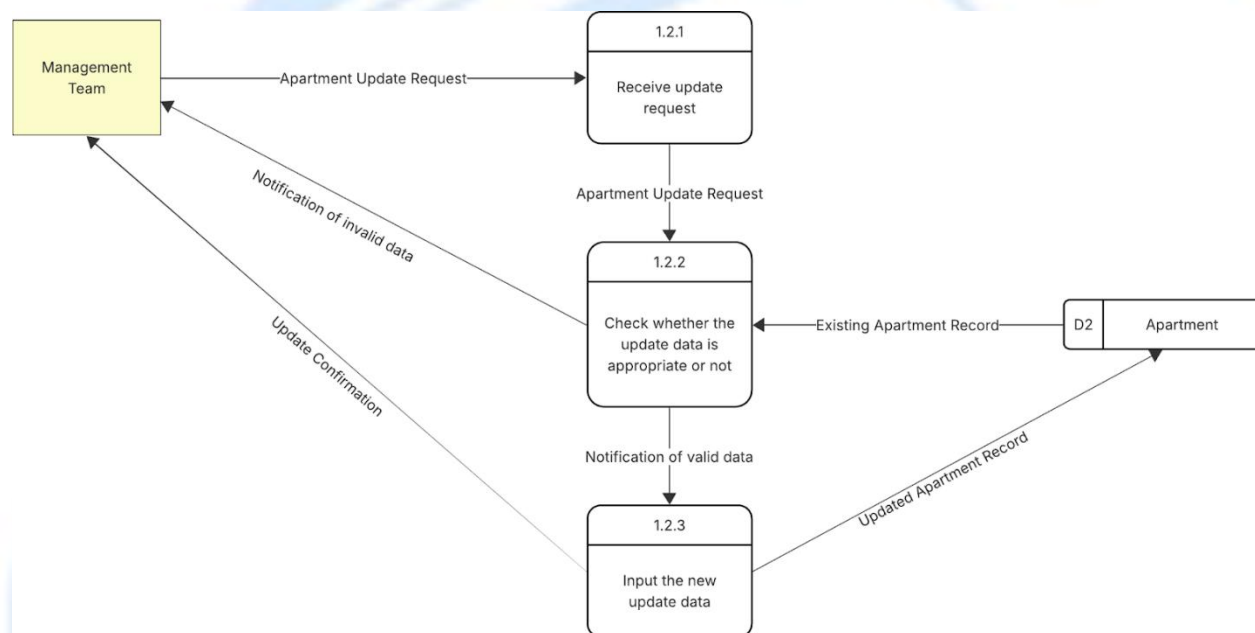


Figure 16: Level-2 DFD for Sub-process 1.2

- **Sub-processes:**

- **1.2.1 Receive Update Request:** Accepts the apartment update request and data.
- **1.2.2 Check Whether the Update Data is Appropriate or Not:** Validates the new information.

- **1.2.3 Input the New Update Data:** Saves the updated apartment information.
- **Data Flows and Interactions:**
 - **Management Team** provides Apartment Update Request to sub-process **1.2.1**.
 - Sub-process **1.2.1** passes Apartment Update Request (containing data) to sub-process **1.2.2**.
 - Sub-process **1.2.2** reads Existing Apartment Record from Data Store **D2 Apartment**.
 - If data is invalid, sub-process **1.2.2** sends Notification of Invalid Data to the **Management Team**.
 - If data is valid, sub-process **1.2.2** sends Notification of Valid Data to sub-process **1.2.3**.
 - Sub-process **1.2.3** writes Updated Apartment Record to Data Store **D2 Apartment** and sends Update Confirmation to the **Management Team**.

3. Level-2 DFD for Sub-process 1.3: Delete Apartment

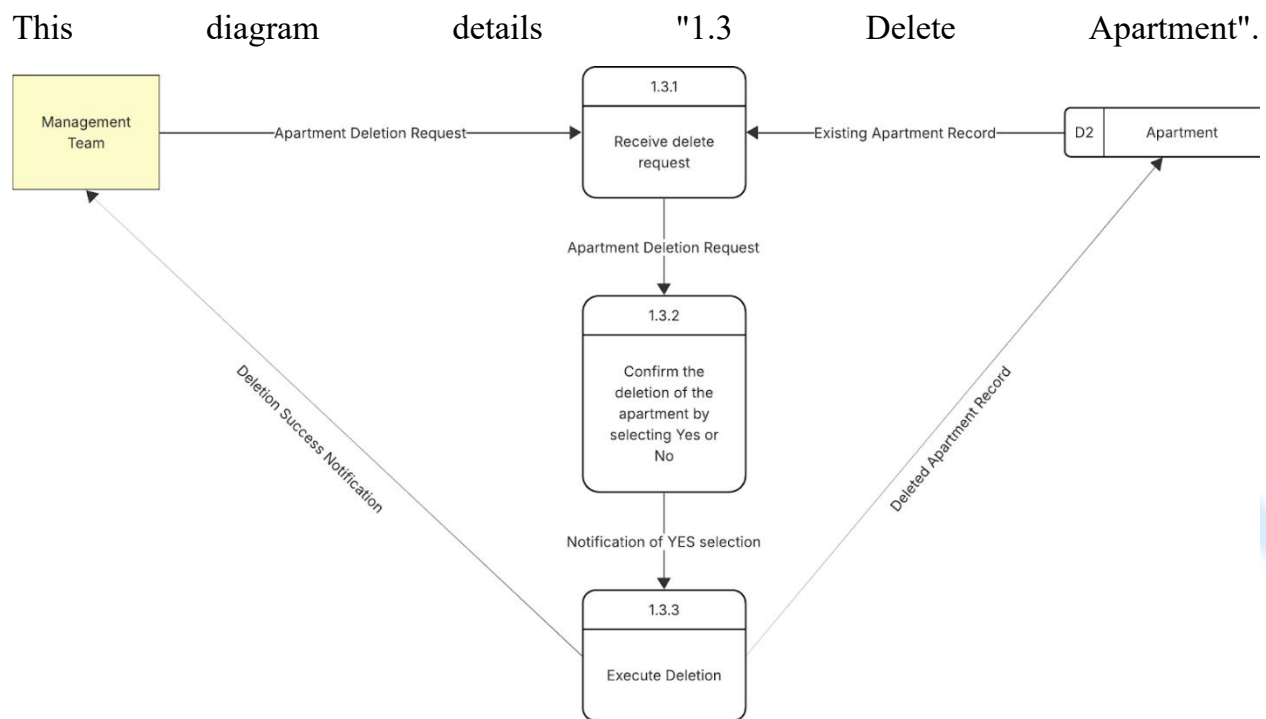


Figure 17: Level-2 DFD for Sub-process 1.3

- **Sub-processes:**

- **1.3.1 Receive Delete Request:** Accepts the request to delete an apartment.
- **1.3.2 Confirm the Deletion of the Apartment by Selecting Yes or No:** Prompts for and receives confirmation for the deletion.
- **1.3.3 Execute Deletion:** Performs the actual deletion from the data store.

- **Data Flows and Interactions:**

- **Management Team** sends Apartment Deletion Request to sub-process **1.3.1**.
- Sub-process **1.3.1** reads Existing Apartment Record from Data Store **D2 Apartment** and passes Apartment Deletion Request (likely with details of the record to be deleted) to sub-process **1.3.2**.

- Sub-process **1.3.2** (after user confirmation, not explicitly shown as a data flow from Management Team at this stage, but implied by the process name) sends Notification of YES selection to sub-process **1.3.3**.
- Sub-process **1.3.3** sends Deleted Apartment Record (instruction to delete) to Data Store **D2 Apartment** and returns Deletion Success Notification to the **Management Team**.

4. Level-2 DFD for Sub-process 2.1: Register New Resident

This diagram breaks down "2.1 Register New Resident".

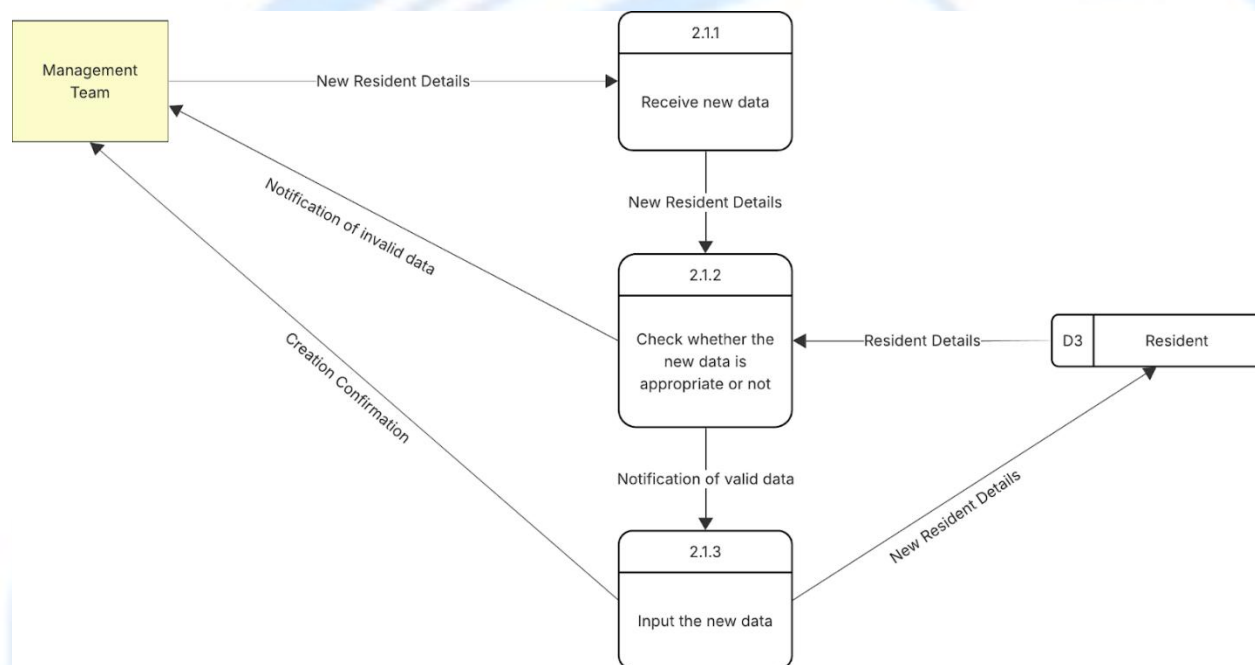


Figure 18: Level-2 DFD for Sub-process 2.1

- **Sub-processes:**

- **2.1.1 Receive New Data:** Accepts initial resident details.
- **2.1.2 Check Whether the New Data is Appropriate or Not:** Validates the resident data.
- **2.1.3 Input the New Data:** Persists the new resident data.

- **Data Flows and Interactions:**

- **Management Team** provides New Resident Details to sub-process **2.1.1**.
- Sub-process **2.1.1** passes New Resident Details to sub-process **2.1.2**.
- Sub-process **2.1.2** reads Resident Details (for validation) from Data Store **D3 Resident**.
- If data is invalid, sub-process **2.1.2** sends Notification of Invalid Data to the **Management Team**.
- If data is valid, sub-process **2.1.2** sends Notification of Valid Data to sub-process **2.1.3**.
- Sub-process **2.1.3** writes New Resident Details to Data Store **D3 Resident** and sends Creation Confirmation to the **Management Team**.

5. Level-2 DFD for Sub-process 2.2: Update Resident Information

This diagram details "2.2 Update Resident Information".

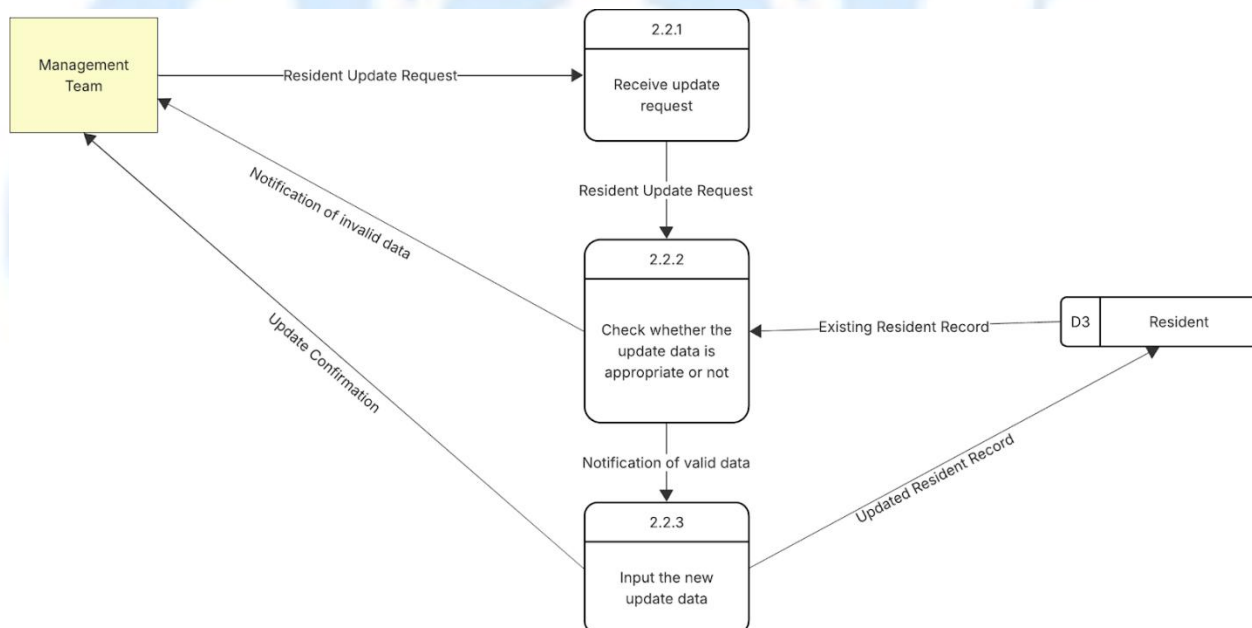


Figure 19: Level-2 DFD for Sub-process 2.2

- **Sub-processes:**

- **2.2.1 Receive Update Request:** Accepts the resident update request.
- **2.2.2 Check Whether the Update Data is Appropriate or Not:** Validates the updated information.
- **2.2.3 Input the New Update Data:** Saves the modified resident data.

- **Data Flows and Interactions:**

- **Management Team** provides Resident Update Request to sub-process **2.2.1**.
- Sub-process **2.2.1** passes Resident Update Request to sub-process **2.2.2**.
- Sub-process **2.2.2** reads Existing Resident Record from Data Store **D3 Resident**.
- If data is invalid, sub-process **2.2.2** sends Notification of Invalid Data to the **Management Team**.
- If data is valid, sub-process **2.2.2** sends Notification of Valid Data to sub-process **2.2.3**.
- Sub-process **2.2.3** writes Updated Resident Record to Data Store **D3 Resident** and sends Update Confirmation to the **Management Team**.

6. Level-2 DFD for Sub-process 2.3: Delete Resident

This diagram breaks down "2.3 Delete Resident".

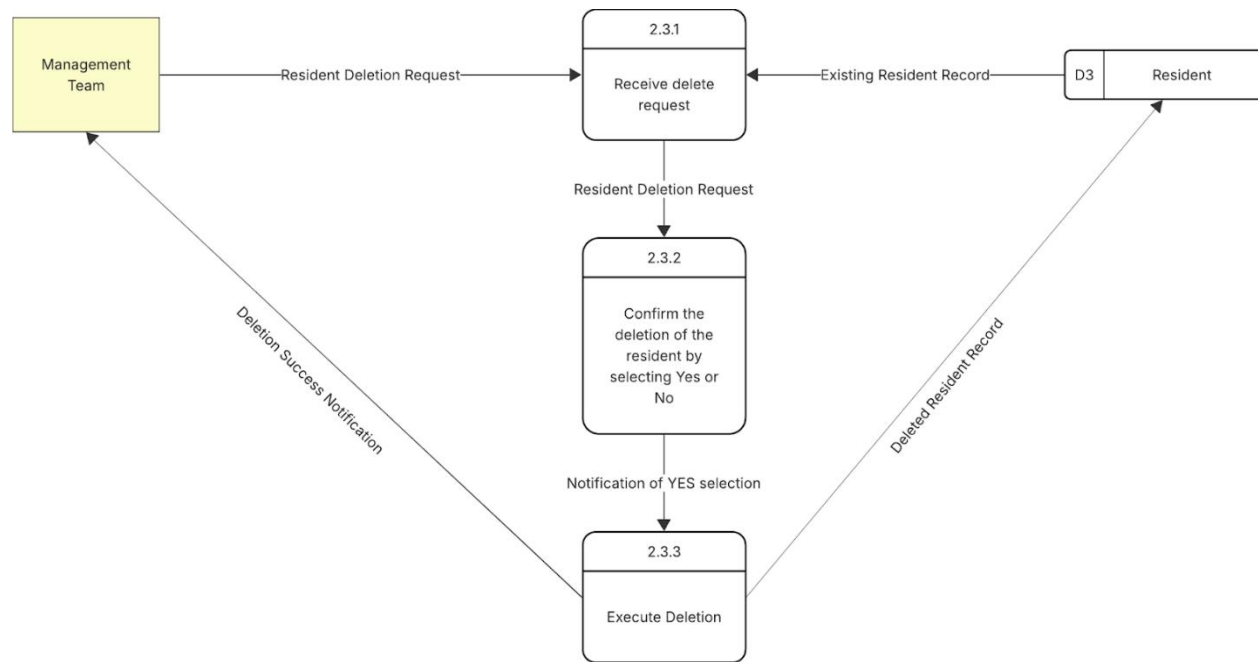


Figure 20: Level-2 DFD for Sub-process 2.3

- **Sub-processes:**

- **2.3.1 Receive Delete Request:** Accepts the request to delete a resident.
- **2.3.2 Confirm the Deletion of the Resident by Selecting Yes or No:** Handles deletion confirmation.
- **2.3.3 Execute Deletion:** Performs the deletion from the data store.

- **Data Flows and Interactions:**

- **Management Team** sends Resident Deletion Request to sub-process **2.3.1**.
- Sub-process **2.3.1** reads Existing Resident Record from Data Store **D3 Resident** and passes Resident Deletion Request to sub-process **2.3.2**.
- Sub-process **2.3.2** sends Notification of YES selection to sub-process **2.3.3**.

- Sub-process **2.3.3** sends Deleted Resident Record (instruction) to Data Store **D3 Resident** and returns Deletion Success Notification to the **Management Team**.

7. Level-2 DFD for Sub-process 3.1: Create New Bill

This diagram details "3.1 Create New Bill".

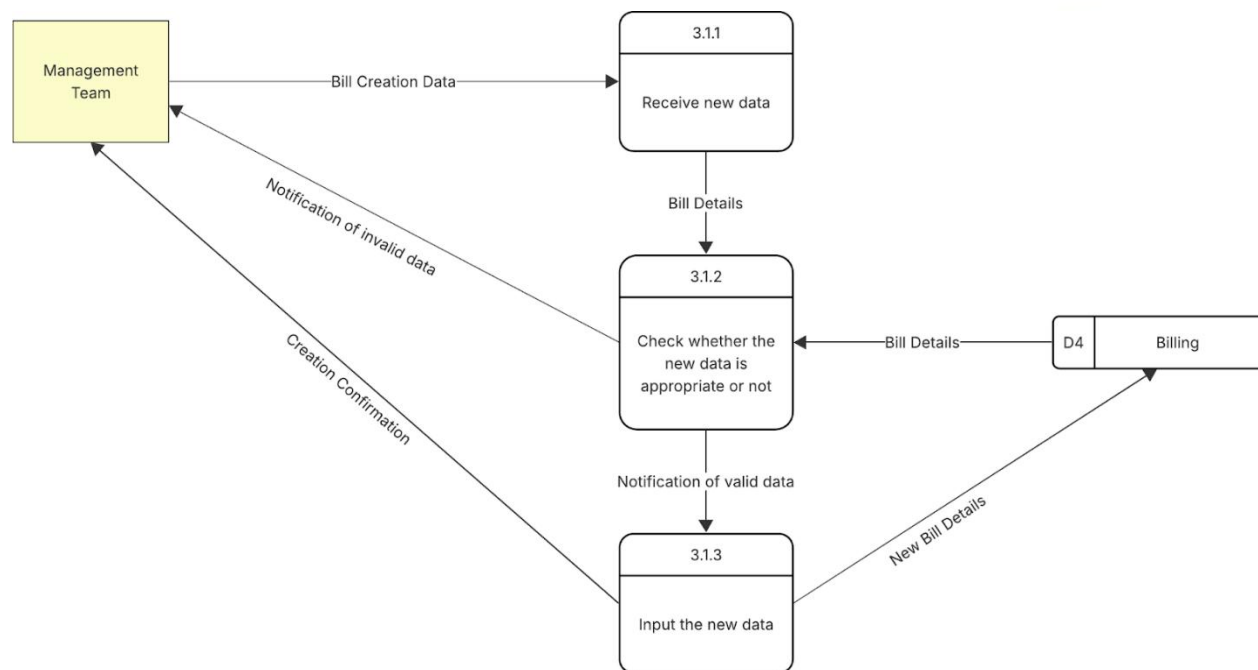


Figure 21: Level-2 DFD for Sub-process 3.1

- **Sub-processes:**

- **3.1.1 Receive New Data:** Accepts bill creation details.
- **3.1.2 Check Whether the New Data is Appropriate or Not:** Validates the bill information.
- **3.1.3 Input the New Data:** Persists the new bill.

- **Data Flows and Interactions:**

- **Management Team** provides Bill Creation Data to sub-process **3.1.1**.

- Sub-process **3.1.1** passes Bill Details to sub-process **3.1.2**.
- Sub-process **3.1.2** reads Bill Details (for validation) from Data Store **D4 Billing**.
- If data is invalid, sub-process **3.1.2** sends Notification of Invalid Data to the **Management Team**.
- If data is valid, sub-process **3.1.2** sends Notification of Valid Data to sub-process **3.1.3**.
- Sub-process **3.1.3** writes New Bill Details to Data Store **D4 Billing** and sends Creation Confirmation to the **Management Team**.

8. Level-2 DFD for Sub-process 3.2: Record Payment

This diagram elaborates on "3.2 Record Payment".

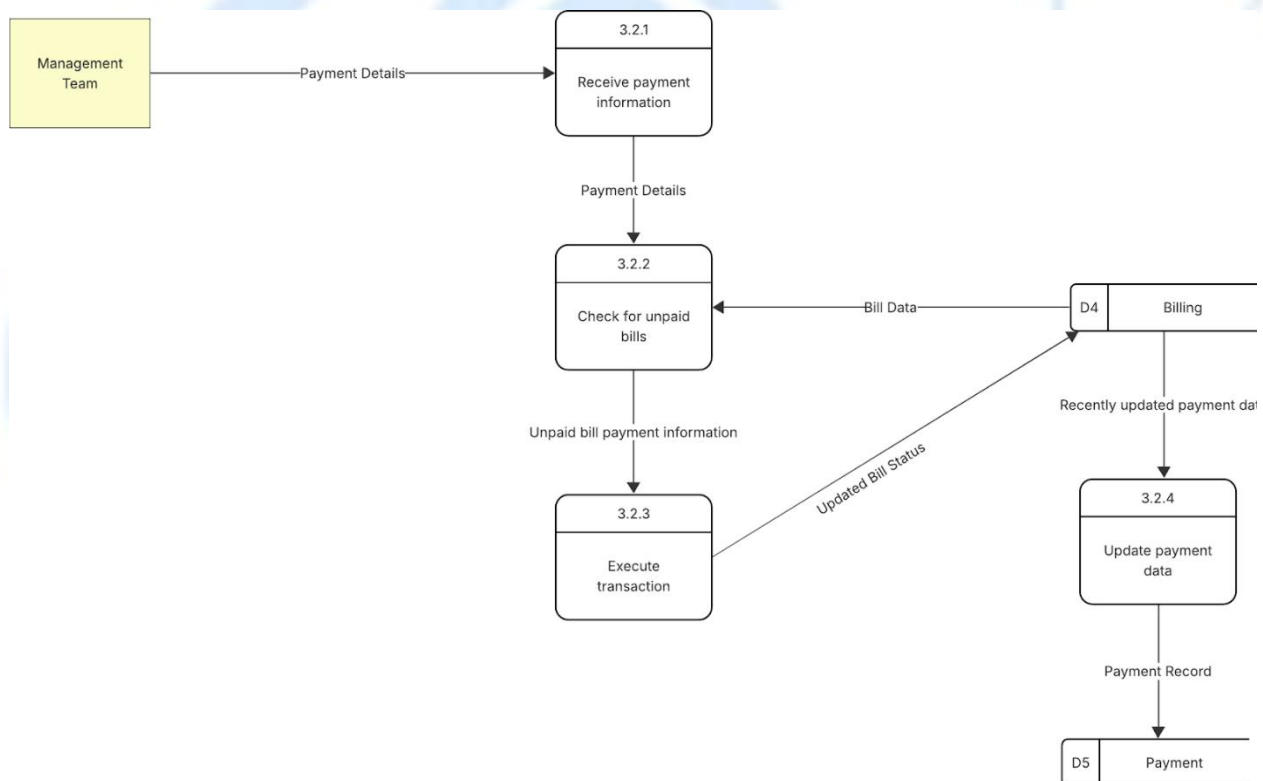


Figure 22: Level-2 DFD for Sub-process 3.2

- **Sub-processes:**

- **3.2.1 Receive Payment Information:** Accepts payment details from the Management Team.
- **3.2.2 Check for Unpaid Bills:** Verifies outstanding bills related to the payment.
- **3.2.3 Execute Transaction:** Processes the payment against the bills.
- **3.2.4 Update Payment Data:** Records the payment in the payment data store.

- **Data Flows and Interactions:**

- **Management Team** provides Payment Details to sub-process **3.2.1**.
- Sub-process **3.2.1** passes Payment Details to sub-process **3.2.2**.
- Sub-process **3.2.2** reads Bill Data from Data Store **D4 Billing** and passes Unpaid Bill Payment Information to sub-process **3.2.3**.
- Sub-process **3.2.3** sends Updated Bill Status to Data Store **D4 Billing** and also sends Recently Updated Payment Data (details of the payment to be recorded) to sub-process **3.2.4**.
- Sub-process **3.2.4** writes Payment Record to Data Store **D5 Payment**.

9. Level-2 DFD for Sub-process 3.3: Delete Payment Record

This diagram details "3.3 Delete Payment Record".

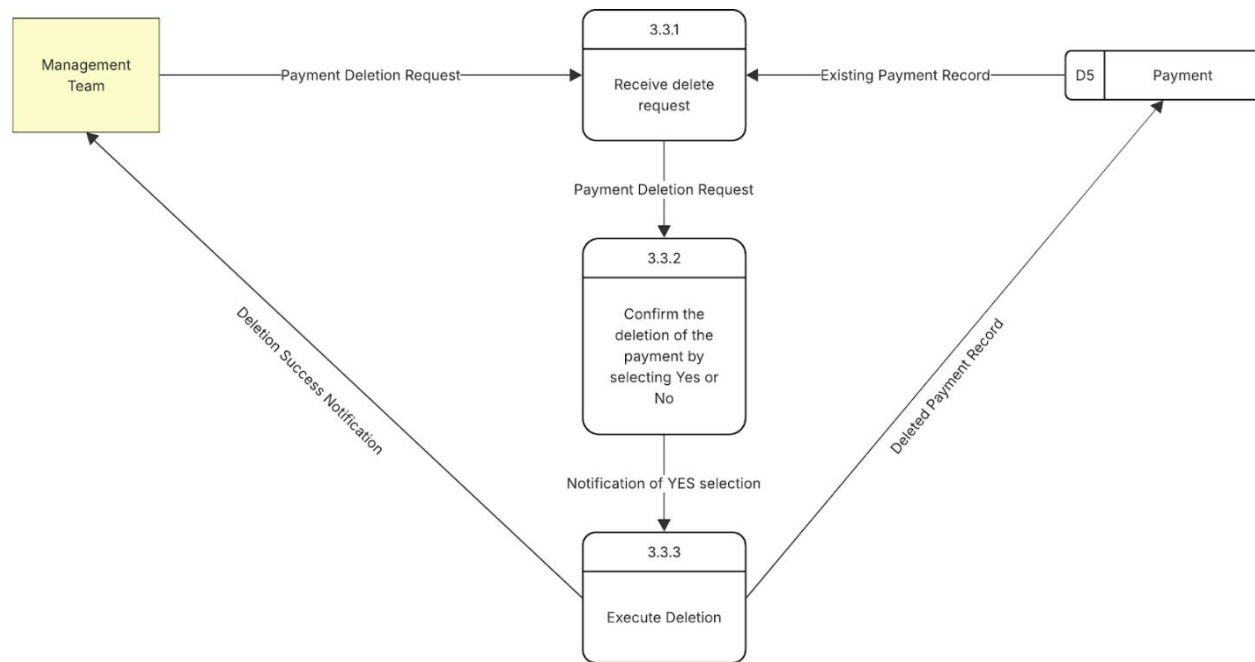


Figure 23: Level-2 DFD for Sub-process 3.3

- **Sub-processes:**

- **3.3.1 Receive Delete Request:** Accepts the request to delete a payment.
- **3.3.2 Confirm the Deletion of the Payment by Selecting Yes or No:** Handles deletion confirmation.
- **3.3.3 Execute Deletion:** Performs the deletion from the data store.

- **Data Flows and Interactions:**

- **Management Team** sends Payment Deletion Request to sub-process **3.3.1**.
- Sub-process **3.3.1** reads Existing Payment Record from Data Store **D5 Payment** and passes Payment Deletion Request to sub-process **3.3.2**.
- Sub-process **3.3.2** sends Notification of YES selection to sub-process **3.3.3**.

- Sub-process **3.3.3** sends Deleted Payment Record (instruction) to Data Store **D5 Payment** and returns Deletion Success Notification to the **Management Team**.

10. Level-2 DFD for Sub-process 4.3: Update Completion Status (Service Request)

This diagram breaks down the "4.3 Update Completion Status" for service requests.

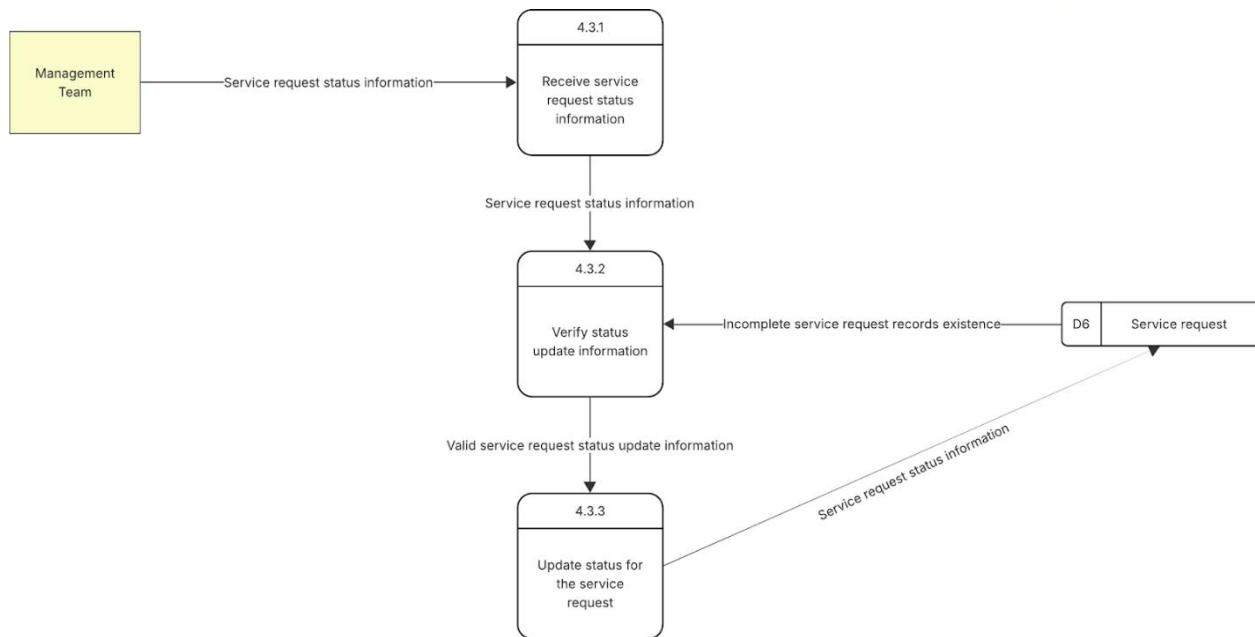


Figure 24: Level-2 DFD for Sub-process 4.3

- **Sub-processes:**

- **4.3.1 Receive Service Request Status Information:** Accepts status update details from the Management Team.
- **4.3.2 Verify Status Update Information:** Validates the received status against the existing request.
- **4.3.3 Update Status for the Service Request:** Persists the new status to the data store.

- **Data Flows and Interactions:**

- **Management Team** provides Service request status information to sub-process **4.3.1**.
- Sub-process **4.3.1** passes Service request status information to sub-process **4.3.2**.
- Sub-process **4.3.2** reads Incomplete service request records existence from Data Store **D6 Service Request** and passes Valid service request status update information to sub-process **4.3.3**.
- Sub-process **4.3.3** writes Service request status information (the updated status) to Data Store **D6 Service Request**.

11. Level-2 DFD for Sub-process 6.1: Manage User Accounts

This diagram details the internal steps of "6.1 Manage User Accounts" from the System Administration process.

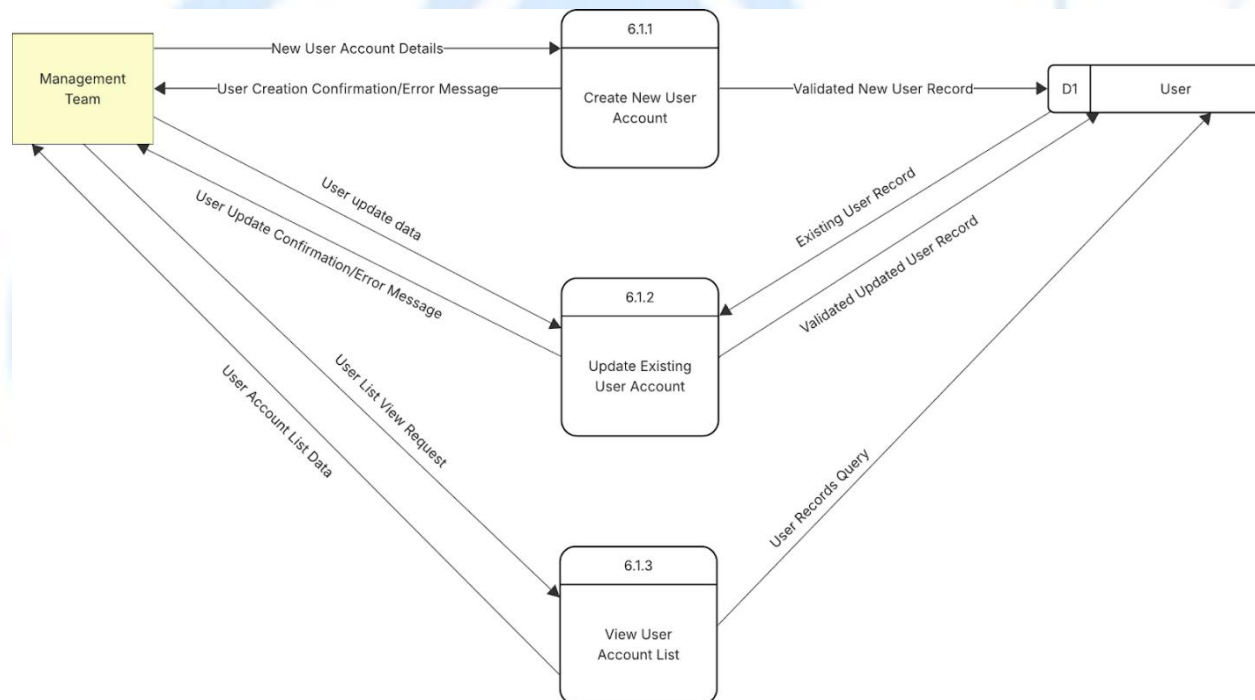


Figure 25: Level-2 DFD for Sub-process 6.1

- **Sub-processes:**

- **6.1.1 Create New User Account:** Handles the registration of new system users.
- **6.1.2 Update Existing User Account:** Manages modifications to existing user account details.
- **6.1.3 View User Account List:** Allows for querying and displaying a list of user accounts.
- **Data Flows and Interactions:**
 - **Management Team** provides New User Account Details to sub-process **6.1.1**.
 - Sub-process **6.1.1** sends Validated New User Record to Data Store **D1 User** and returns User Creation Confirmation/Error Message to the **Management Team**.
 - **Management Team** sends User update data to sub-process **6.1.2**.
 - Sub-process **6.1.2** reads Existing User Record from Data Store **D1 User**, writes Validated Updated User Record to Data Store **D1 User**, and sends User Update Confirmation/Error Message to the **Management Team**.
 - **Management Team** sends a User List View Request to sub-process **6.1.3**.
 - Sub-process **6.1.3** reads User Records Query (result of query) from Data Store **D1 User** and provides User Account List Data to the **Management Team**.

12. Level-2 DFD for Sub-process 6.2: Authenticate User

This diagram breaks down the "6.2 Authenticate User" process, typically initiated by a Customer (Resident or Management Team member attempting to log in).

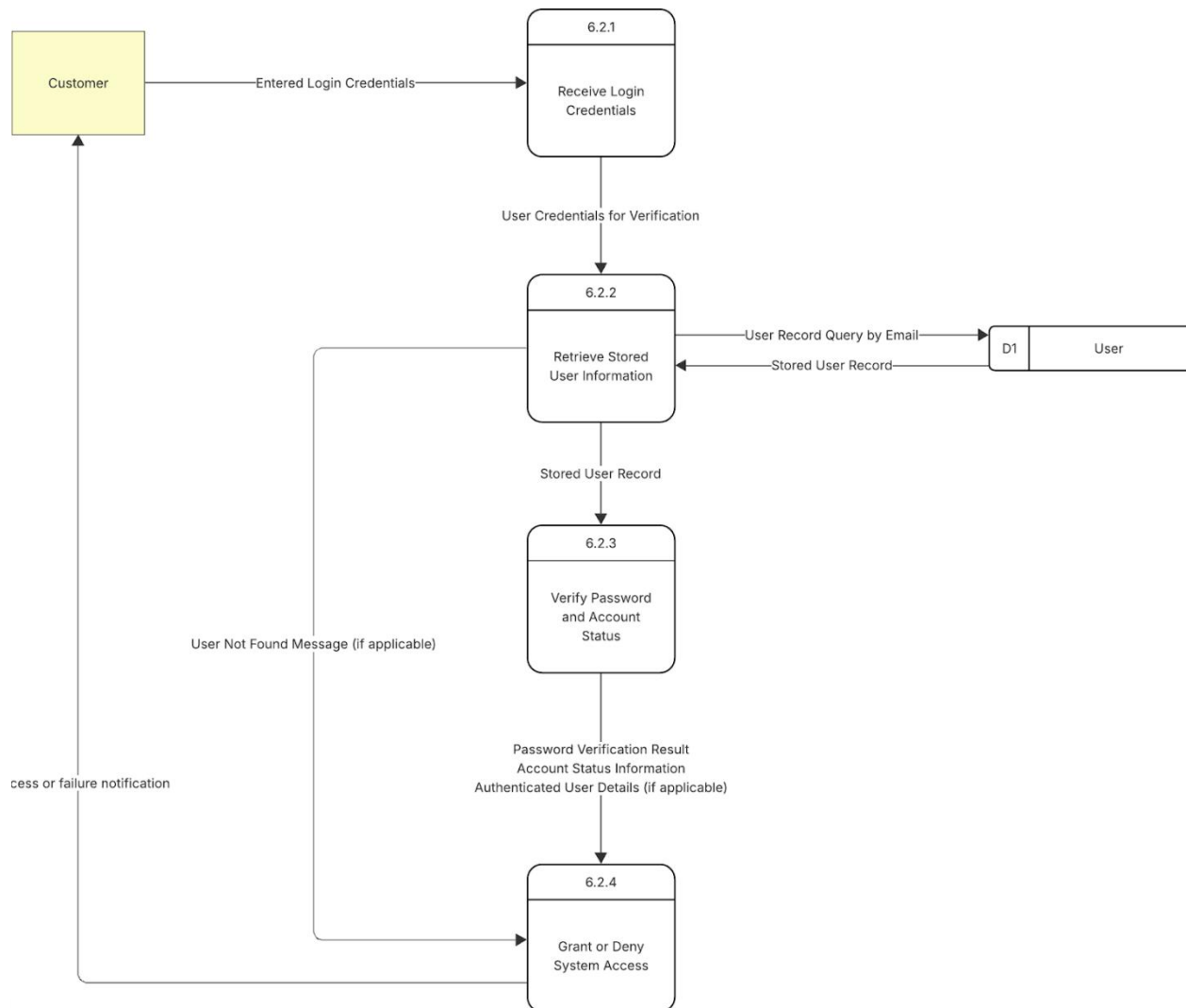


Figure 26: Level-2 DFD for Sub-process 6.2

- **Sub-processes:**

- **6.2.1 Receive Login Credentials:** Accepts the login credentials entered by the user.
- **6.2.2 Retrieve Stored User Information:** Fetches the user's record from the database based on the provided credentials.
- **6.2.3 Verify Password and Account Status:** Compares the entered password with the stored hash and checks if the account is active.

- **6.2.4 Grant or Deny System Access:** Based on verification, allows or disallows access to the system.
- **Data Flows and Interactions:**
 - **Customer** (acting as a general user trying to log in) provides Entered Login Credentials to sub-process **6.2.1**.
 - Sub-process **6.2.1** passes User Credentials for Verification to sub-process **6.2.2**.
 - Sub-process **6.2.2** sends User Record Query by Email (or username) to Data Store **D1 User** and receives Stored User Record.
 - If the user is not found, sub-process **6.2.2** sends User Not Found Message (if applicable) to sub-process **6.2.4**.
 - If the user is found, Stored User Record is passed to sub-process **6.2.3**.
 - Sub-process **6.2.3** performs verification and passes Password Verification Result, Account Status Information, and Authenticated User Details (if applicable) to sub-process **6.2.4**.
 - Sub-process **6.2.4** sends Access or failure notification to the **Customer**.

3.3 Entity Relational Data model – ERD

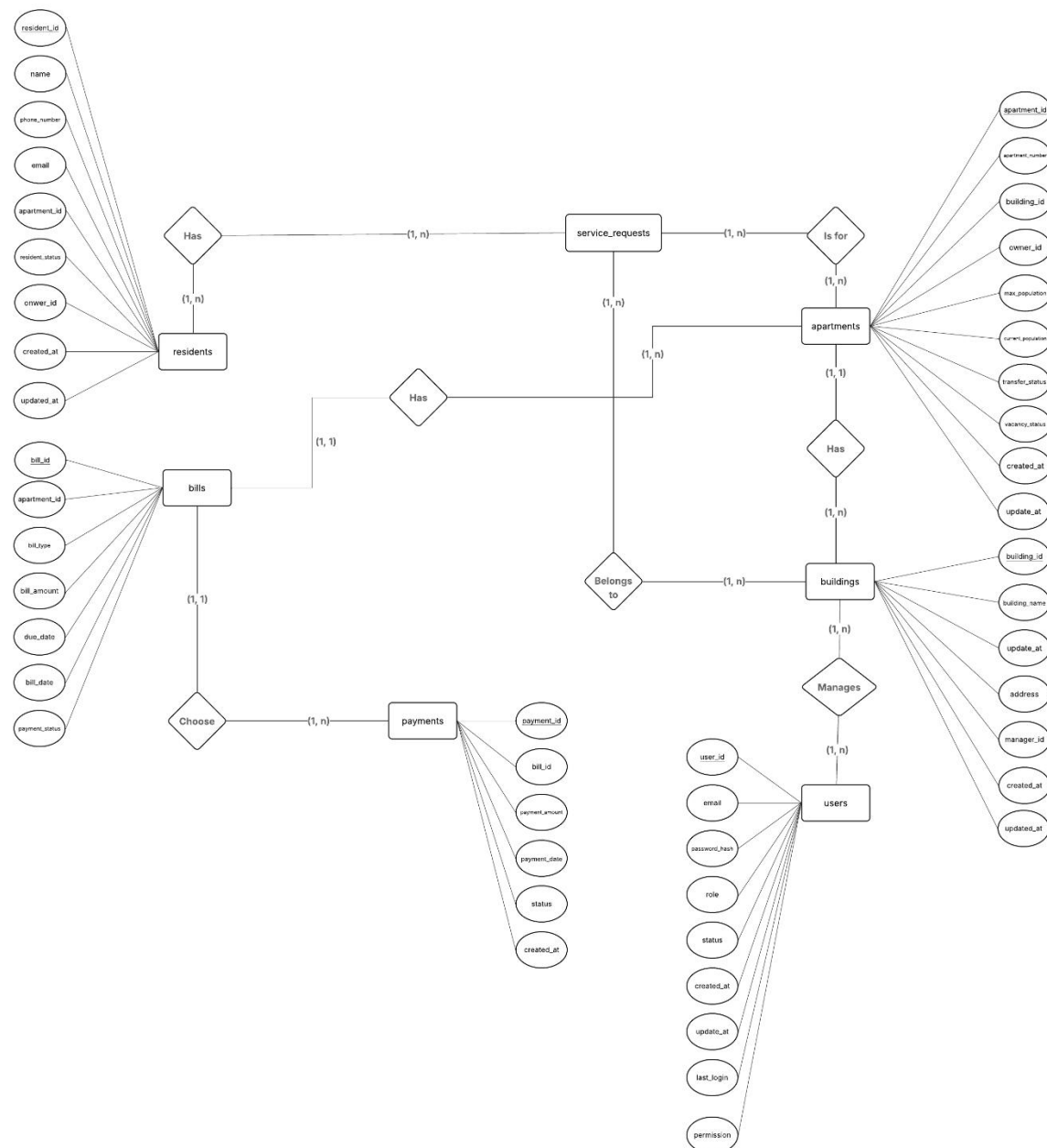


Figure 27: ERD diagram

3.4 Relational Data model – RDM

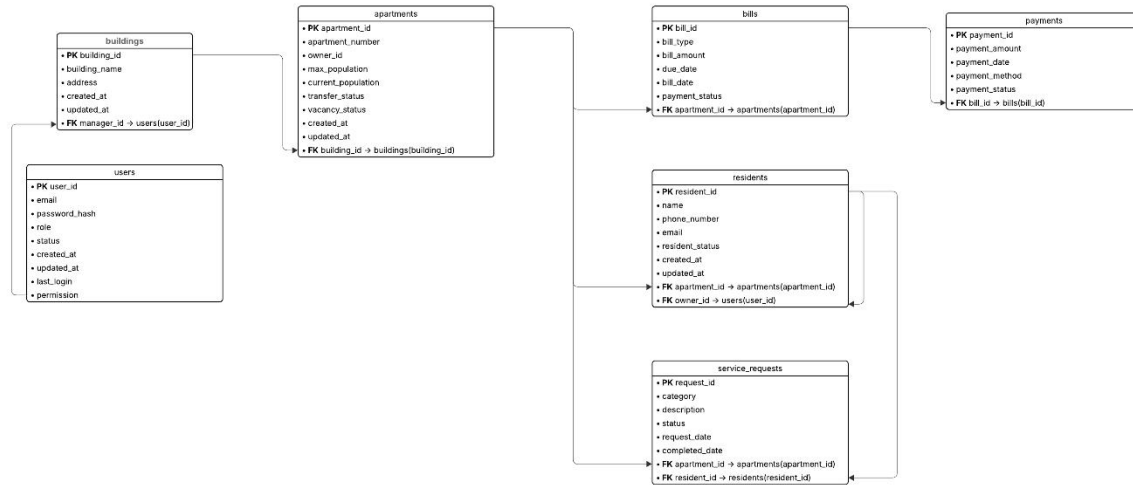


Figure 28: RDM diagram

3.5 System Architecture Design

The Apartment Management System is built upon a robust and scalable architecture, primarily leveraging the 3-Tier Architecture pattern, the Model-View-ViewModel (MVVM) design pattern, and a unique Multi-Tenant Schema Mechanism for data isolation.

3.5.1 3-Tier Architecture

The system rigorously adheres to a classical 3-Tier Architecture, providing a clear separation of concerns that enhances development, deployment, and scalability. Each tier

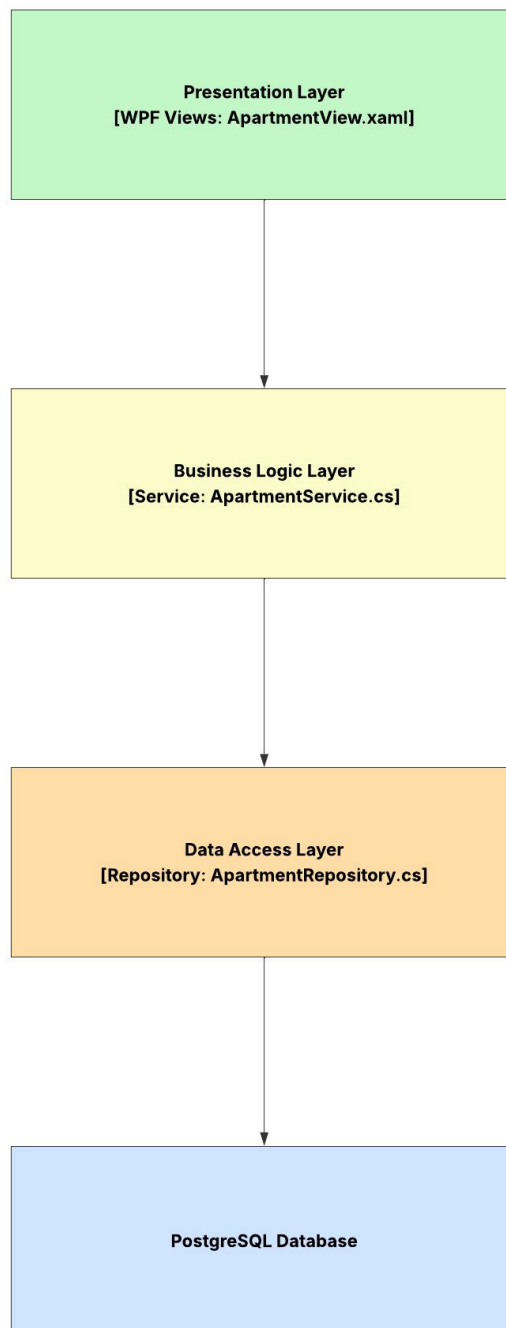


Figure 29: 3-Tier Architecture Diagram

performs a specific set of functions, promoting loose coupling and improved maintainability.

Presentation Layer (Client Layer):

- This is the topmost layer responsible for **user interaction** and **visual data representation**. Built using **Windows Presentation Foundation (WPF)**, it consists of various XAML-based View components such as those for apartment management, resident information, and service tracking. These Views focus solely on presenting data and capturing user input through UI elements, with no embedded business or data logic.

Business Logic Layer (Service Layer):

- Acting as the intermediary between the UI and data layers, this tier encapsulates the core operational logic of the application. It handles tasks such as validation, rule enforcement, and coordination between multiple data entities. Service classes in this layer receive requests from the Presentation Layer, apply business rules, and invoke the appropriate data access methods. This abstraction enhances the modularity and reusability of core functionality.

Data Access Layer (Repository Layer):

- This layer is solely responsible for **persisting and retrieving data** from the database. It provides a set of standardized data operations that hide database complexities from the business logic. The repositories within this layer use **Entity Framework Core** to interact with the underlying **PostgreSQL** database. Each repository is tailored to a specific entity (e.g., apartments, residents) and performs data operations in a consistent and structured manner.

3.5.2 MVVM (Model-View-ViewModel) Model

The application leverages the **MVVM architectural pattern**, which is particularly suitable for WPF applications. This pattern enhances **code organization**,

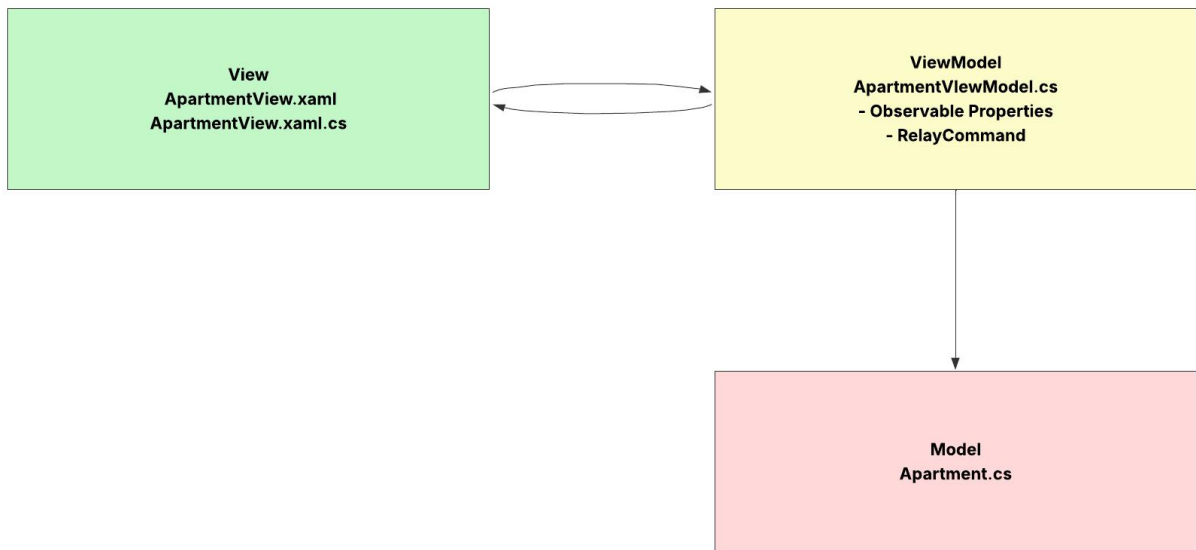


Figure 30: MVVM Model Diagram

data binding, and **testability**, while promoting separation between the UI and the logic.

Model:

- The Model represents the domain-level entities and encapsulates core data structures of the system. These are plain C# objects (POCOs) containing fields that reflect the business entities of the application, such as apartments, residents, services, and payments. The Model is completely independent of the UI layer and may contain validation rules directly associated with the data it represents.

View:

- The View is the visual component of the application and is defined declaratively in **XAML**. It provides the layout and visual representation of the data but does not include logic. Through **data binding**, the View connects to the ViewModel to

display content and capture user interactions. Each View corresponds to a specific use case or module within the system.

ViewModel:

- Serving as the bridge between the View and the Model, the ViewModel exposes data and operations to the UI. It processes UI events, prepares data for display, and forwards user actions to the business logic layer. The ViewModel implements property change notification (e.g., via `INotifyPropertyChanged`) and command interfaces (e.g., `ICommand`) to enable two-way data binding and interaction handling without tight coupling to the View or Model.

3.5.3 Multi-Tenant Schema Mechanism

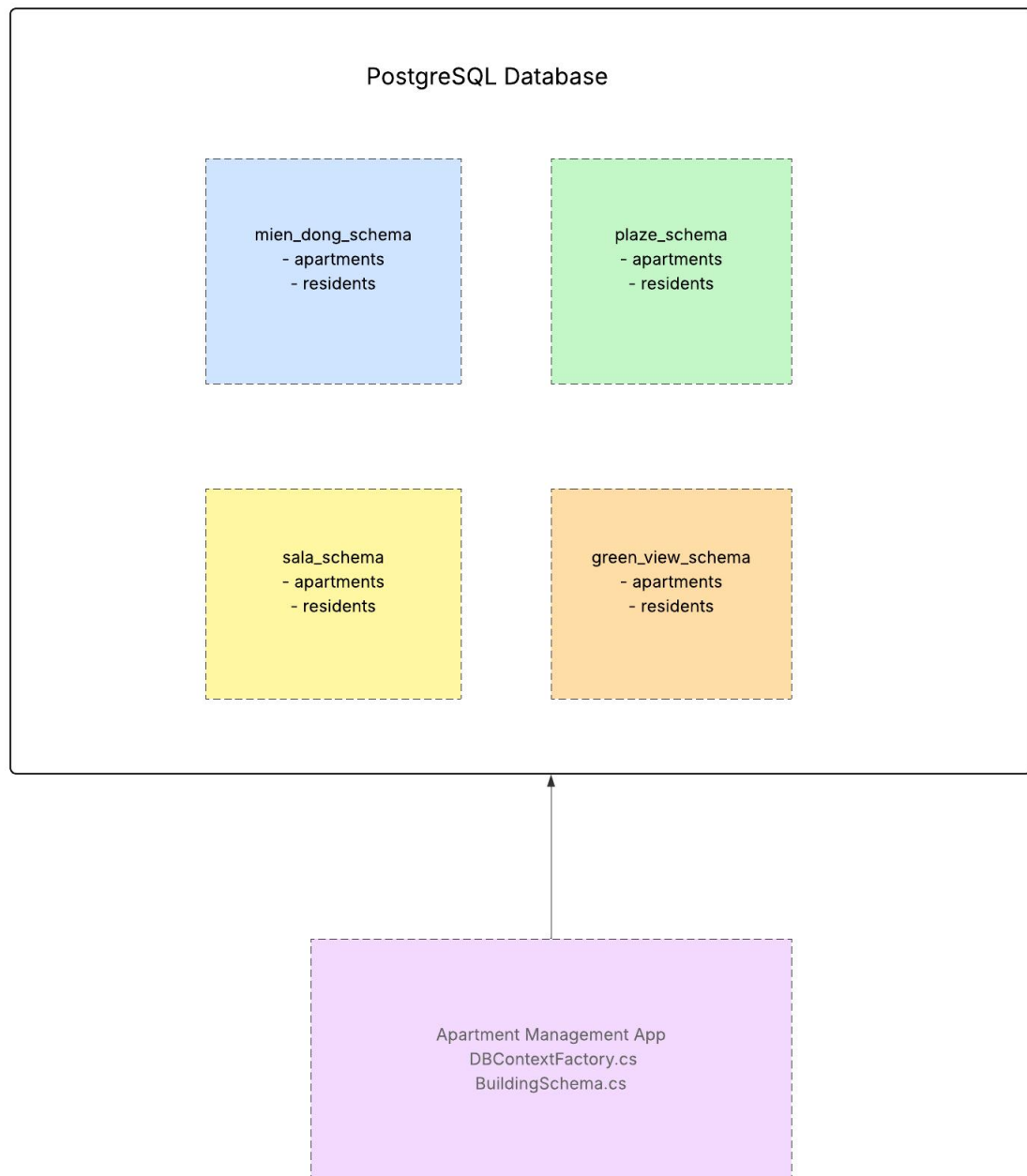


Figure 31: Multi-Tenant Schema Mechanism Diagram

To support multi-tenancy, the system employs a **schema-based isolation strategy** within a single PostgreSQL database. This architecture allows each tenant (i.e., apartment complex) to have logically separated data without requiring separate databases.

Centralized Schema Management

- A singleton class is responsible for maintaining the **currently active tenant schema**. This value is dynamically updated at runtime based on user selection and governs the scope of data access throughout the application. This ensures that all operations are tenant-specific and no cross-tenant data leakage occurs.

Context Factory and Dynamic Configuration

- To accommodate the active schema, the system uses a **factory pattern** to instantiate the database context with a **tenant-specific configuration**. The schema name is injected into the connection context as part of the PostgreSQL SearchPath, directing queries to the appropriate schema. Additionally, to ensure correctness and prevent stale caching by Entity Framework Core, a custom model caching strategy is implemented, enabling schema-aware metadata generation.

Tenant-Specific Table Mapping

- Within the customized DbContext, all entities are mapped to tables using the active schema. This mapping occurs during the model creation phase and ensures that all CRUD operations are executed within the correct tenant's schema. This design promotes **strong data isolation**, **cost-effective scalability**, and a **unified codebase** capable of serving multiple clients.

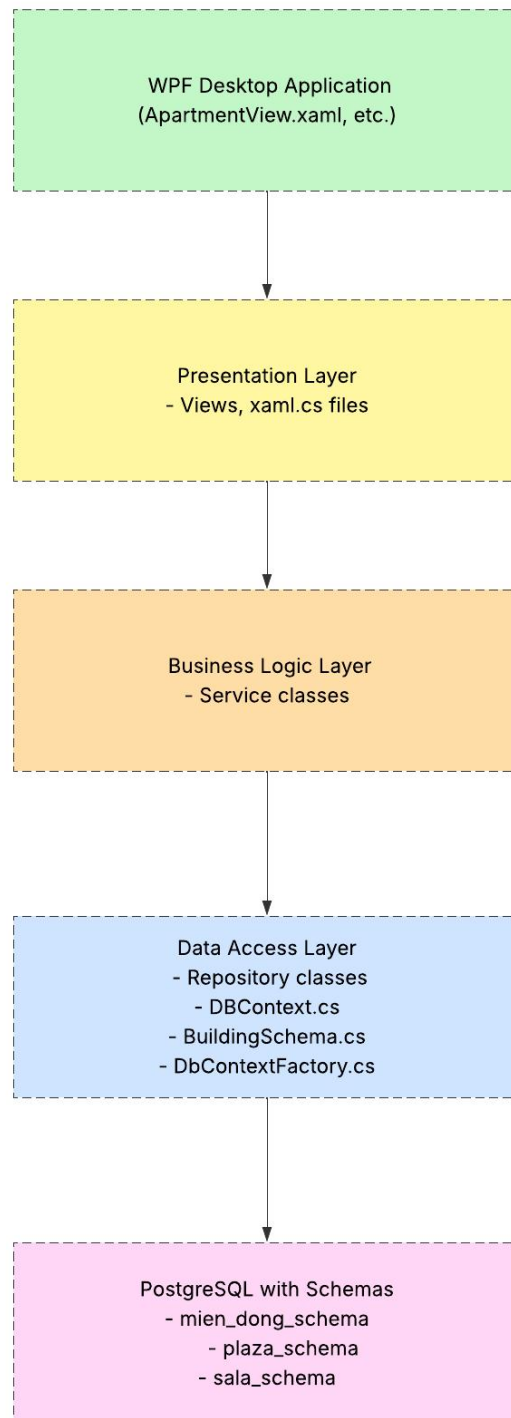


Figure 32: Overall System Architecture Diagram

3.5.4 Overall System Architecture Diagram

The Overall System Architecture provides a comprehensive view of how the different layers, modules, and infrastructure components of the Apartment Management System interact to deliver functionality in a scalable, maintainable, and modular manner.

At its core, the system is structured around two architectural patterns:

- The 3-Tier Architecture, which separates responsibilities across the Presentation, Business Logic, and Data Access layers.
- The Model-View-ViewModel (MVVM) pattern, which governs the internal interaction between UI components and data logic in the WPF desktop application.

Additionally, the system integrates a multi-tenant schema mechanism at the database level, allowing it to support multiple apartment complexes (tenants) with strict data isolation while maintaining a single codebase and database instance.

Components and Their Interactions:

- Client Application (WPF Desktop App): The entry point for users, built using WPF and following the MVVM pattern. Users interact with Views, which bind to ViewModels for data presentation and user interaction.
- Presentation Layer: Contains Views and UI logic. It communicates exclusively with the Business Logic Layer through ViewModels and commands.
- Business Logic Layer (Service Layer): Handles domain rules, input validation, and transaction orchestration. It receives requests from ViewModels and delegates data operations to the Data Access Layer.
- Data Access Layer (Repository Layer): Provides an abstraction over direct database access. Repository classes communicate with the PostgreSQL database through Entity Framework Core.

- Database Layer: A centralized PostgreSQL instance containing multiple schemas, each representing a distinct tenant (e.g., mien_dong, plaza, sala). Data from different tenants is logically isolated.

Tenant Management Mechanism:

- BuildingSchema Singleton: Holds the active tenant's schema at runtime, ensuring that all operations target the correct dataset.
- DbContextFactory: Generates a schema-aware DbContext, appending the appropriate SearchPath to the connection string for PostgreSQL.
- ApartmentDbContext: Maps application entities to tables under the specified schema and ensures schema-specific operations.

This architecture ensures that each functional concern is clearly separated, enabling individual development, testing, and scaling of components. The multi-tenant schema mechanism minimizes infrastructure overhead while preserving data isolation and security. Moreover, by combining MVVM and 3-Tier patterns, the application remains modular, flexible, and aligned with modern software design best practices.

CHAPTER 4. IMPLEMENTATION AND TESTING

4.1 Introduction to the implementation environment

The Apartment Management System is implemented as a Windows-based desktop application. The selection of technologies was carefully guided by the requirements for **robustness, scalability, maintainability**, and alignment with **modern software engineering principles**.

4.1.1 Programming Language

The application is primarily developed using **C#**, a strongly-typed, object-oriented programming language within the **.NET ecosystem**. C# was chosen due to its expressive

syntax, extensive standard libraries, and deep integration with Windows-based frameworks, making it a natural fit for building enterprise-grade desktop solutions.

4.1.2 User Interface Framework

The graphical user interface (GUI) is constructed using **Windows Presentation Foundation (WPF)**, a powerful framework for creating modern, responsive desktop applications on Windows. WPF supports **declarative UI development** through **XAML (Extensible Application Markup Language)** and is particularly well-suited for the **MVVM (Model-View-ViewModel)** pattern, enabling a clean separation between interface design and application logic. Representative Views in the system include components for managing apartments, residents, payments, and services.

4.1.3 Database Management System

The system employs **PostgreSQL** as its **relational database management system (RDBMS)**. PostgreSQL is widely recognized for its **stability, extensibility, and advanced feature set**, including native support for **schemas**, which is leveraged in the system's multi-tenant architecture. All core entities — such as users, buildings, apartments, residents, bills, payments, and service requests — are organized within structured schemas (e.g., `mien_dong`) as defined in the initial database migration scripts.

4.1.4 Object-Relational Mapping (ORM)

To streamline the interaction between the application and the relational database, the system uses **Entity Framework Core (EF Core)** as its ORM layer. EF Core allows developers to interact with the database using **strongly-typed C# objects**, thereby eliminating the need for verbose SQL queries and enabling **LINQ-based data manipulation**. The `ApartmentDbContext` class encapsulates the database context, managing entity configurations and schema mapping.

4.1.5 Configuration Management

Application configurations — including environment-specific settings and database connection strings — are externalized in the `jsconfig1.json` file. These settings are accessed programmatically through the `ConfigManager.cs` utility class. This external configuration model supports environment flexibility and promotes maintainability by **decoupling runtime settings from application logic**. The PostgreSQL connection string is referenced by the key `DefaultConnection`.

4.1.6 Development Environment

The project is developed within **Microsoft Visual Studio**, a comprehensive Integrated Development Environment (IDE) that provides advanced tools for building, testing, and debugging C#, WPF, and .NET-based applications. Visual Studio's integration with version control systems and NuGet package management further streamlines the development workflow.

4.2 User Interface – UI

The design and implementation of the User Interface (UI) within the Apartment Management System are central to achieving its objectives of operational efficiency and enhanced user experience. Developed on the Windows Presentation Foundation (WPF) framework, the UI embodies principles of intuitive interaction, visual clarity, and responsiveness, directly addressing the pain points identified during stakeholder analysis.

4.2.1 Design Philosophy and Usability Paradigm

The UI's foundational design paradigm is rooted in user-centric principles, aiming to provide a coherent and accessible interface for administrative and management personnel. Key aspects guiding its conception include:

Intuitiveness and Minimal Learning Curve

- The interface is structured to be immediately understandable, minimizing the cognitive load on new users and facilitating rapid task completion. This is achieved through logical navigation paths and consistent control placements across modules.

Consistency and Predictability

- A unified visual language, encompassing typography, color palettes, and interaction patterns, is rigorously applied throughout the application. This consistency ensures that users can anticipate system behavior, fostering a predictable and reliable user experience.

Clarity and Information Hierarchy:

- Information is presented with a clear visual hierarchy, distinguishing essential data from supplementary details. This approach mitigates visual clutter and directs user attention to critical operational metrics and task-relevant information.

Responsiveness and Performance:

- Beyond aesthetics, the UI is engineered for operational efficiency. Data grids are designed to load promptly, with performance benchmarks targeting sub-3-second load times for up to 1000 records and sub-2-second completion for search, filter, and sort operations. This ensures fluid interaction even with substantial datasets.

4.2.2 Modular UI Architecture and Key Components

The UI is architecturally decomposed into distinct modules, each corresponding to a primary functional domain of the apartment management system. Navigation between these modules is facilitated by a centralized control, enhancing overall system usability.

4.2.2.1 Dashboard Module

The Dashboard serves as the command center of the application, offering a real-time, consolidated overview of critical operational and financial key performance

indicators (KPIs). This module is implemented within `MainWindowView.xaml.cs` and powered by `DashboardViewModel.cs`.

Key Performance Indicators (KPIs): Prominently displayed widgets present essential metrics, including total apartment count, occupied apartment count, total resident count, and comparative revenue figures for the current and preceding months.

Trend Analysis: The UI dynamically calculates and visualizes month-over-month changes for key operational dimensions such as revenue, apartment occupancy rate, resident count, and service request volume. This provides immediate insight into performance trajectories.

Financial Overview: A dedicated graphical representation (e.g., a chart) illustrates the system's monthly revenue for the current fiscal year, offering a concise financial snapshot.

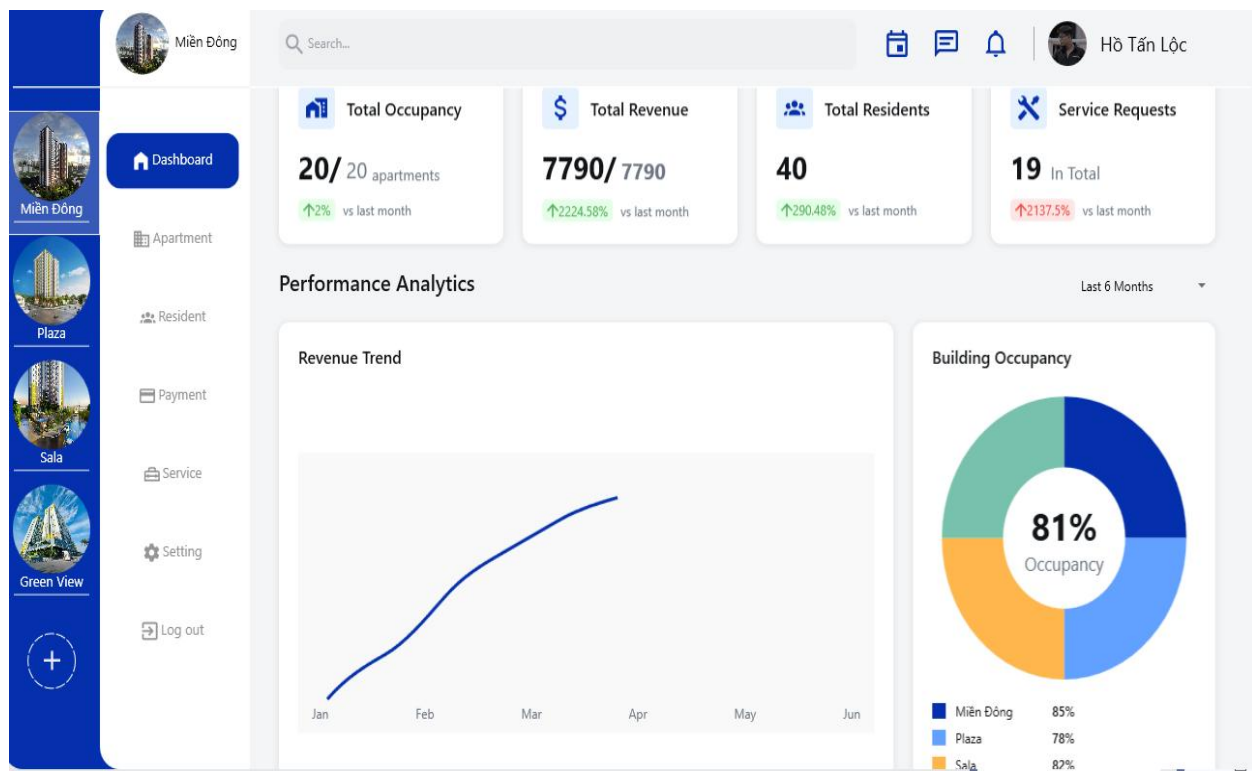


Figure 33: Dashboard UI 1

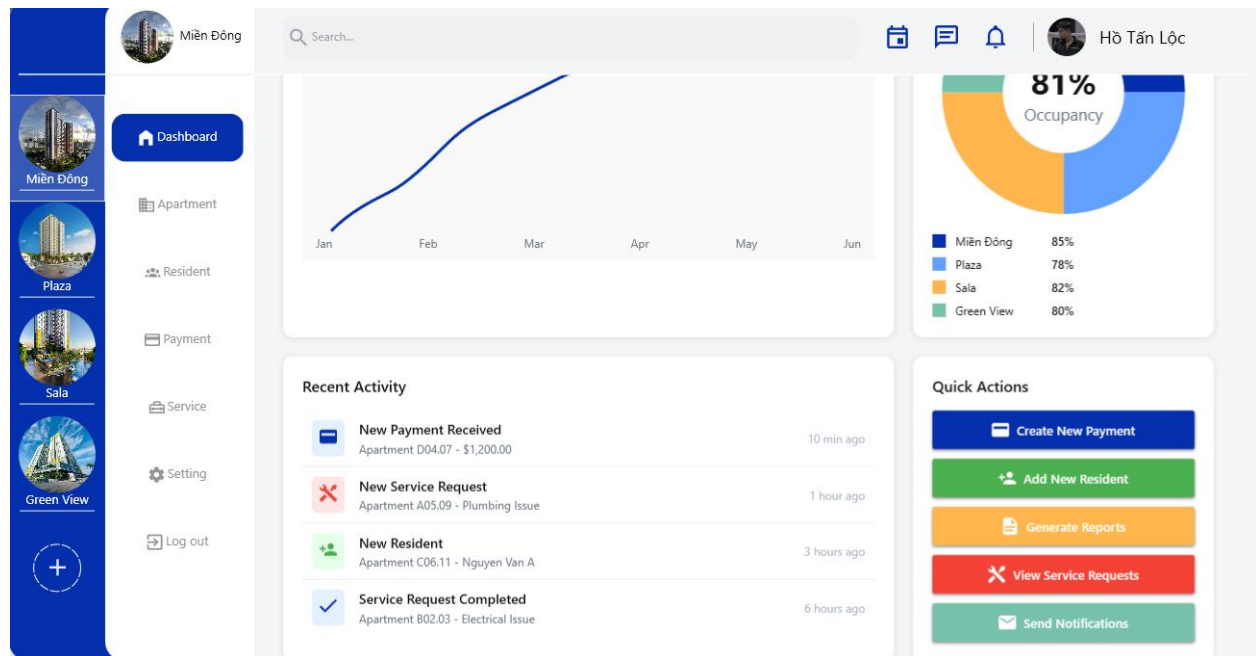


Figure 34: Dashboard UI 2

4.2.2.2 Apartment Management Module

This module provides a structured and interactive interface for the **comprehensive lifecycle management of apartment units**. It is implemented following the MVVM pattern, with multiple dedicated views and forms designed to enhance user efficiency and data clarity. Key interface components include ApartmentView.xaml.cs, ApartmentInfo.xaml.cs, ApartmentCreateView.xaml.cs, and ApartmentEdit.xaml.cs.

Paginated Data Presentation

- Apartment records are presented within a **paginated data grid**, enabling users to efficiently browse and navigate large inventories without performance degradation. Pagination improves responsiveness and usability, particularly when managing extensive datasets.

Dynamic Filtering and Sorting

- The user interface integrates a set of **intuitive filtering controls** that allow apartments to be dynamically filtered based on key attributes such as **vacancy status** (e.g., *Vacant, Occupied*) and **transaction status** (e.g., *Available, Pending, Sold*). Users can also apply **sorting operations** based on apartment number or status, improving data visibility and ease of access.

Real-Time Search Functionality

- A **predictive search field** is embedded within the view to facilitate **instantaneous filtering**. As the user types, the apartment list is dynamically updated in real-time, significantly accelerating the data retrieval process and enhancing user experience.

Detail Views and Form-Based Operations

The module includes a suite of **dedicated views and forms** for viewing, creating, and updating apartment records. These include:

- ApartmentInfo.xaml.cs for detailed read-only information,
- ApartmentCreateView.xaml.cs for registering new units, and
- ApartmentEdit.xaml.cs for modifying existing data.

Each form is designed with **clarity, validation, and usability in mind**, ensuring accurate data input and seamless user interaction.

QR Code Integration

- An innovative feature integrated into the ApartmentInfoViewModel.cs enables the **automatic generation of QR codes** containing apartment-specific metadata. This capability facilitates potential integrations with external systems and supports **mobile-based information access**, offering modern enhancements to operational efficiency.

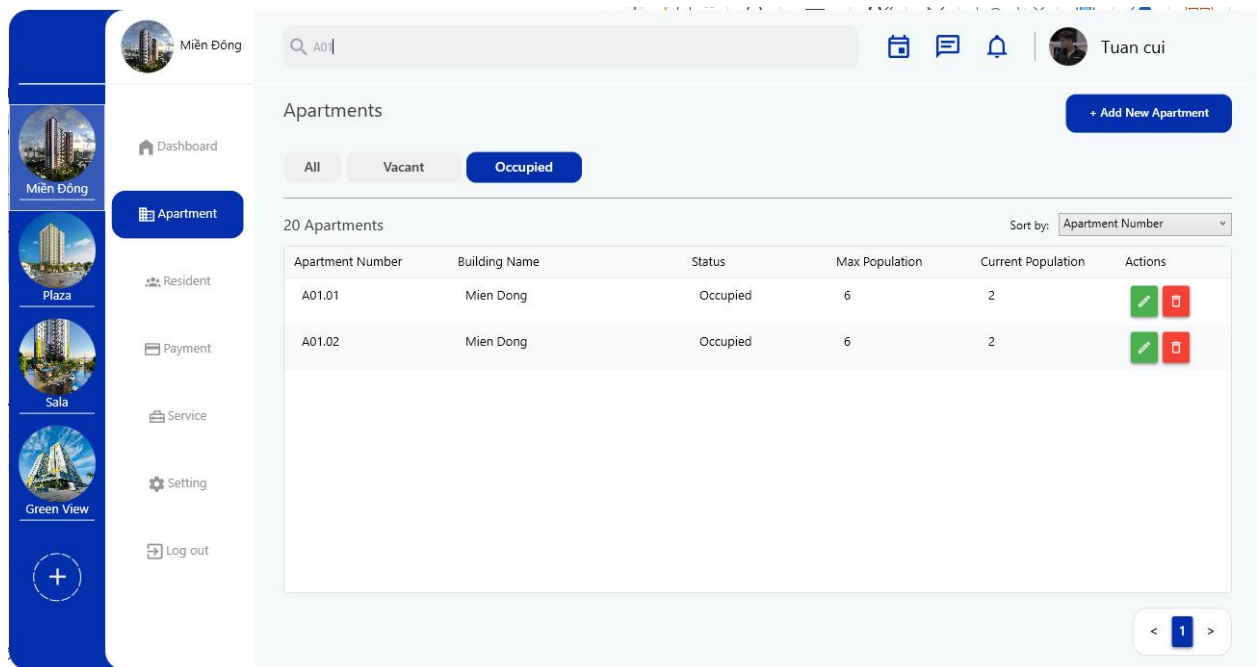


Figure 36: Apartmentnet List View UI - Search Function

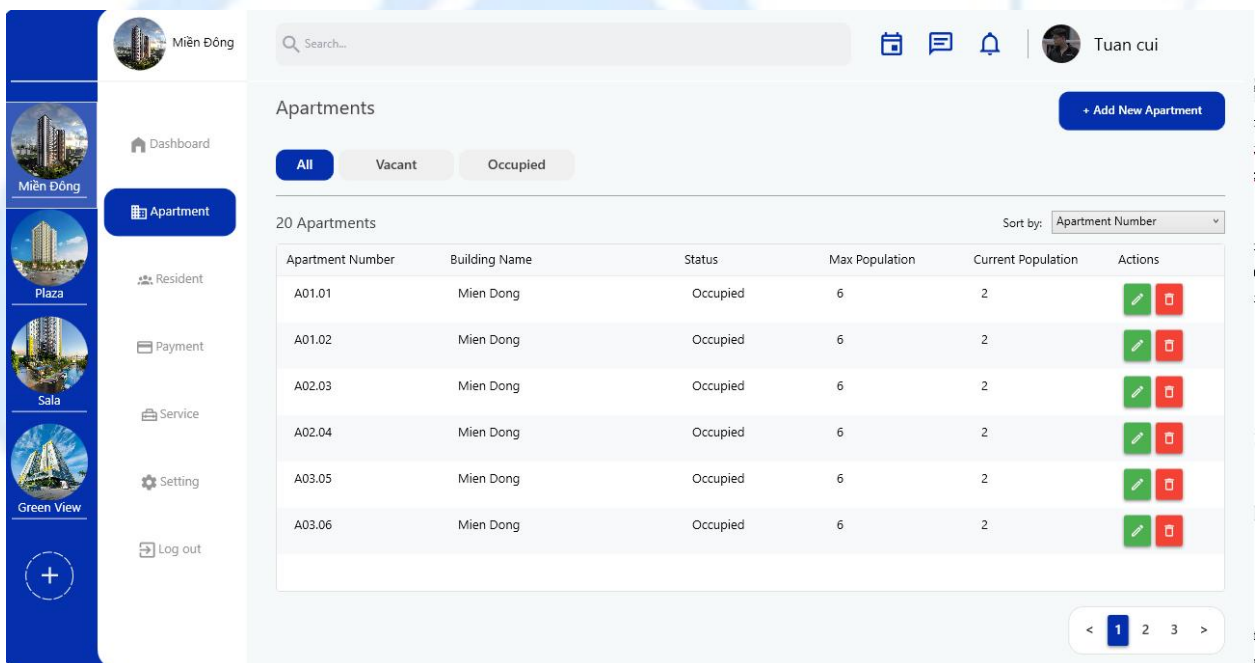


Figure 35: Apartmentnet List View UI

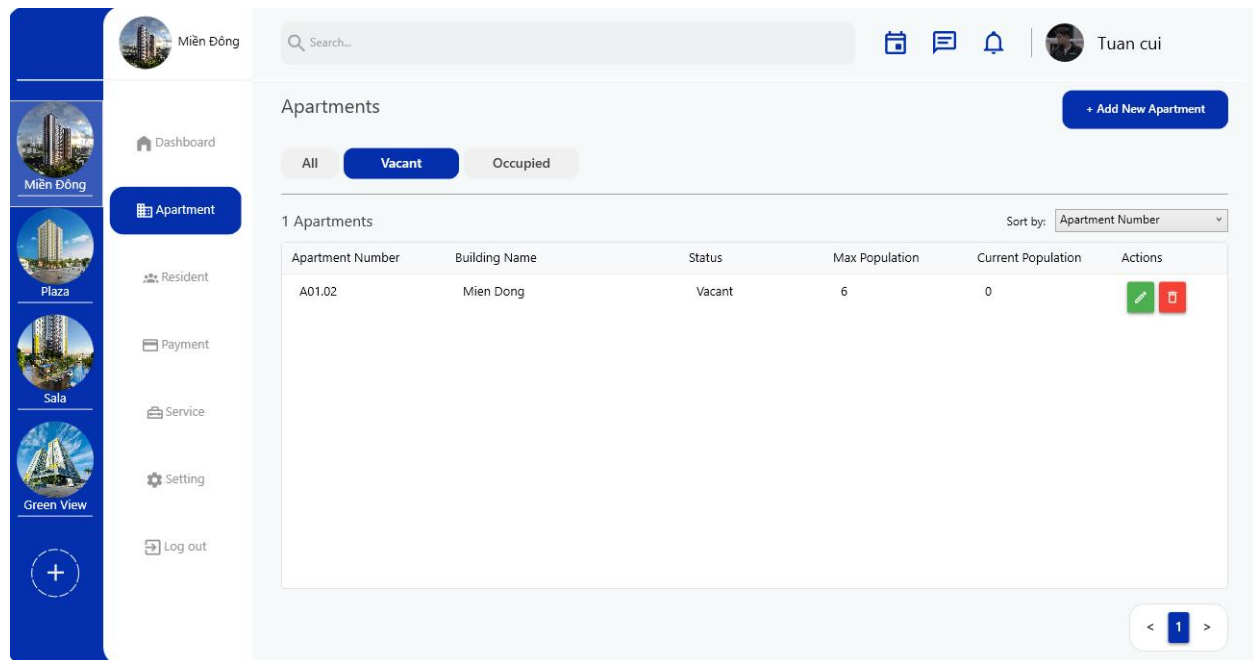


Figure 38: Apartment List View UI - Filter Function

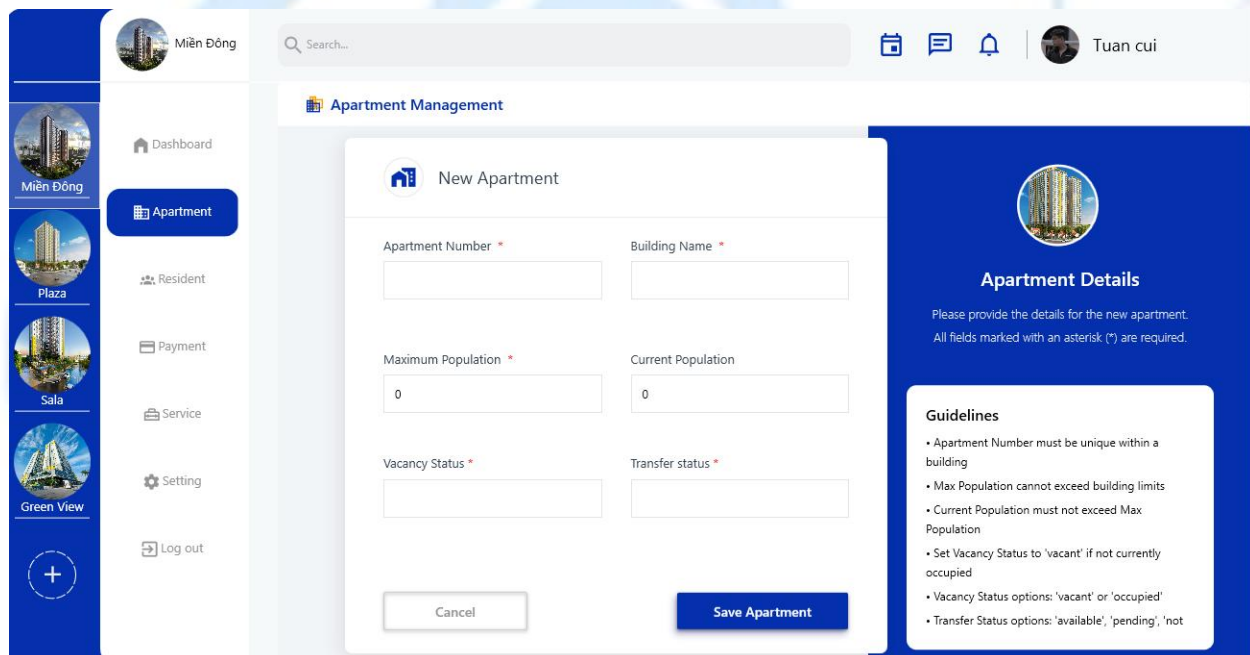


Figure 37: Create Apartment View UI

The screenshot displays the 'Apartment Management' interface. On the left is a sidebar with a user profile 'Miễn Đông' and navigation links: Dashboard, Apartment (selected), Resident, Payment, Service, Setting, and Log out. The main area shows the 'Edit Apartment' form with the following fields:

Apartment Number *	Building Name *
A01.01	Mien Dong
Maximum Population *	Current Population
6	2
Vacancy Status *	Transfer status *
Occupied	Available

At the bottom of the form are 'Cancel' and 'Save Apartment' buttons. To the right, the 'Apartment Details' panel includes a note: 'Please provide the details for the new apartment. All fields marked with an asterisk (*) are required.' Below this is a 'Guidelines' section with the following rules:

- Apartment Number must be unique within a building
- Max Population cannot exceed building limits
- Current Population must not exceed Max Population
- Set Vacancy Status to 'vacant' if not currently occupied
- Vacancy Status options: 'vacant' or 'occupied'
- Transfer Status options: 'available', 'pending', 'not'

Figure 39: Apartment Edit View UI

4.2.3 Resident Management Module

This module provides a comprehensive interface for the management of resident profiles and their association with apartment units. Its views include ResidentView.xaml.cs, ResidentInfoView.xaml.cs, ResidentCreateView.xaml.cs, and ResidentEditView.xaml.cs.

Resident Directory

- A paginated data grid presents a complete list of residents, supporting efficient Browse, sorting, and search capabilities based on resident name, phone number, or associated apartment identifier.

Comprehensive Resident Profiles:

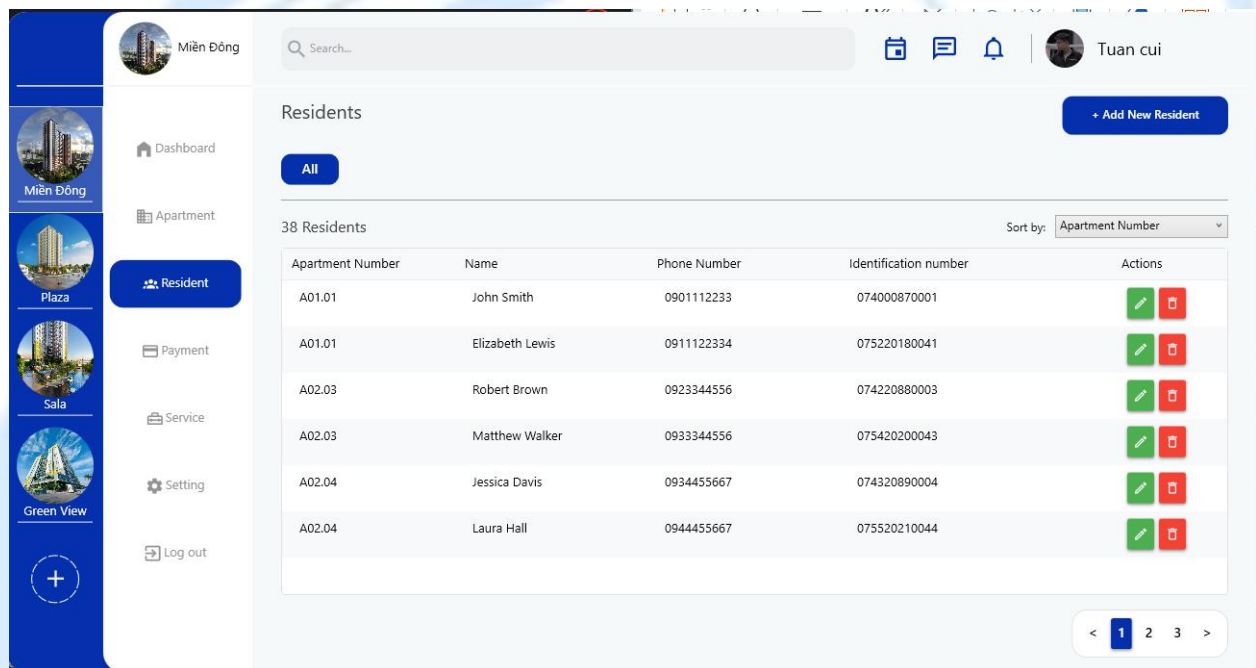
- The ResidentInfoView provides an aggregated view of a selected resident's data, encompassing personal details, apartment association, primary owner information, and a summary of their most recent payment status. This addresses the need for centralized and transparent resident information.

Relationship Management:

- The UI is designed to clearly delineate the relationship between apartment owners/main tenants and other household members, reflecting the system's capacity to manage complex family structures within units.

Intuitive Data Entry:

- Forms for adding (ResidentCreateView) and modifying (ResidentEditView) resident records are streamlined for usability, ensuring clear data input and validation for personal and identification details.



Apartment Number	Name	Phone Number	Identification number	Actions
A01.01	John Smith	0901112233	074000870001	 
A01.01	Elizabeth Lewis	0911122334	075220180041	 
A02.03	Robert Brown	0923344556	074220880003	 
A02.03	Matthew Walker	0933344556	075420200043	 
A02.04	Jessica Davis	0934455667	074320890004	 
A02.04	Laura Hall	0944455667	075520210044	 

Figure 40: Resident List View UI

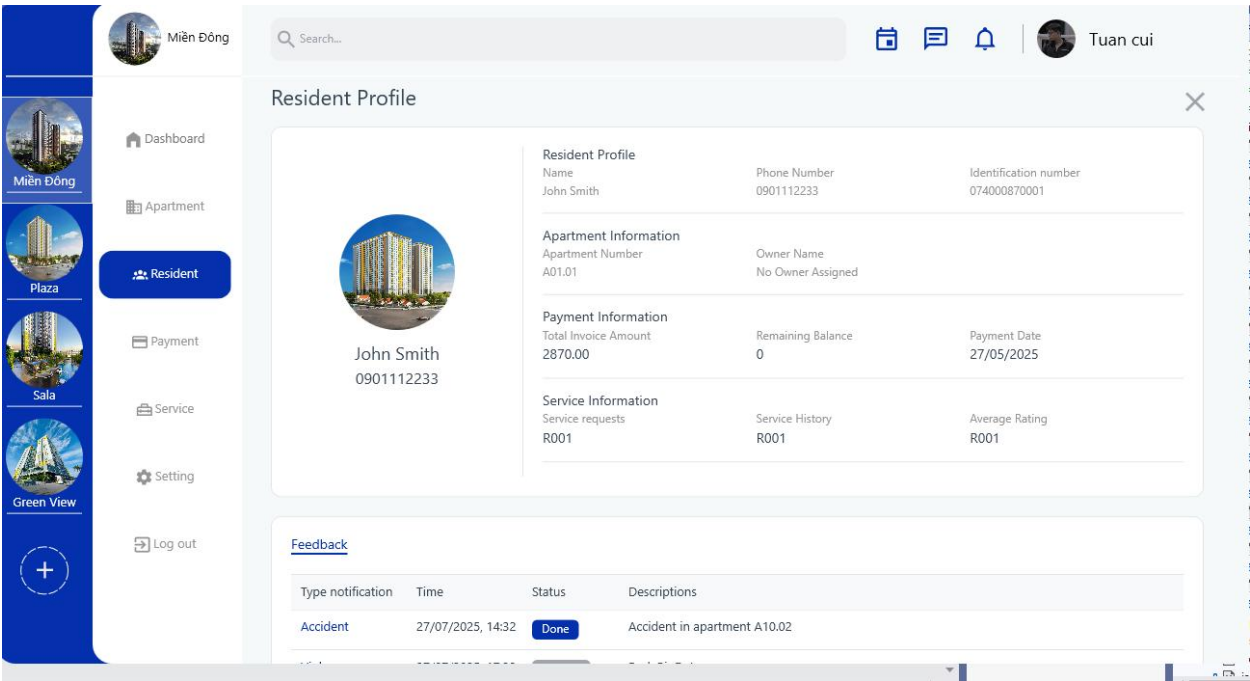


Figure 41: Resident Info Detail View

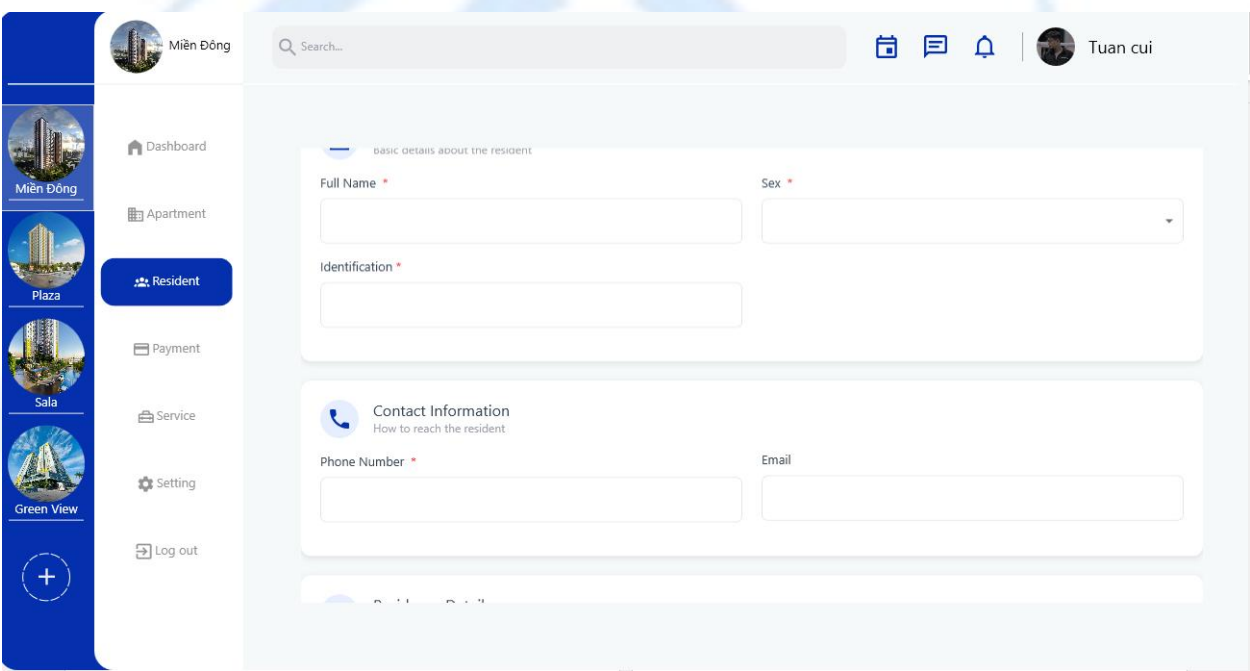


Figure 42: Create Resident View UI

4.2.4 Billing and Payment Management Module

This module provides a critical interface for managing the financial transactions of the apartment complex, from bill generation to payment tracking. Its views include PaymentView.xaml.cs, PaymentInfoView.xaml.cs, and PaymentCreateView.xaml.cs.

Status-Driven Filtering

- Dedicated UI controls (e.g., distinct buttons for "All", "Pending", "Completed", and "Overdue") enable rapid filtering of payment records by their current financial status, offering immediate financial oversight and highlighting overdue accounts.

Transactional Transparency

- The PaymentInfoView provides a detailed breakdown of a selected payment, explicitly linking it to all associated bills. This ensures complete transparency regarding charges and paid amounts.

Streamlined Bill Creation

- The PaymentCreateView offers a simplified interface for recording new bills. Client-side validation mechanisms (e.g., restricting TxtAmount input to numeric and decimal values) ensure data integrity at the point of entry.

Payment Status Management

- A dedicated UI element (e.g., a "Check" button) facilitates the direct marking of

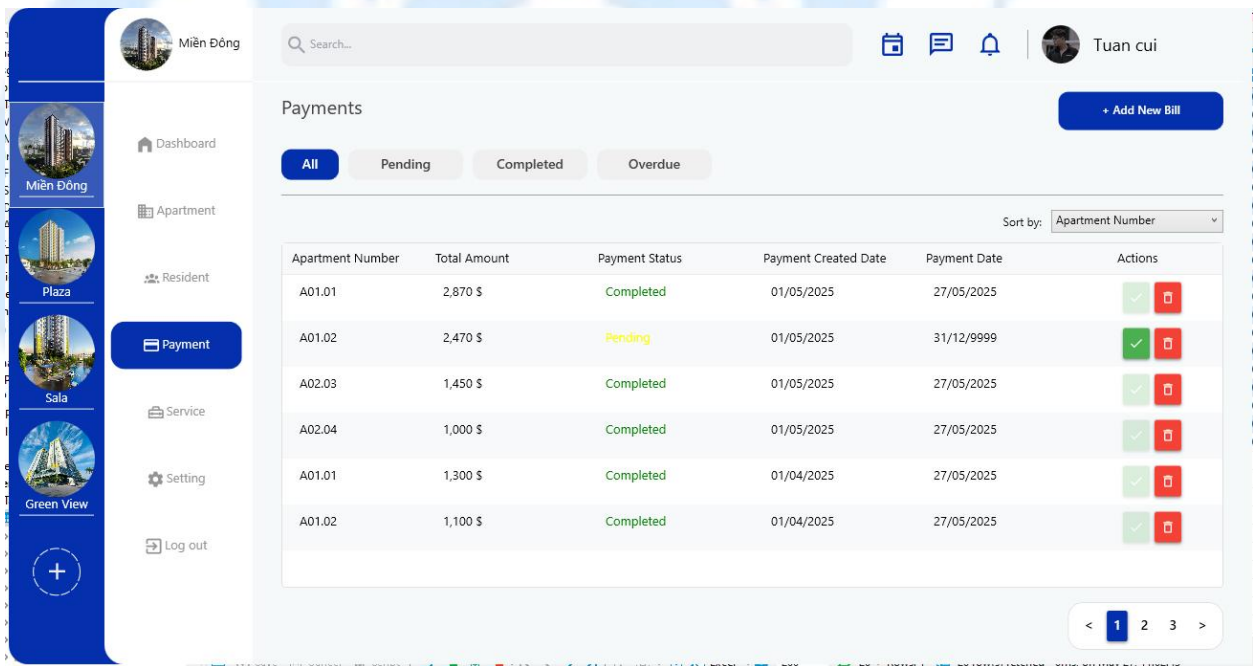


Figure 43: Payment List View UI

payments as "Completed," streamlining the workflow for financial reconciliation.

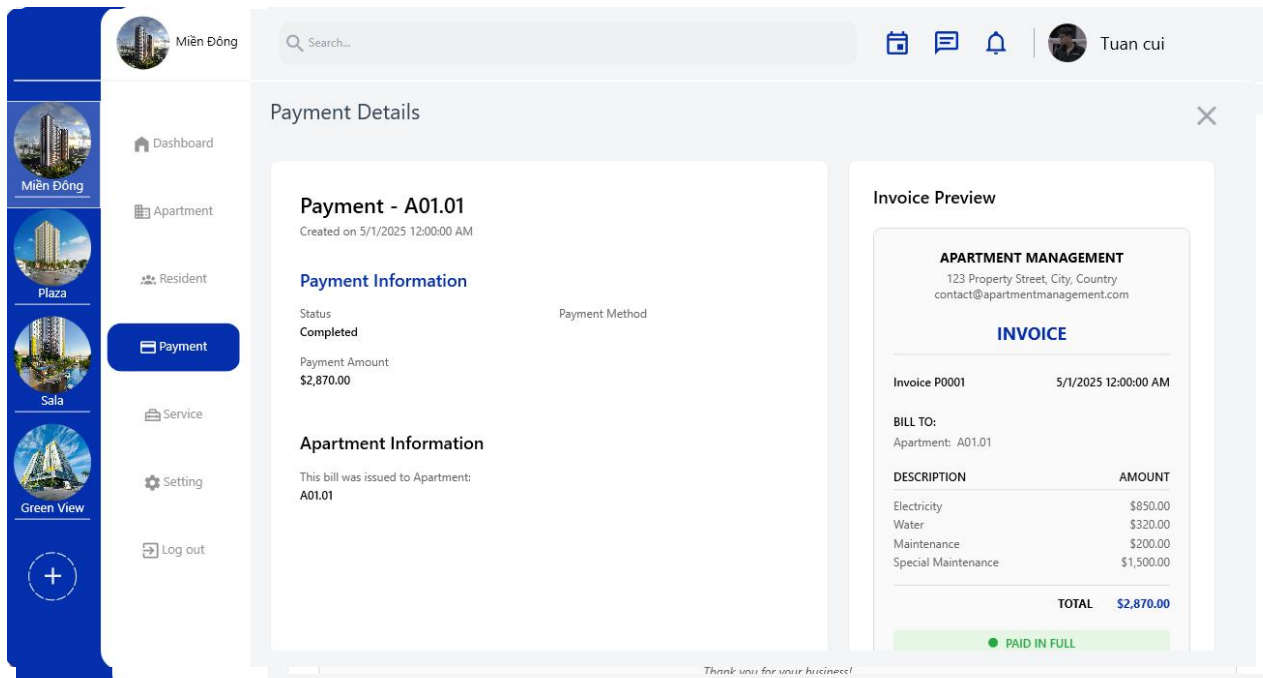


Figure 45: Payment Detail View UI

Figure 44: Create Bill View UI

4.2.5 Service Request Management Module

This module provides a structured and efficient interface for managing service requests initiated by residents, ensuring timely processing and resolution. Its primary view is ServiceView.xaml.cs.

Status-Based Request Overview

- The UI allows for rapid filtering of service requests by their current status ("In Progress", "Completed", "All"), enabling management to prioritize and track pending tasks effectively.

Direct Completion Action

- A direct interaction point (e.g., a "Check" button associated with each request) permits marking service requests as completed, streamlining the workflow for issue resolution and updating records instantaneously.

Efficient Search and Sorting:

- Standardized search and sort functionalities are integrated, facilitating quick retrieval and organization of service requests based on criteria such as apartment ID or request date.

Apartment	Resident	Category	Payment Status	Request Date	Completed Date	Actions
A02.03	Robert Brown	HVAC	Completed	06/03/2025	27/05/2025	
A03.06	Sarah Wilson	Maintenance	In Progress	19/04/2025		
A04.07	David Moore	Plumbing	In Progress	08/04/2025		
A04.08	James Taylor	Electrical	In Progress	22/05/2025		
A05.09	Linda Anderson	HVAC	In Progress	24/03/2025		
A05.10	Barbara Thomas	Cleaning	In Progress	14/04/2025		

Figure 46: Service List View UI

4.3 Database Design

4.3.1 Relational Data Model (RDM) Overview

The foundational structure of the database is defined by a comprehensive Relational Data Model. This model organizes the application's data into a series of interconnected tables, each meticulously designed to represent a distinct entity within the

apartment management domain. Relationships between these entities are rigorously enforced through the implementation of foreign keys, ensuring referential integrity and maintaining data consistency across the entire dataset. The primary tables, their key attributes, and inter-table relationships are defined within the mien_dong.sql script as follows:

users: Stores user authentication and authorization credentials.

- **Columns:** user_id (PRIMARY KEY, VARCHAR(20)), email (UNIQUE, VARCHAR(255)), password_hash (TEXT), role (VARCHAR(50)), status (VARCHAR(50), DEFAULT 'Active'), created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), last_login (TIMESTAMP).

buildings: Manages information pertaining to each apartment building.

- **Columns:** building_id (PRIMARY KEY, VARCHAR(20)), building_name (VARCHAR(255)), address (VARCHAR(255)), manager_id (VARCHAR(20), FOREIGN KEY referencing users.user_id ON DELETE SET NULL), created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP).

apartments: Contains detailed records for individual apartment units.

- **Columns:** apartment_id (PRIMARY KEY, VARCHAR(20)), building_id (VARCHAR(20), FOREIGN KEY referencing buildings.building_id ON DELETE CASCADE), owner_id (VARCHAR(20)), max_population (INTEGER), current_population (INTEGER, DEFAULT 0), transfer_status (VARCHAR(50), DEFAULT 'Available'), vacancy_status (VARCHAR(50), DEFAULT 'Vacant'), created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), date_registered (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP).

residents: Stores comprehensive resident profiles.

- **Columns:** resident_id (PRIMARY KEY, VARCHAR(20)), name (VARCHAR(255)), phone_number (VARCHAR(15)), email (VARCHAR(255)), sex (VARCHAR(10), CHECK (sex IN ('Male', 'Female', 'Other'))), identification_number (UNIQUE, VARCHAR(50)), apartment_id (VARCHAR(20), FOREIGN KEY referencing apartments.apartment_id ON DELETE CASCADE), resident_status (VARCHAR(50), DEFAULT 'Resident'), owner_id (VARCHAR(20), FOREIGN KEY referencing residents.resident_id ON DELETE CASCADE for self-referencing owner), created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), date_registered (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP).

bills: Records all generated bills for services and fees.

- **Columns:** bill_id (PRIMARY KEY, VARCHAR(20)), apartment_id (VARCHAR(20), FOREIGN KEY referencing apartments.apartment_id ON DELETE CASCADE), bill_type (VARCHAR(50)), bill_amount (DECIMAL(12, 2)), due_date (TIMESTAMP), bill_date (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP), payment_status (VARCHAR(50), DEFAULT 'Pending').

payments: Tracks payment transactions received from residents.

- **Columns:** payment_id (PRIMARY KEY, VARCHAR(20)), apartment_id (VARCHAR(20), FOREIGN KEY referencing apartments.apartment_id ON DELETE CASCADE), total_amount (DECIMAL(12, 2)), payment_date (TIMESTAMP, DEFAULT '9999-12-31 23:59:59'), payment_method (VARCHAR(50), DEFAULT ""), payment_status (VARCHAR(50), DEFAULT

'Pending'), payment_created_date (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP).

paymentsdetail: A junction table that establishes a many-to-many relationship between bills and payments.

- **Columns:** bill_id (PRIMARY KEY, VARCHAR(20), FOREIGN KEY referencing bills.bill_id ON DELETE CASCADE), payment_id (PRIMARY KEY, VARCHAR(20), FOREIGN KEY referencing payments.payment_id ON DELETE CASCADE).

service_requests: Records and manages service requests submitted by residents.

- **Columns:** request_id (PRIMARY KEY, VARCHAR(20)), apartment_id (VARCHAR(20), NOT NULL, FOREIGN KEY referencing apartments.apartment_id ON DELETE CASCADE), resident_id (VARCHAR(20), NOT NULL, FOREIGN KEY referencing residents.resident_id ON DELETE CASCADE), category (VARCHAR(50)), description (TEXT), status (VARCHAR(50), DEFAULT 'Pending'), request_date (TIMESTAMP WITHOUT TIME ZONE, DEFAULT CURRENT_TIMESTAMP), completed_date (TIMESTAMP WITHOUT TIME ZONE).

ERD diagram

4.4 Test Cases

CHAPTER 5. CONCLUSION

This chapter provides a reflective overview of the Apartment Management System project, evaluating its achievements against the initial objectives, acknowledging inherent limitations, outlining future development trajectories, and summarizing the key lessons learned by the development team.

5.1 Evaluation of Achieved Results

The Apartment Management System successfully addresses the core challenges identified in traditional apartment management practices, delivering on its primary objectives of centralization, automation, and enhanced operational visibility.

Comprehensive Management Centralization

- The system effectively centralizes critical information pertaining to apartments, residents, billing, and service requests, mitigating data fragmentation and significantly improving data accessibility and consistency. This directly resolves the issue of decentralized and inconsistent information management prevalent in existing methods.

Streamlined Financial Tracking

- Robust modules for billing and payment processing have been implemented, enabling efficient bill generation, accurate payment tracking, and real-time monitoring of outstanding dues. This enhancement directly tackles the previous lack of streamlined financial tracking and the challenges of managing outstanding payments.

Optimized Operational Efficiency

- The system provides a unified platform for managing diverse operational workflows, from apartment and resident lifecycle management to service request handling. The introduction of features like paginated data grids, advanced filtering,

instant search, and automated status updates across modules (Apartments, Residents, Payments, Service Requests) significantly boosts operational efficiency and reduces manual errors.

Enhanced Operational Insights

- A centralized dashboard delivers real-time Key Performance Indicators (KPIs) on apartments, residents, revenue, and service requests. The ability to calculate and display month-over-month changes for key metrics provides critical insights for informed decision-making, addressing the prior absence of a comprehensive operational overview.

Scalability and Robustness

- The implementation of a multi-tenant schema mechanism in PostgreSQL allows the system to manage multiple apartment buildings within a single database instance, ensuring data isolation and demonstrating high scalability without requiring significant code restructuring. This fulfills a critical non-functional requirement.

User Experience (UX) Focus

- The UI, built with WPF and adhering to the MVVM pattern, is intuitive, consistent, and designed for efficient task completion. Features like dynamic schema switching and clear validation messages contribute to a user-friendly experience for non-technical staff.

Overall, the project has transitioned fragmented manual processes into a cohesive, automated, and insightful management framework, notably improving transparency, reducing errors, and enhancing service quality for all stakeholders.

5.2 Project Limitations

Despite its successful implementation and adherence to core objectives, the Apartment Management System operates within several defined limitations, primarily due to project scope, resource constraints, and strategic focus:

Limited Accounting Features

- The system does not incorporate advanced accounting functionalities such as general ledger, payroll, or integration with external accounting software. Financial tracking is focused on billing and payments specific to resident dues.

No Direct External System Integration

- The current iteration lacks direct integration with third-party systems. This includes online payment gateways, security systems, smart home devices, or utility company APIs for automated meter reading.

Absence of Resident-Facing Interface

- The system is confined to a standalone desktop application for management use. It does not include a web or mobile interface for residents to directly interact with the system (e.g., viewing bills, submitting requests, accessing personal information).

Standalone Desktop Application:

- The system is designed as a desktop application compatible with Windows 10 and 11. This limits accessibility to devices where the application is installed and precludes remote access via web browsers or mobile devices.

No Real-time Notifications:

- While service requests and payment statuses are tracked, the system does not feature automated real-time notification alerts (e.g., SMS, email) for residents or

management regarding new bills, overdue payments, or service request updates, beyond what is handled by database triggers.

These limitations represent avenues for future development rather than deficiencies in the current scope.

5.3 Future Development Directions

Building upon the solid foundation established by the current implementation, several key areas have been identified for future development to enhance the Apartment Management System's capabilities and extend its reach:

Automated Billing Enhancements

- Implement advanced automation for recurring bill generation (e.g., monthly management fees, utilities based on consumption data), potentially integrating with external meter reading systems.

Integration with Online Payment Gateways

- Develop secure integrations with popular online payment platforms to provide residents with convenient digital payment options, thereby automating payment collection and reconciliation.

Development of a Resident Portal

- Create a resident-facing web or mobile application. This portal would empower residents to view their detailed bills, track payment history, submit and monitor service requests, receive important announcements, and update their personal information directly.

Advanced Reporting and Analytics

- Expand the dashboard and reporting capabilities to include more sophisticated financial analysis, predictive analytics for occupancy trends, and granular reports on service request resolution times and common issues.

Communication Module:

- Integrate features for automated notifications (SMS, email) for critical events such as bill due dates, overdue payments, service request status changes, and general announcements.

User Management and Role Customization

- Provide a more comprehensive user management interface for administrators to define custom roles and permissions, offering granular control over system access and functionalities.

Integration with Smart Building Systems

- Explore potential integrations with smart building technologies (e.g., smart locks, CCTV, environmental sensors) to further automate operations and enhance security.

5.4 Team's Lessons Learned

The development of the Apartment Management System provided invaluable learning experiences for the team, reinforcing the importance of structured methodologies and collaborative practices:

Importance of Comprehensive Requirement Elicitation

- The in-depth stakeholder interviews and resident surveys proved crucial in accurately capturing diverse needs and translating them into precise functional and non-functional requirements. This process highlighted the critical role of user-centric design.

Value of Architectural Planning

- Adhering to the 3-Tier and MVVM architectural patterns significantly contributed to the system's modularity, maintainability, and testability. Early planning around these patterns reduced complexity during implementation and allowed for clear separation of responsibilities among team members.

Power of Database-Level Automation

- Leveraging PostgreSQL functions and triggers for data integrity checks and automated processes proved highly effective. This approach minimized application-level boilerplate code and ensured robust data consistency, even under concurrent operations.

Challenges of Multi-Tenancy

- Implementing the multi-tenant schema mechanism, while powerful, presented unique challenges in ensuring correct context switching and preventing data leakage. It underscored the necessity of meticulous planning and thorough testing for such complex architectural features.

Agile and Iterative Development

- The project benefited from an iterative approach, allowing for continuous feedback and adjustments. This flexibility was crucial in addressing unforeseen issues and refining features as development progressed.

Effective Team Collaboration

- Clear role assignments (e.g., Project Leader, Backend Developer, UI/UX Designer, Tester, Documentation Preparing) and consistent communication were instrumental in successfully navigating the project's complexities and achieving its objectives within the given timeline. The collaborative spirit and commitment of every team member were critical for the project's successful completion.

